Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Ansh Sarfare** of **D20A** semester **VIII,** have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026.**

Subject Teacher
Name:Mrs. Kajal Joseph
Signature:

Head of Department
Name: Dr. Mrs. Shalu Chopra
Signature:

Lab Teacher Name: Kajal Joseph Signature:

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	**Computing & Emerging Technology:** Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	**Professionalism & Innovation :** Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab

CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Ansh Sarfare **D20A - 51**

Experiment -1

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Theory:

1. Cryptographic Hash Function in Blockchain

A cryptographic hash function is a mathematical function that takes an input string of any length and converts it into a fixed-length output string known as a **hash value**. These hash functions are designed to be secure, fast, and irreversible, making them suitable for blockchain applications.

To be cryptographically secure and useful, a hash function must satisfy the following properties:

- **Collision Resistance:**
  It should be computationally difficult to find two different messages m_1 and m_2 such that hash(k, m_1) = hash(k, m_2) where *k* is a key value.

- **Preimage Resistance:**
  Given a hash value *h*, it should be infeasible to determine the original input *m* such that h = hash(k, m). The original data cannot be derived from the hash.

- **Second Preimage Resistance:**
  Given a message m_1, it should be difficult to find another message m_2 such that both produce the same hash.

- **Large Output Space:**
  Hash functions produce a large number of possible outputs, making brute-force collision attacks impractical.

- **Deterministic:**
  For the same input, the hash function must always generate the same output.

- **Avalanche Effect:**
  A very small change in input results in a completely different hash output.

- **Puzzle Friendliness:**
  Even if a portion of the hash is known, it should be impossible to predict the remaining bits.

- **Fixed-Length Mapping:**
  Regardless of input size, the output hash is always of fixed length.

Role of Hash Functions in Blockchain

- Ensures data integrity
- Links blocks securely using hashes
- Used in Merkle Trees
- Used in block headers and digital signatures

2. Merkle Tree (Hash Tree)

A Merkle Tree, also known as a **Hash Tree**, is a data structure used for efficient data verification and synchronization. It is a binary tree in which each non-leaf node is the cryptographic hash of its child nodes.

All leaf nodes are located at the same depth and positioned as far left as possible. The hash at the top of the tree is called the **Merkle Root**, which represents the entire dataset.

Key characteristics:

- Enables efficient handling of large datasets
- Any change in data can be easily detected
- The root hash acts as a fingerprint for the complete data

3. Structure of a Merkle Tree

A Merkle Tree is an **inverted binary tree** constructed from the bottom up using cryptographic hashes. It consists of the following components:

- **Leaf Nodes:**
  The lowest level of the tree. Each leaf node contains the hash of an individual data block or transaction.

- **Internal Nodes:**
  Each internal node is generated by concatenating the hashes of its two child nodes and hashing the combined value.

- **Merkle Root:**
  The topmost node of the tree. It uniquely represents all the data below it. Any modification in the underlying data results in a different Merkle Root, indicating

tampering.

4. Merkle Root

The **Merkle Root** is the final hash at the top of a Merkle Tree. It serves as a concise summary of all transactions within a block. In blockchains like Bitcoin, the Merkle Root is stored in the block header.

Importance of Merkle Root

1. Acts as a unique digital fingerprint of all transactions in a block
2. Stored in the block header instead of the full transaction list
3. Any transaction modification changes the Merkle Root
4. Enables fast and efficient transaction verification
5. Ensures data integrity, security, and immutability

5. Working of Merkle Tree

A Merkle Tree is built in a **bottom-up manner** by repeatedly hashing pairs of nodes until a single hash remains.

Example:
Consider four transactions: T_1, T_2, T_3, T_4

Step 1: Compute individual hashes

- $H_1 = \text{Hash}(T_1)$
- $H_2 = \text{Hash}(T_2)$
- $H_3 = \text{Hash}(T_3)$
- $H_4 = \text{Hash}(T_4)$

Step 2: Form parent nodes

- $H_{12} = \text{Hash}(H_1 + H_2)$
- $H_{34} = \text{Hash}(H_3 + H_4)$

Step 3: Compute Merkle Root

- $H_{1234} = Hash(H_{12} + H_{34})$

The final hash H_{1234} is the **Merkle Root**.

6. Use of Merkle Tree in Blockchain

In distributed blockchain networks, every node maintains its own copy of the ledger. Without Merkle Trees, validating transactions would require transferring and comparing entire ledgers, which is inefficient and resource-intensive.

Merkle Trees solve this problem by:

- Allowing verification using only a small subset of hashes
- Reducing network bandwidth usage
- Eliminating the need to download full transaction data

Thus, Merkle Trees enable scalable, secure, and efficient blockchain operation.

Program and Output:

```
import hashlib

# 1. SHA-256 Hash Function
def generate_hash(message):
    return hashlib.sha256(message.encode()).hexdigest()

user_text = input("Enter a message to hash: ")
hash_result = generate_hash(user_text)
print("Generated Hash:", hash_result)
```

```
Enter a message to hash: Hello Ansh
Generated Hash: c35f50e7afce646b6211b5487fdae273159b01ce3d7cdd864aff00aa86e914c5
```

```python
#2. Execution: Hash with Nonce
def generate_hash_with_nonce(message, nonce):
    combined = message + str(nonce)
    return generate_hash(combined)

nonce_value = int(input("Enter a nonce value: "))
nonce_hash = generate_hash_with_nonce(user_text, nonce_value)
print("Hash with Nonce:", nonce_hash)
```

```
Enter a nonce value: 3
Hash with Nonce: d51352c6698752911f09fff6252c715e6271a5ee599605aeddff69e1659a3c7c
```

```python
# 3. Proof of Work
def proof_of_work(message, difficulty):
    print("\nMining started...")
    prefix = "0" * difficulty
    nonce = 0

    while True:
        new_hash = generate_hash_with_nonce(message, nonce)
        if new_hash.startswith(prefix):
            return nonce, new_hash
        nonce += 1

difficulty_level = int(input("Enter difficulty (number of leading zeros): "))
valid_nonce, valid_hash = proof_of_work(user_text, difficulty_level)

print("Nonce Found:", valid_nonce)
print("Valid Hash:", valid_hash)
```

```
Enter difficulty (number of leading zeros): 4

Mining started...
Nonce Found: 67746
Valid Hash: 00008d13c735d52906bf7e7ab2c8953e1cc80276305bae1d36baaebd506f0a8f
```

```python
# 4. Merkle Tree & Merkle Root
def calculate_merkle_root(transactions):
    hash_list = [generate_hash(tx) for tx in transactions]

    while len(hash_list) > 1:
        if len(hash_list) % 2 != 0:
            hash_list.append(hash_list[-1])
        temp_list = []
        for i in range(0, len(hash_list), 2):
            combined_hash = hash_list[i] + hash_list[i + 1]
            temp_list.append(generate_hash(combined_hash))
        hash_list = temp_list
    return hash_list[0]

print("Enter transactions (type 'stop' to finish):")
transactions = []
while True:
    tx = input()
    if tx.lower() == "stop":
        break
    transactions.append(tx)
if len(transactions) == 0:
    print("⚠️ No transactions entered.")
else:
    root_hash = calculate_merkle_root(transactions)
    print("Merkle Root Hash:")
    print(root_hash)
```

```
Enter transactions (type 'stop' to finish):
1
2
stop
Merkle Root Hash:
33b675636da5dcc86ec847b38c08fa49ff1cace9749931e0a5d4dfdbdedd808a
```

Conclusion

This experiment successfully demonstrated the foundational cryptographic concepts used in blockchain technology. SHA-256 hashing and Proof-of-Work illustrated how data integrity and consensus are achieved, while the Merkle Tree demonstrated efficient and secure verification of large transaction datasets. Together, these mechanisms ensure the security, immutability, and scalability of blockchain systems.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain & DLT Lab

Experiment 02

Aim:Create a Blockchain using Python

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Ansh Sarfare **D20A - 51**

Experiment -2

Aim: Create a Blockchain using Python

Theory:

1. What is Blockchain

Blockchain is an innovative technology that works as a shared, tamper-resistant digital ledger. Its name comes from the way data is structured — information is stored in blocks, each connected to the previous one, forming a continuous chain.

Each block contains important information such as a list of transactions, a timestamp, and a cryptographic hash, which uniquely identifies the block. The hash is generated from both the block's content and the previous block's hash, which links the blocks together securely.

- This linked design ensures that any attempt to alter data becomes immediately noticeable, as it breaks the chain.
- Blockchain serves as a distributed database, storing transaction data across all network participants.
- Transactions are verified collectively by the network, maintaining legitimacy and trust.
- Because it is decentralized, no single entity can manipulate the data.

In a blockchain network, multiple computers (called nodes) hold copies of the ledger. Once a transaction is added and confirmed, it becomes practically immutable, meaning it cannot be changed or deleted without consensus from the network.

2. What is a Block in Blockchain

A block is a digital container that securely stores verified transactions. It is a key building block of the blockchain. Each block contains:

- Transaction Data – details of the transaction such as sender, receiver, and amount.
- Timestamp – records the exact creation time of the block.
- Unique Hash – a cryptographic fingerprint of the block, ensuring its integrity.

- Previous Block Hash – links the block to the one before it, maintaining an unbroken chain.

By storing this information, blocks create a chronological and transparent record of all transactions across the network.

3. Components of a Blockchain Block

Key Components Include:

- Transaction Data: Captures details of who made the transaction, what was exchanged, and when.
- Timestamp: Marks when the block was created, maintaining chronological order.
- Hash: A unique identifier for the block, produced from its contents. Any change in data changes the hash.
- Previous Hash: Connects this block to the preceding one, forming a secure chain.

4. What is Mining

Mining is the process of validating transactions and adding new blocks to the blockchain. It is crucial for network security, decentralization, and transparency.

In cryptocurrencies like Bitcoin, mining uses Proof of Work (PoW), where miners solve complex computational puzzles. Miners compete to validate transactions and append blocks, receiving rewards for success.

Steps in Mining Process

4.1 Transaction Initiation

- Users submit transactions to the blockchain network.
- These transactions are broadcast to all nodes.

4.2 Transaction Pool (Mempool)

- Unconfirmed transactions are collected in a temporary area called the mempool.
- Miners select transactions from this pool to include in new blocks.

4.3 Block Creation

A new block is formed containing:

- Selected transactions
- Hash of the previous block
- Timestamp
- Nonce (a random number used for hashing)

4.4 Hash Generation

- Miners compute a cryptographic hash (e.g., SHA-256) of the block.
- The hash must satisfy the network's difficulty rules, such as starting with a certain number of zeros.

4.5 Proof of Work

- Miners repeatedly adjust the nonce and recompute the hash until a valid hash is found.
- The first miner to find a valid hash proves they have completed the PoW.

4.6 Block Verification

- The mined block is broadcast to all network nodes.
- Nodes verify:
  - Hash correctness
  - Transaction authenticity
  - Proper block structure

4.7 Adding Block to Blockchain

- Once verified, the block is permanently added to the chain.
- The chain remains tamper-proof and synchronized across all nodes.

4.8 Reward Distribution

- The miner who successfully adds the block receives:
    - New coins (block reward)
    - Transaction fees from transactions in the block

5. How to Check Block Validity

Block validity ensures that a block adheres to all blockchain rules before it is accepted. Key checks include:

- Block Structure Verification: Confirm the block has all required elements like header, previous hash, timestamp, nonce, and transactions.
- Previous Hash Validation: Compare the block's previous hash with the hash of the latest block to maintain the chain's integrity.
- Hash and Proof of Work Verification: Recompute the block hash and ensure it meets the network's difficulty conditions.
- Transaction Validation: Ensure every transaction has valid digital signatures, sufficient balance, and no double-spending.
- Merkle Root Verification: Recalculate the Merkle root from all transactions and compare it with the stored root.
- Timestamp Verification: Ensure the block's timestamp is valid and sequential relative to the previous block.
- Block Size and Rule Compliance: Check that the block follows size limits and other protocol rules.
- Consensus Rule Verification: Ensure the block adheres to the network's consensus mechanisms, confirming it is legitimate.

Program:

```
# Module 1 - Create a Simple Blockchain (Fully Commented)

# Required installation:(run cmd as administrator)

# pip install flask==2.2.5

# Importing libraries

import datetime      # To generate timestamps for blocks
```

```
import hashlib       # To generate SHA256 hashes
import json          # To convert block data into JSON
from flask import Flask, jsonify    # To create API endpoints
# Part 1 - Building the Blockchain
class Blockchain:
    def _init_(self):
        # This list will store the entire chain of blocks
        self.chain = []
        # Create the genesis block (first block)
        # proof = 1 → arbitrary
        # previous_hash = '0' → since no previous block exists
        self.create_block(proof=1, previous_hash='0')
    def create_block(self, proof, previous_hash):
        Creates a new block and adds it to the chain.
        Each block contains:
        - index
        - timestamp
        - proof (PoW output)
        -          previous
        block hash block = {
            'index': len(self.chain) + 1,              # Position of block in chain
            'timestamp': str(datetime.datetime.now()), # Current timestamp
            'proof': proof,                            # PoW result
            'previous_hash': previous_hash             # Hash of the previous block
        }
        # Add block to the chain
        self.chain.append(block)
        return block
    def get_previous_block(self):
```

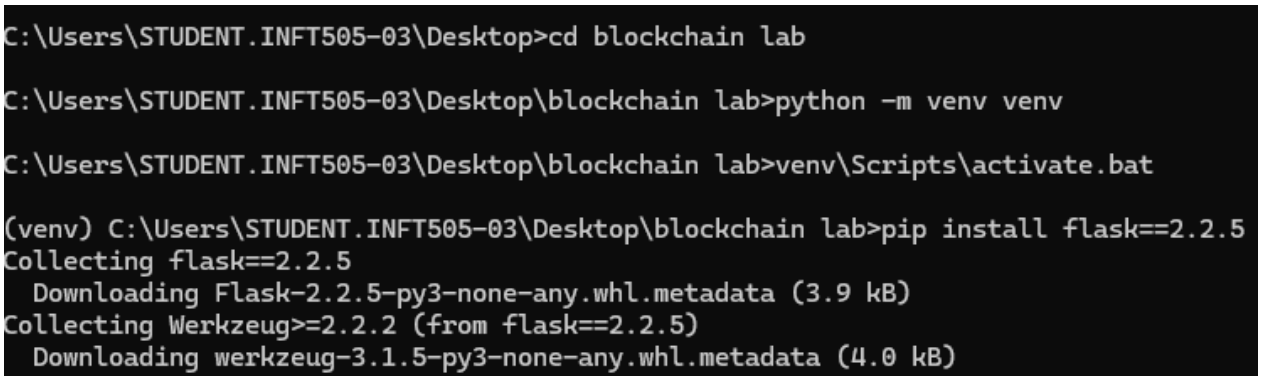
```
    Return the last block in the chain.
    return self.chain[-1]
def proof_of_work(self, previous_proof):
    Simple Proof of Work (PoW) algorithm:
    - Find a number (new_proof)
    - Such that the SHA256 hash of (new_proof^2 - previous_proof^2)
      starts with '0000'
    This is a computational puzzle to secure the network.
    new_proof = 1
    check_proof = False

    while check_proof is False:
        # Cryptographic puzzle: hash difference of squares
        hash_operation = hashlib.sha256(
            str(new_proof**2 - previous_proof**2).encode()
        ).hexdigest()
        # Check if hash begins with 4 leading zeros
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof
def hash(self, block):
        Creates a SHA256 hash of a block.
    -           Sort keys to ensure consistent
    hashing """
    encoded_block = json.dumps(block, sort_keys=True).encode()
    return hashlib.sha256(encoded_block).hexdigest()
def is_chain_valid(self, chain):
```

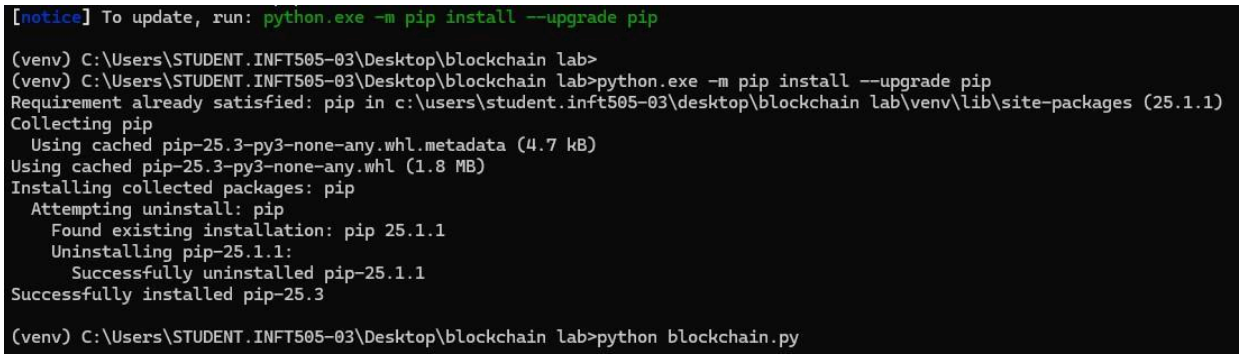
```
        """
        Validates the blockchain by checking:
        1. The previous_hash field matches the actual hash of the previous block.
        2.      The PoW condition (hash starting with '0000') is satisfied for each
        block. """
        previous_block = chain[0] # Genesis block
        block_index = 1             # Start checking from block 2
        while block_index < len(chain):
            block = chain[block_index]
            # Check previous hash correctness
            if block['previous_hash'] != self.hash(previous_block):
                return False
            # Validate Proof of Work
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(
                str(proof**2 - previous_proof**2).encode()
            ).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            # Move to next block
            previous_block = block
            block_index += 1
        return True
# Part 2 - Mining our Blockchain (Flask API)
# Initialize Flask web application
app = Flask(_name_)


# Create an instance of the Blockchain
```

```
blockchain = Blockchain()
# API Endpoint: Mine a new block
@app.route('/mine_block', methods=['GET'])
def mine_block():
    # Get the previous block
    previous_block = blockchain.get_previous_block()
        # Extract previous proof
    previous_proof = previous_block['proof']
        # Run Proof of Work algorithm
    proof = blockchain.proof_of_work(previous_proof)
        # Get hash of previous block
    previous_hash = blockchain.hash(previous_block)
        # Create the new block
    block = blockchain.create_block(proof, previous_hash)
    # Prepare response
    response = {
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash']
    }
    return jsonify(response), 200
# API Endpoint: Get the full blockchain
@app.route('/get_chain', methods=['GET'])
def get_chain():
    response = {
        'chain': blockchain.chain,
        'length':
        len(blockchain.chain)
```

```
    return jsonify(response), 200
# API Endpoint: Check if blockchain is
valid @app.route('/is_valid',
methods=['GET']) def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200
# Run the Flask app
app.run(host='0.0.0.0', port=5000)
```

Output:





```
(venv) C:\Users\STUDENT.INFT505-03\Desktop\blockchain lab>python blockchain.py
 * Serving Flask app 'blockchain'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://192.168.32.230:5001
Press CTRL+C to quit
127.0.0.1 - - [30/Jan/2026 12:47:19] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:47:39] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:47:44] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:47:47] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:04] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:09] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:11] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [30/Jan/2026 12:48:25] "GET /is_valid HTTP/1.1" 200 -
```

127.0.0.1:5001/mine_block

Pretty-print


```
{
  "index": 4,
  "message": "Ansh, you just mined a block!",
  "previous_hash": "d0863156188d5c027df2bd30b848824245c3ce19176588e2e6debbb2df65450d",
  "proof": 21391,
  "timestamp": "2026-01-30 12:48:09.371936"
}
```


127.0.0.1:5001/get_chain

Pretty-print


```
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-01-30 12:47:07.313020"
    },
    {
      "index": 2,
      "previous_hash": "5ab6acdb01cd2d0c7bcb1ca286b4d820348d71c7db288bd6f1aeb7f4a80bfc29",
      "proof": 533,
      "timestamp": "2026-01-30 12:47:19.626099"
    },
    {
      "index": 3,
      "previous_hash": "12f847513780c8368e276efff21a4e98d611d32e9d52f03796a2f036534d4c5d",
      "proof": 45293,
      "timestamp": "2026-01-30 12:47:44.855424"
    },
    {
      "index": 4,
      "previous_hash": "d0863156188d5c027df2bd30b848824245c3ce19176588e2e6debbb2df65450d",
      "proof": 21391,
      "timestamp": "2026-01-30 12:48:09.371936"
    }
  ],
  "length": 4
}
```

Conclusion

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles decentralization, immutability, and consensus in a simplified environment, providing a strong foundation for building more advanced blockchain systems.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Ansh Sarfare **D20A - 51**

Blockchain Exp - 3

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

Blockchain is a **distributed and decentralized ledger** that stores information in a series of linked blocks.

Each block contains:

- Transaction data
- Timestamp
- Previous block's hash
- Its own unique hash (digital fingerprint)

Once data is recorded in a blockchain, it becomes **immutable** because altering one block would require recalculating all subsequent blocks.

2. Mining

Mining is the process of:

1. Collecting pending transactions into a block.
2. Performing a computational puzzle (Proof-of-Work) to find a valid hash.
3. Adding the new block to the blockchain.
   Broadcasting it to all connected peers.

Miners are rewarded with cryptocurrency for successfully mining a block.

3. Multi-Node Blockchain Network

In this lab, we simulate **three independent blockchain nodes** (5001, 5002, 5003).

Each node:

- Runs on a separate port.
- Maintains its own copy of the blockchain.
- Can connect with peers to share and validate blocks.

4. Consensus Mechanism

We use the **Longest Chain Rule**:

- If multiple versions of the chain exist, the **longest valid chain** is chosen.
- This ensures all nodes agree on a single transaction history.

5. Transactions & Mining Reward

Each transaction has:

- Sender
- Receiver
- Amount

When mining a block:

- Pending transactions are added to the block.
- A **reward transaction** is added automatically to pay the miner.

6. Chain Replacement

When /replace_chain is called:

1. Node requests chains from peers.
2. If it finds a longer and valid chain, it replaces its own.
3. This keeps the blockchain consistent across all nodes.

Tools & Libraries Used

- **Python 3.x**
- **Flask** – Web framework for API endpoints pip install Flask
- **Requests** – For HTTP communication between nodes pip install requests==2.18.4
- **Postman** – For testing API requests
- Python Standard Libraries:
    - datetime
    - jsonify
    - hashlib
    - uuid4
    - urlparse
    - request

Code :

```
# Module 2 - Create a Cryptocurrency

# To be installed:
# Flask==0.12.2: pip install Flask==0.12.2
# Postman HTTP Client: https://www.getpostman.com/
# requests==2.18.4: pip install requests==2.18.4

# Importing the
libraries import
datetime
import hashlib
import json
from flask import Flask, jsonify, request
import requests
from uuid import uuid4                                  # Generate a unique id that is in
hex from urllib.parse import urlparse                   # To parse url of the nodes

# Part 1 - Building a Blockchain
class Blockchain:
    def _init_(self):
        self.chain = []
        self.transactions = []                          # Adding transactions before they
are added to a block
        self.create_block(proof = 1, previous_hash = '0')
        self.nodes = set()                              # Set is used as there is no order to
be maintained as the nodes can be from all around the globe

    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash,
                 'transactions': self.transactions}     # Adding transactions to make
the blockchain a cryptocurrency
        self.transactions = []                          # The list of transacction should
become empty after they are added to a block
        self.chain.append(block)
        return block
```

```
    def get_previous_block(self):
        return self.chain[-1]

    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof

    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()

    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 -
            previous_proof**2).encode()).hexdigest() if hash_operation[:4] != '0000':
                return False
            previous_block = block
            block_index += 1
        return True
```

- This method will add the trqnsaction to the list of

```
    trnsactions def add_transaction(self, sender, receiver, amount):
        self.transactions.append({'sender': sender,
                                  'receiver': receiver,
                                  'amount': amount})
        previous_block = self.get_previous_block()
```

```
        return previous_block['index'] + 1                                  # It will return the block index
to which the transaction should be added
```

- This function will add the node containing an address to the set of nodes created in init function

```
    def add_node(self, address):
        parsed_url = urlparse(address)                                      # urlparse will parse the url from
the address
        self.nodes.add(parsed_url.netloc)                                   # Add is used and not append as
it's a set. Netloc will only return '127.0.0.1:5000'
```

- Consensus Protocol. This function will replace all the shorter chain with the longer chain in all the nodes on the network

```
    def replace_chain(self):
        network = self.nodes                                                # network variable is the set of nodes
all around the globe
        longest_chain = None                                                # It will hold the longest chain when
we scan the network
        max_length = len(self.chain)                                        # This will hold the length of the
chain held by the node that runs this function
        for node in network:
            response = requests.get(f'http://{node}/get_chain')             # Use get chain
method already created to get the length of the chain
            if response.status_code == 200:
                length = response.json()['length']                          # Extract the length of the chain
from get_chain fiunction
                chain = response.json()['chain']
                if length > max_length and self.is_chain_valid(chain):      # We check if the length
is bigger and if the chain is valid then
                    max_length = length                                     # We update the max length
                    longest_chain = chain                                   # We update the longest chain
        if longest_chain:                                                   # If longest_chain is not none that means
it was replaced
            self.chain = longest_chain                                      # Replace the chain of the current
node with the longest chain
            return True
        return False                                                        # Return false if current chain is the longest
one
```

```
# Part 2 - Mining our Blockchain
```

```
# Creating a Web App
```

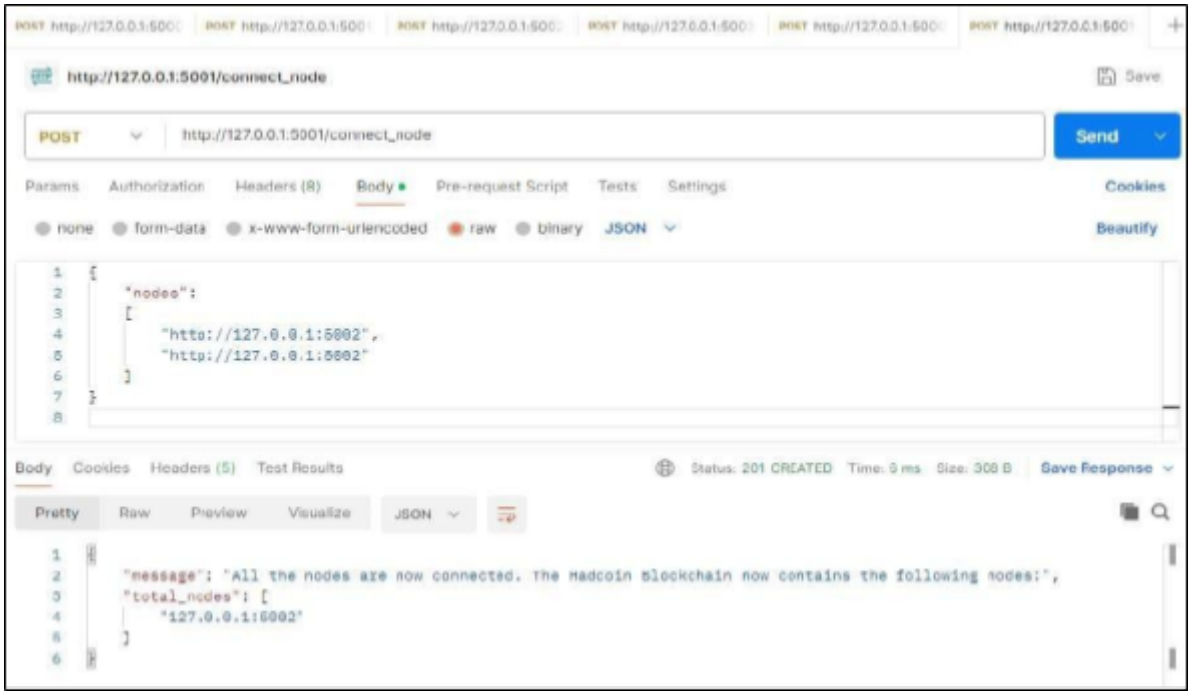
```
app = Flask(_name_)

 # Creating an address for the node on Port 5000. We will create some other nodes as well on different ports
node_address = str(uuid4()).replace('-', '')                                #

 # Creating a Blockchain
blockchain = Blockchain()

 # Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(sender = node_address, receiver = 'Richard', amount = 1) # Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the block
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash': block['previous_hash'],
                'transactions': block['transactions']}
    return jsonify(response), 200

 # Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return jsonify(response), 200

 # Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}
    else:
```

```
        response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200

# Adding a new transaction to the Blockchain
@app.route('/add_transaction', methods = ['POST'])                    # Post method as we
have to pass something to get something in return
def add_transaction():
    json = request.get_json()                                          # This will get the json file from
postman. In Postman we will create a json file in which we will pass the values for the keys in
the json file
    transaction_keys = ['sender', 'receiver', 'amount']
    if not all(key in json for key in transaction_keys):               # Checking if all keys
are available in json
        return 'Some elements of the transaction are missing', 400
    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])
    response = {'message': f'This transaction will be added to Block {index}'}
    return jsonify(response), 201                                      # Code 201 for creation

# Part 3 - Decentralizing our Blockchain

# Connecting new nodes
@app.route('/connect_node', methods = ['POST'])                        # POST request to
register the new nodes from the json file
def connect_node():
    json = request.get_json()
    nodes = json.get('nodes')                                          # Get the nodes from json
    file if nodes is None:
        return "No node", 400
    for node in nodes:
        blockchain.add_node(node)
    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now
contains the following nodes:',
                'total_nodes': list(blockchain.nodes)}
    return jsonify(response), 201

# Replacing the chain by the longest chain if needed
@app.route('/replace_chain', methods = ['GET'])
def replace_chain():
    is_chain_replaced = blockchain.replace_chain()
    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the
longest one.',
```

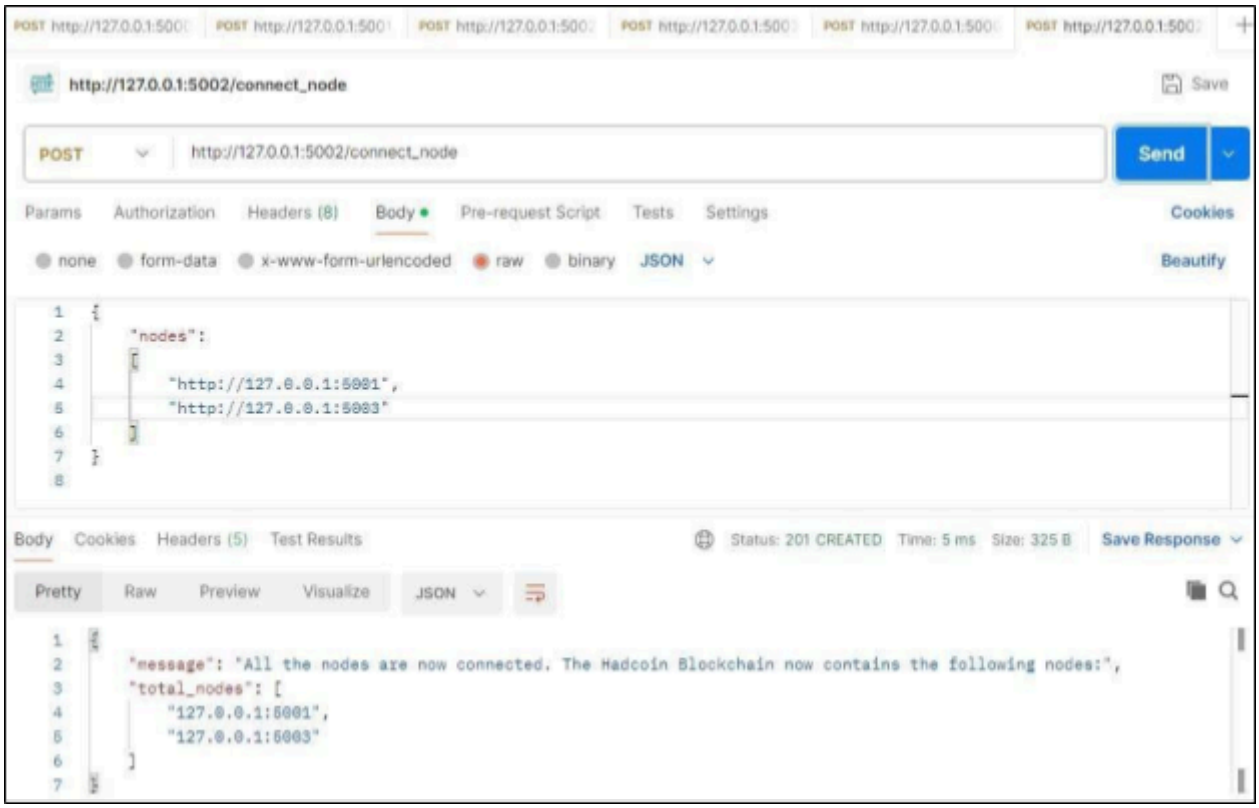
```
            'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
            'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 500*)
```

Output :

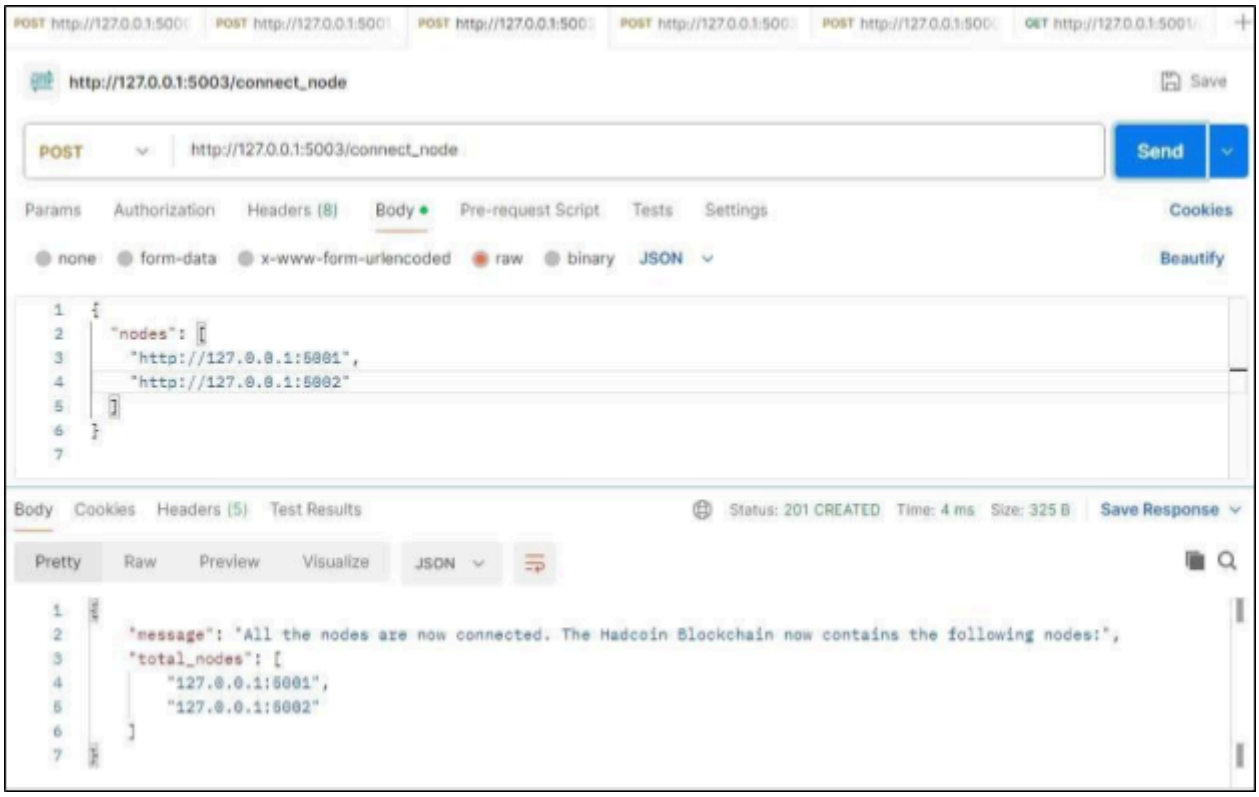
1. **Connecting nodes: Node 1 -> 2,3**



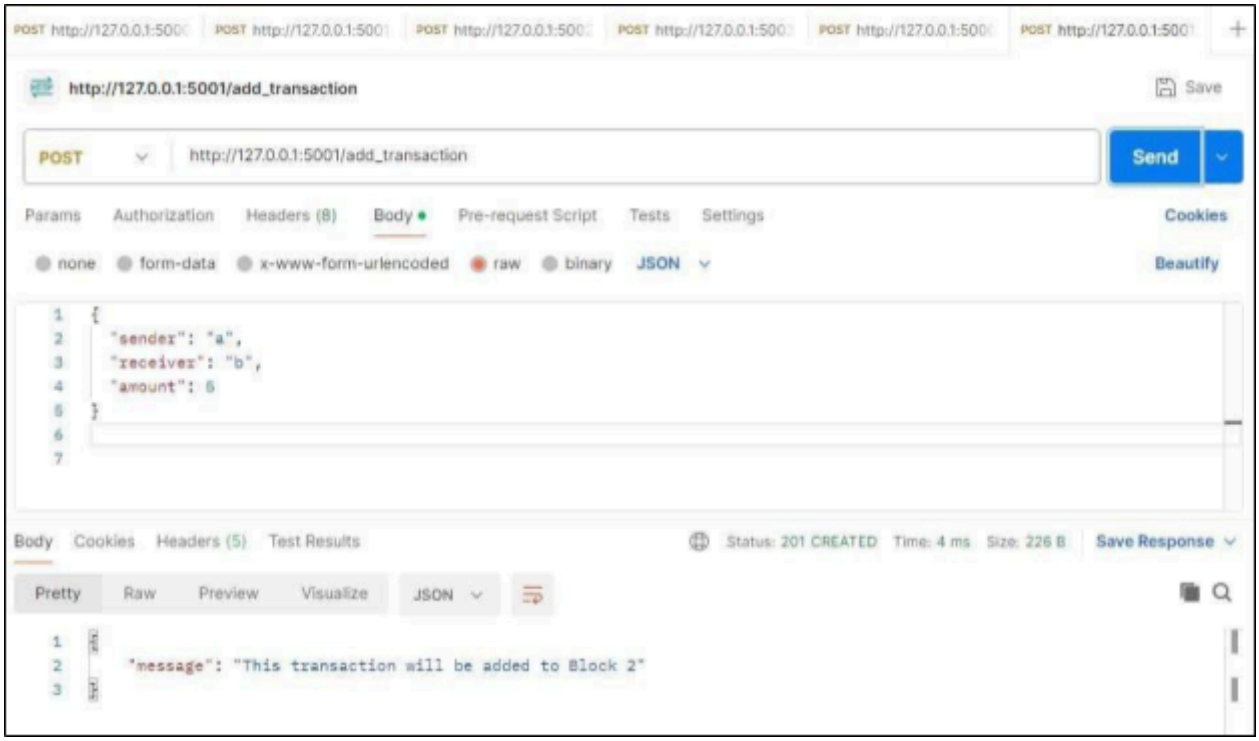
Node 2 -> 1,3



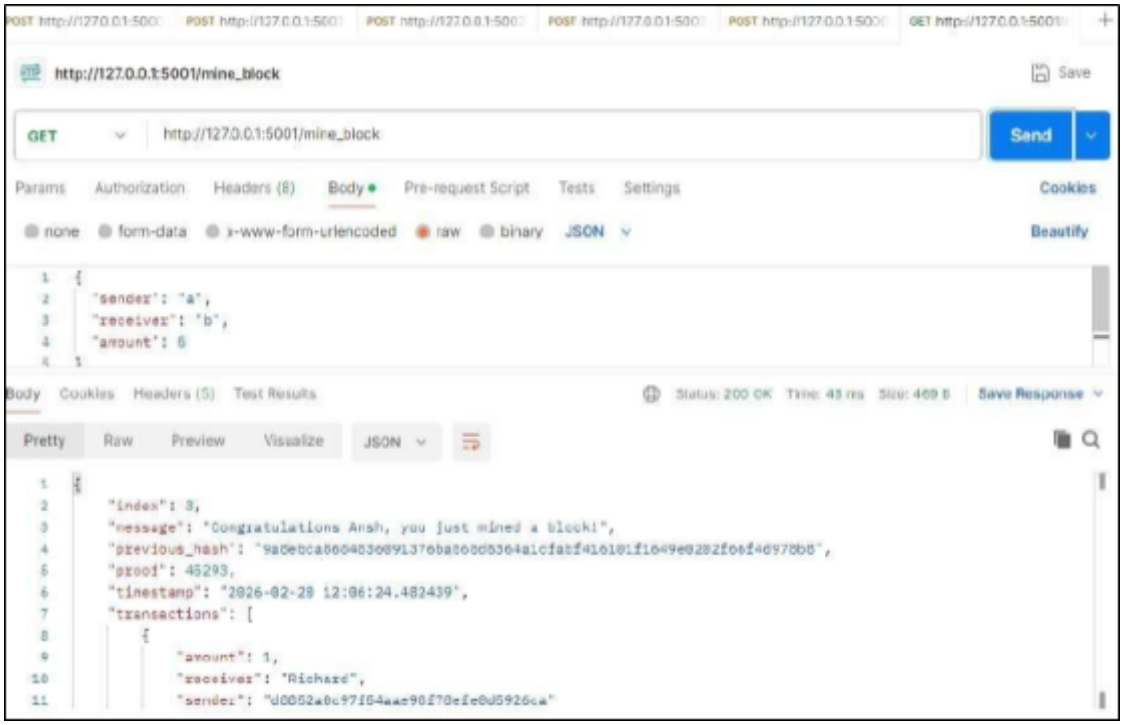
Node 3 -> 1,2



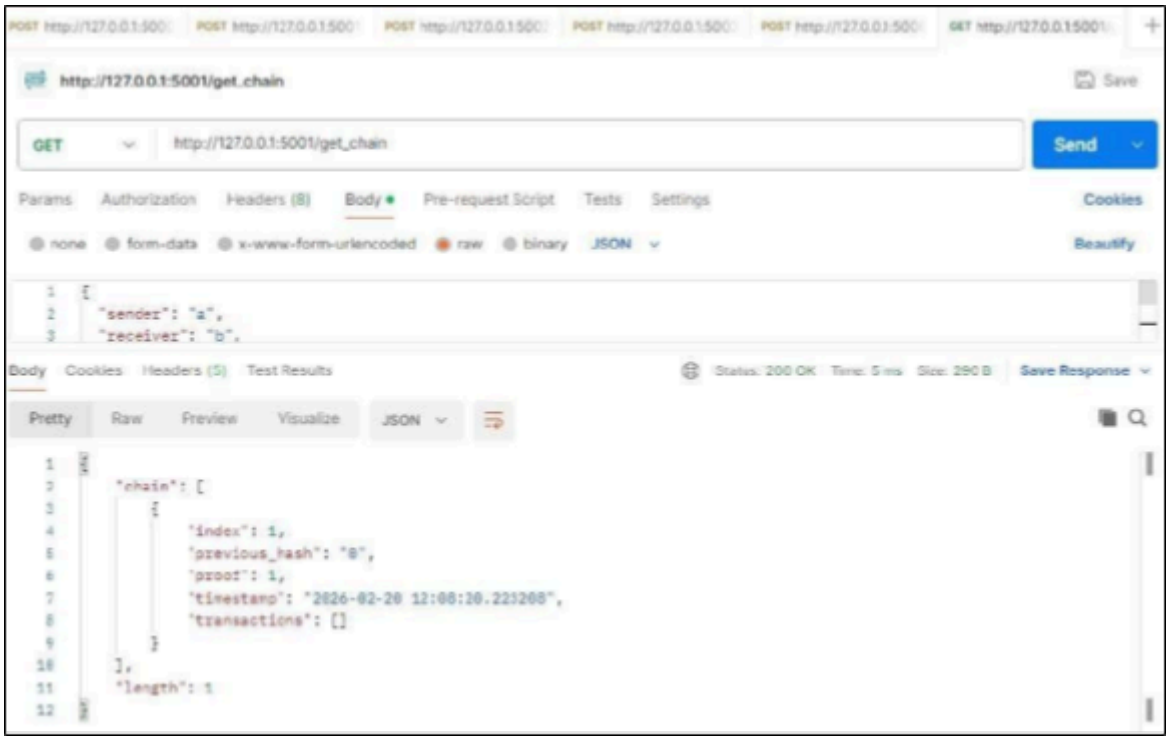
2. **Adding transaction from node 1**



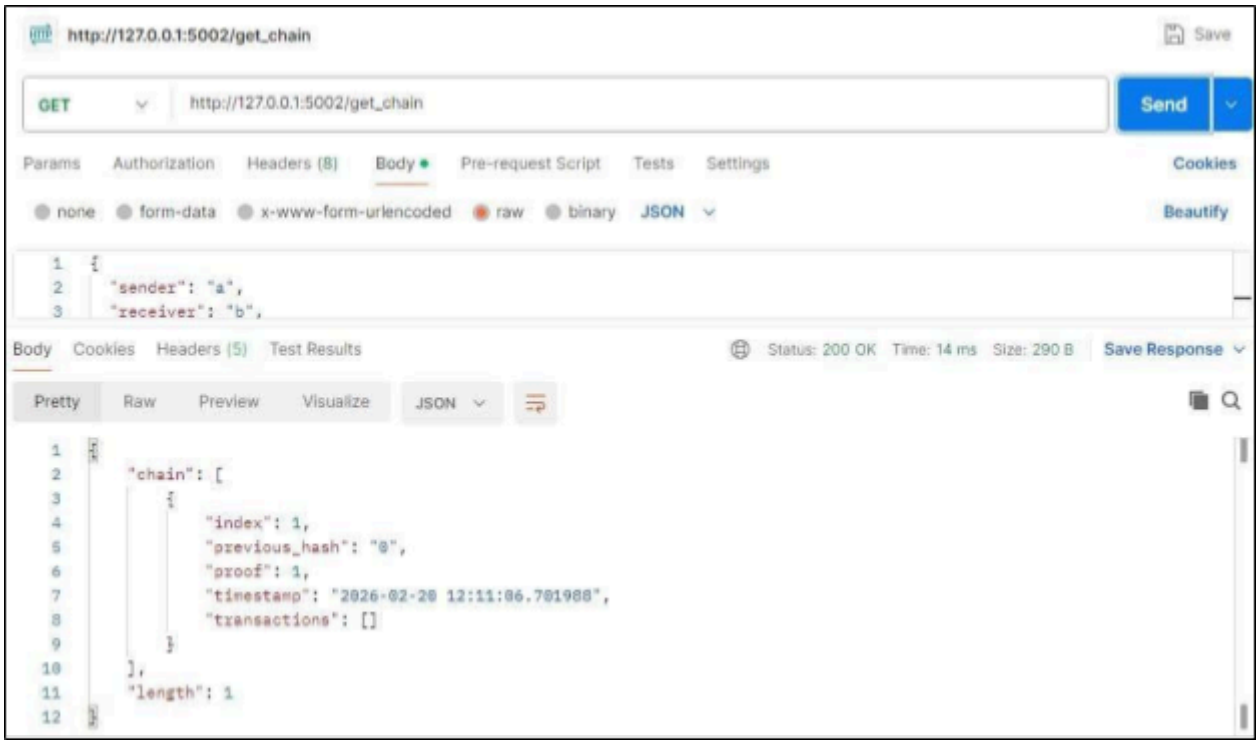
3. **Mining a Block (Only on 5001)**



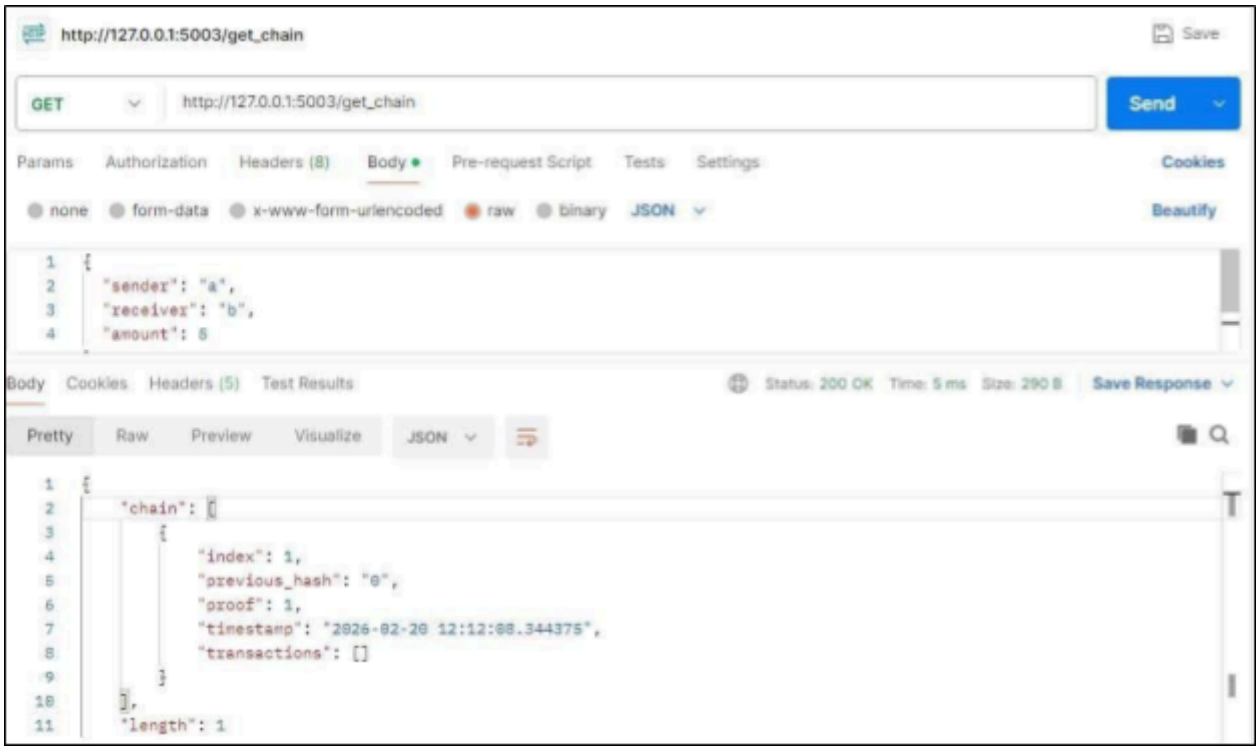
4. **Checking chain length difference: Node 1**



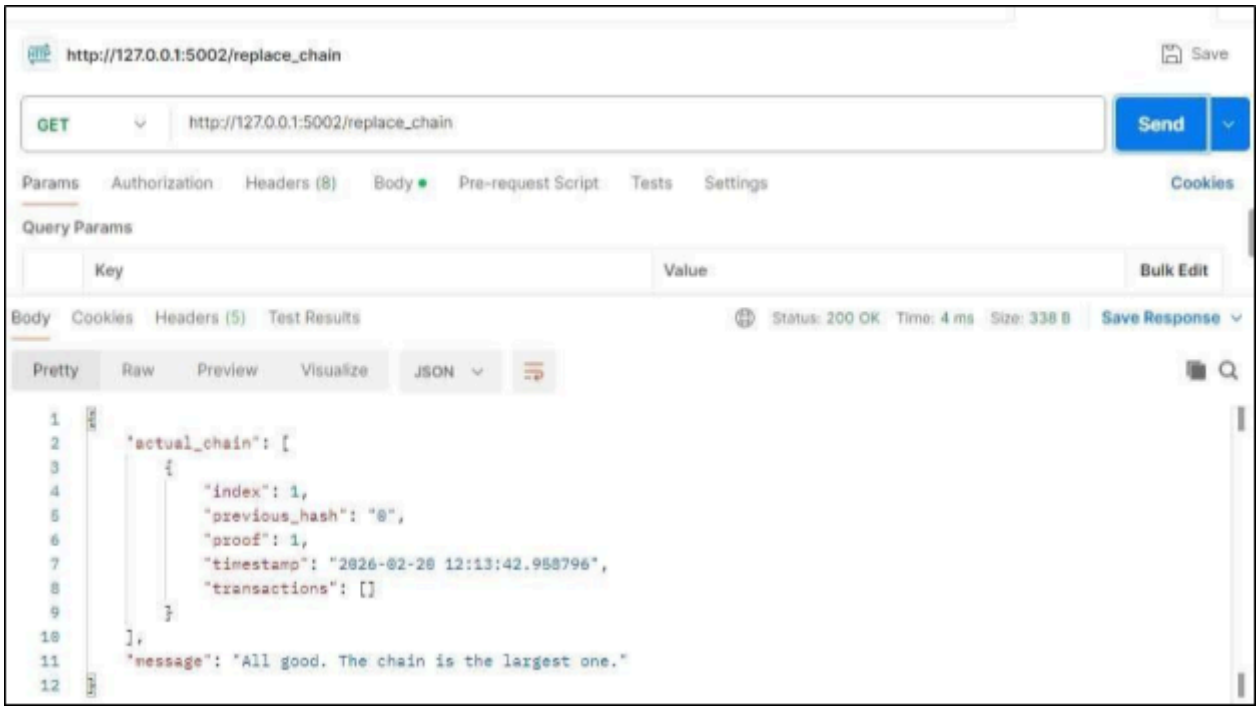
Node 2


http://127.0.0.1:5002/get_chain
Save
GET
http://127.0.0.1:5002/get_chain
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"sender": "a",
"receiver": "b",
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 14 ms
Size: 290 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-02-20 12:11:06.701988",
"transactions": []
}
],
"length": 1
}

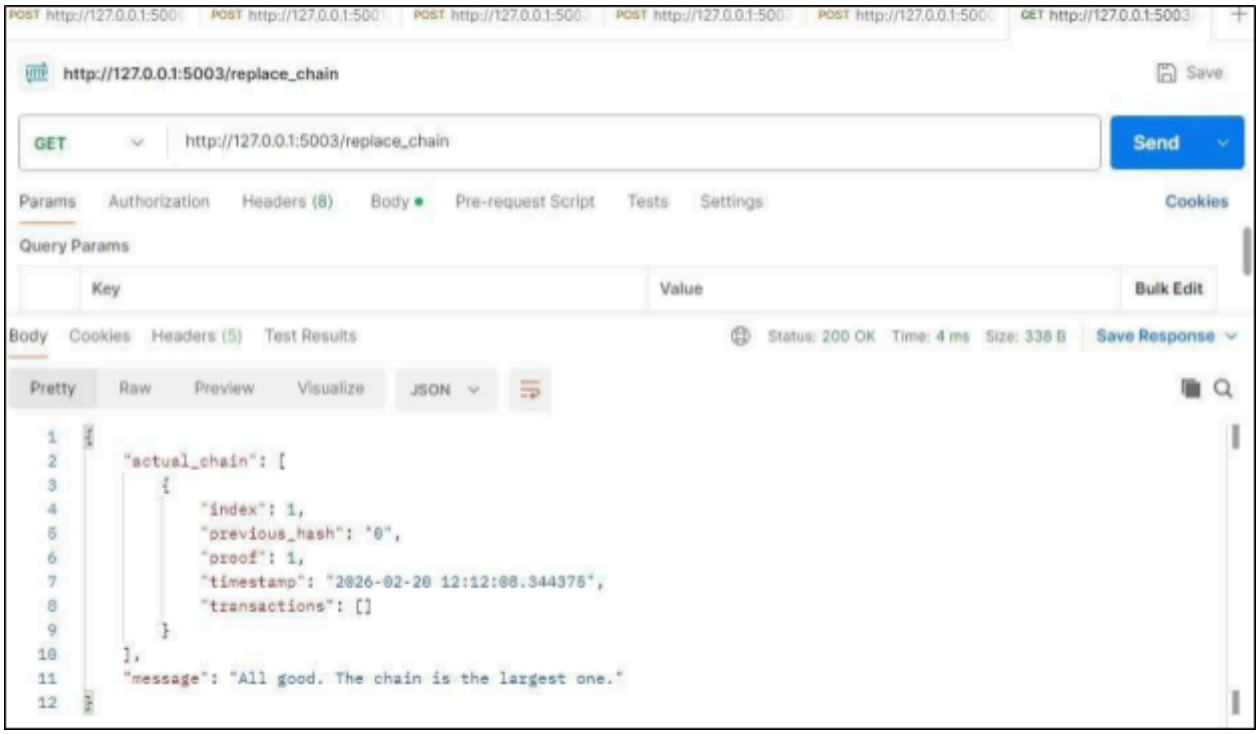
Node 3


http://127.0.0.1:5003/get_chain
Save
GET
http://127.0.0.1:5003/get_chain
Send
Params
Authorization
Headers (8)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"sender": "a",
"receiver": "b",
"amount": 5
Body
Cookies
Headers (5)
Test Results
Status: 200 OK
Time: 5 ms
Size: 290 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-02-20 12:12:08.344375",
"transactions": []
}
],
"length": 1

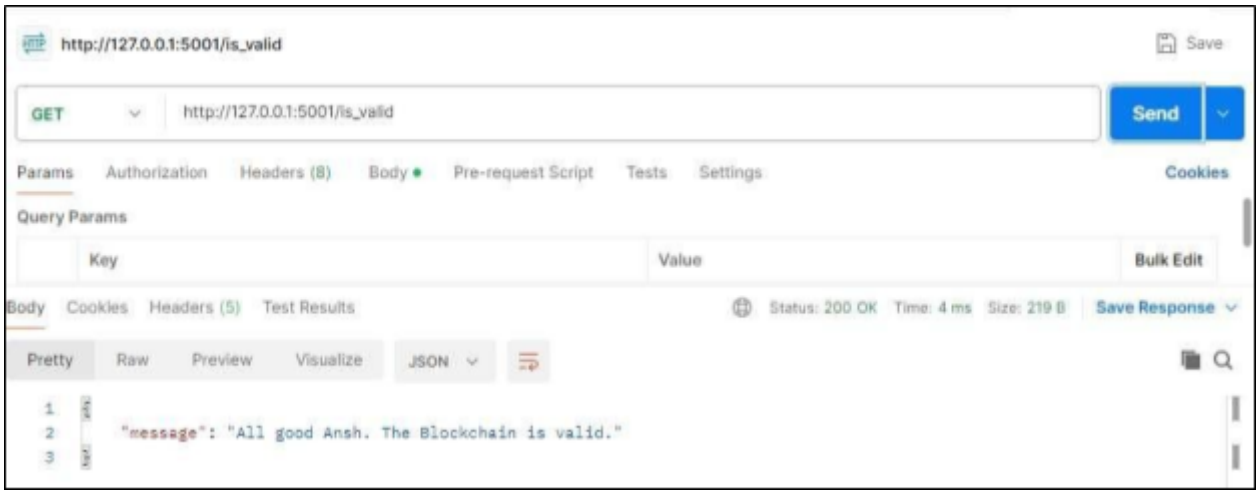
5. **Replace Chain (Consensus Mechanism) : Node 2**



Node 3



6. **Final Validation**



7. **Terminal Activity**

```
(venv) PS C:\Users\Student\Documents\Lab_3_Create a Cryptocurrency> python hadcoin_node_5003.py
WARNING: This is a development server. Do not use it in a production deployment. Use a production
 WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5003
 * Running on http://192.168.36.127:5003
Press CTRL+C to quit
127.0.0.1 - - [20/Feb/2026 12:12:15] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [20/Feb/2026 12:13:06] "GET /replace_chain HTTP/1.1" 200 -
(venv) PS C:\Users\Student\Documents\Lab_3_Create a Cryptocurrency> python hadcoin_node_5002.py
 * Serving Flask app 'hadcoin_node_5002'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
 WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5002
 * Running on http://192.168.36.127:5002
Press CTRL+C to quit
127.0.0.1 - - [20/Feb/2026 12:13:48] "GET /replace_chain HTTP/1.1" 200 -
(venv) PS C:\Users\Student\Documents\Lab_3_Create a Cryptocurrency> python hadcoin_node_5001.py
 * Serving Flask app 'hadcoin_node_5001'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production
 WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5001
 * Running on http://192.168.36.127:5001
```

Conclusion: In this experiment, we successfully created a basic cryptocurrency using Python and Flask and implemented mining in a multi-node blockchain network. We demonstrated how transactions are added, stored temporarily, and included in a block during the mining process using a Proof-of-Work mechanism. By running three separate nodes, we simulated a peer-to-peer network and applied the Longest Chain Rule to maintain consensus across all nodes. The experiment helped us understand blockchain structure, decentralized networking, mining rewards, and chain synchronization in a practical manner.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Ansh Sarfare **D20A - 51**

Blockchain Exp - 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory:

1. **Primitive Data Types, Variables, Functions – pure, view**

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int**: unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool**: represents logical values (true or false).
- **address**: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string**: store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure**: cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view**: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. **Inputs and Outputs to Functions**

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
    - public: available both inside and outside the contract.
    - private: only accessible within the same contract.
    - internal: accessible within the contract and its child contracts.
    - external: can be called only by external accounts or other contracts.

- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price**: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions**: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

Implementation:

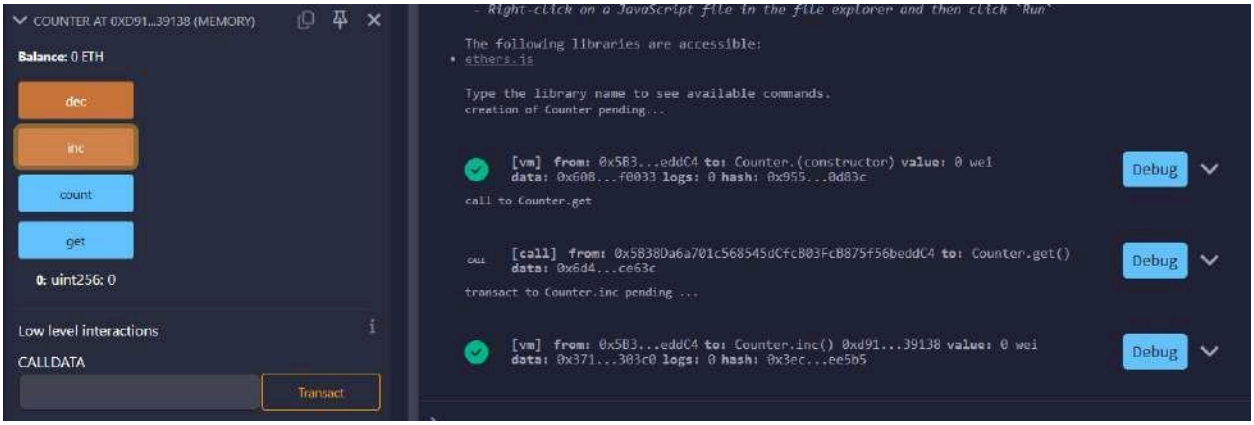
Tutorial - 1 (compile)

```
1   // SPDX-License-Identifier: MIT
2   pragma solidity ^0.8.3;
3   //Ansh Sarfare D20A51
4   contract Counter {
5       uint public count;
6
7       // Function to get the current count
8       function get() public view returns (uint) {    2453 gas
9           return count;
10      }
11
12      // Function to increment count by 1
13      function inc() public {    infinite gas
14          count += 1;
15      }
16
17      // Function to decrement count by 1
18      function dec() public {    infinite gas
19          count -= 1;
20      }
21  }
```

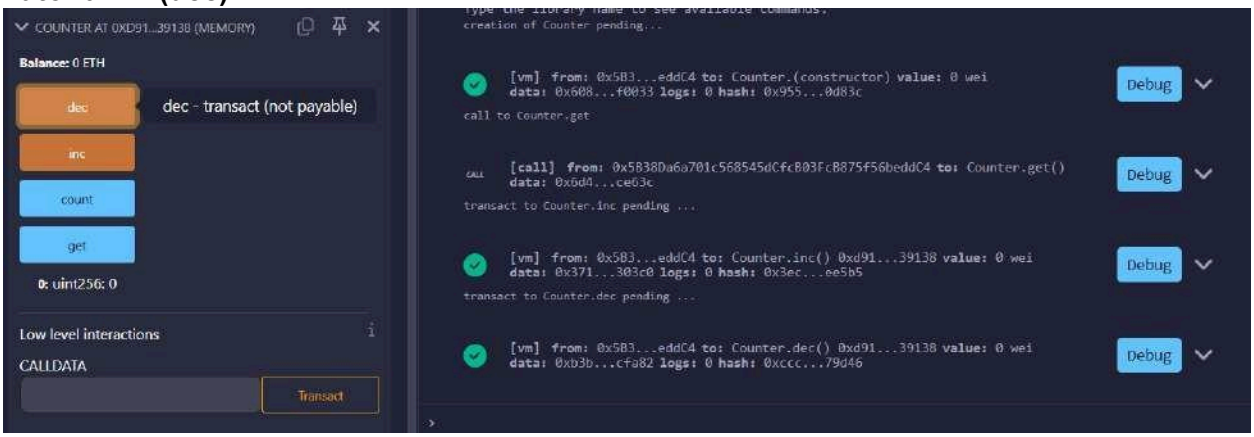
```
Welcome to Remix 1.5.1

Your files are stored in indexedDB, 2.65 KB / 108.86 GB used

You can use this terminal to:
• Check transactions details and start debugging.
• Execute JavaScript scripts:
  - Input a script directly in the command line interface
  - Select a Javascript file in the file explorer and then run `remix.execute()` or
`remix.exeCurrent()` in the command line interface
  - Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:
• ethers.js

Type the library name to see available commands.
creation of Counter pending...

[vm] from: 0x5B3...eddC4 to: Counter.(constructor) value: 0 wei
data: 0x608...f0033 logs: 0 hash: 0x955...0d83c                    Debug
```

Tutorial - 1 (get)

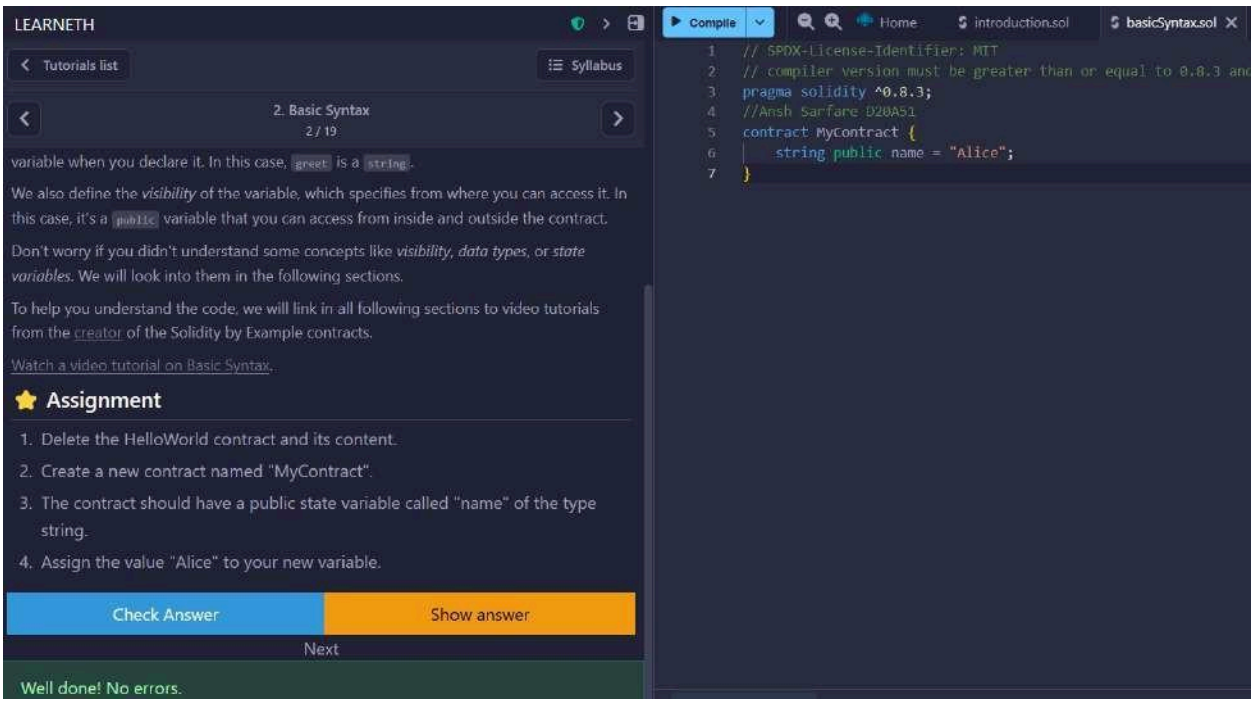
Tutorial - 1 (inc)



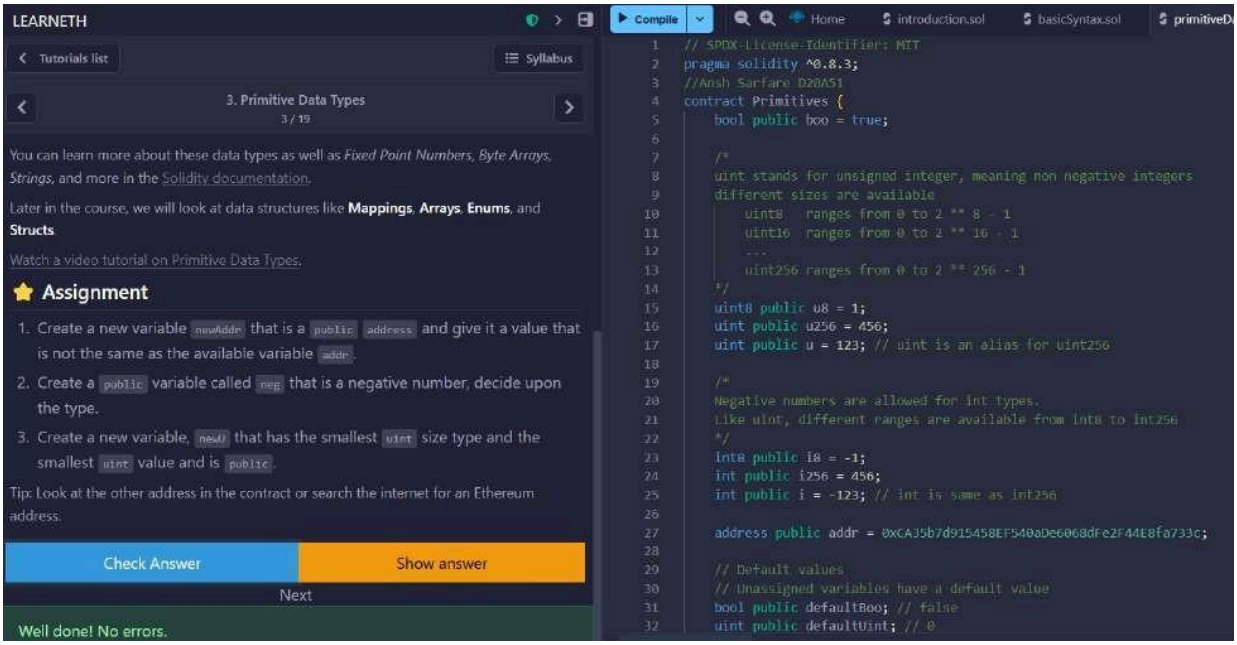
Tutorial - 1 (dec)



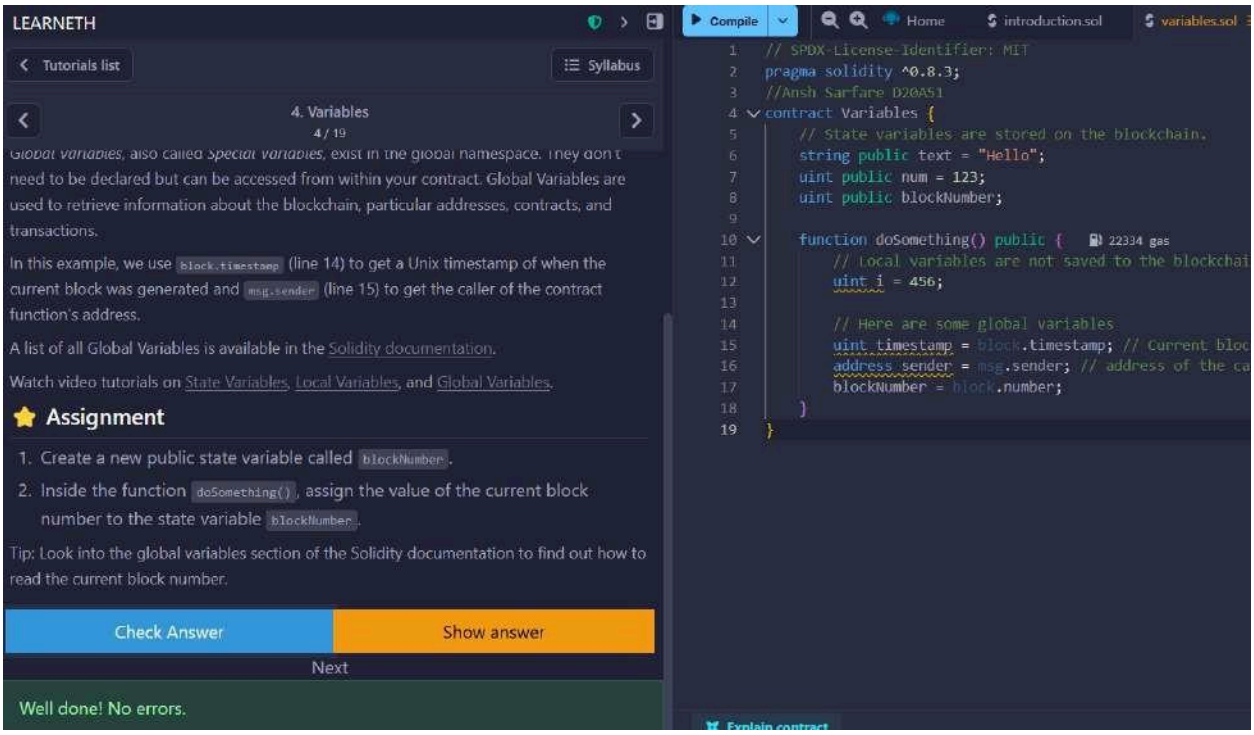
Tutorial - 2



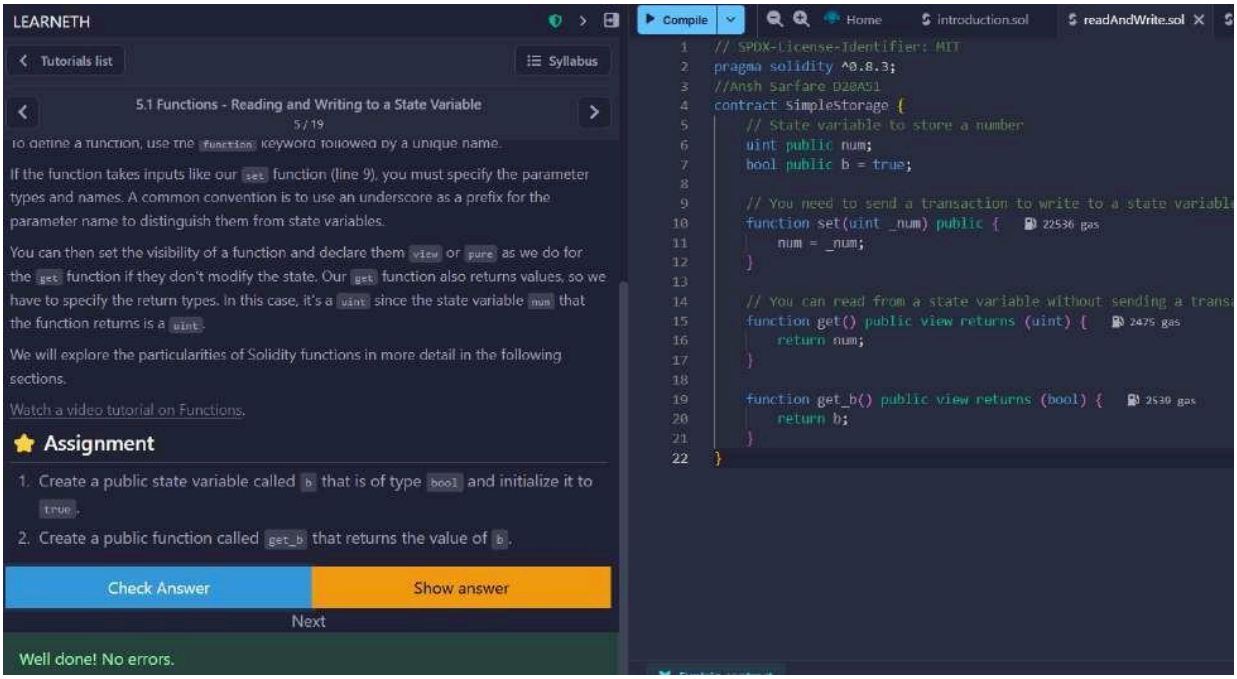
Tutorial - 3



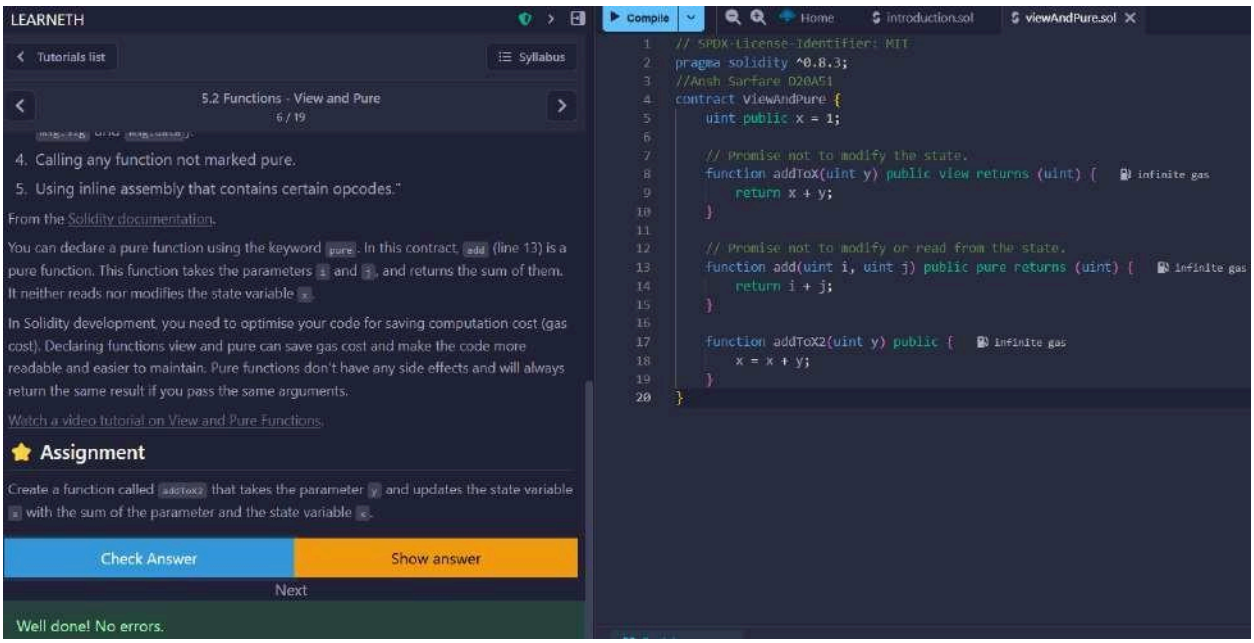
Tutorial - 4



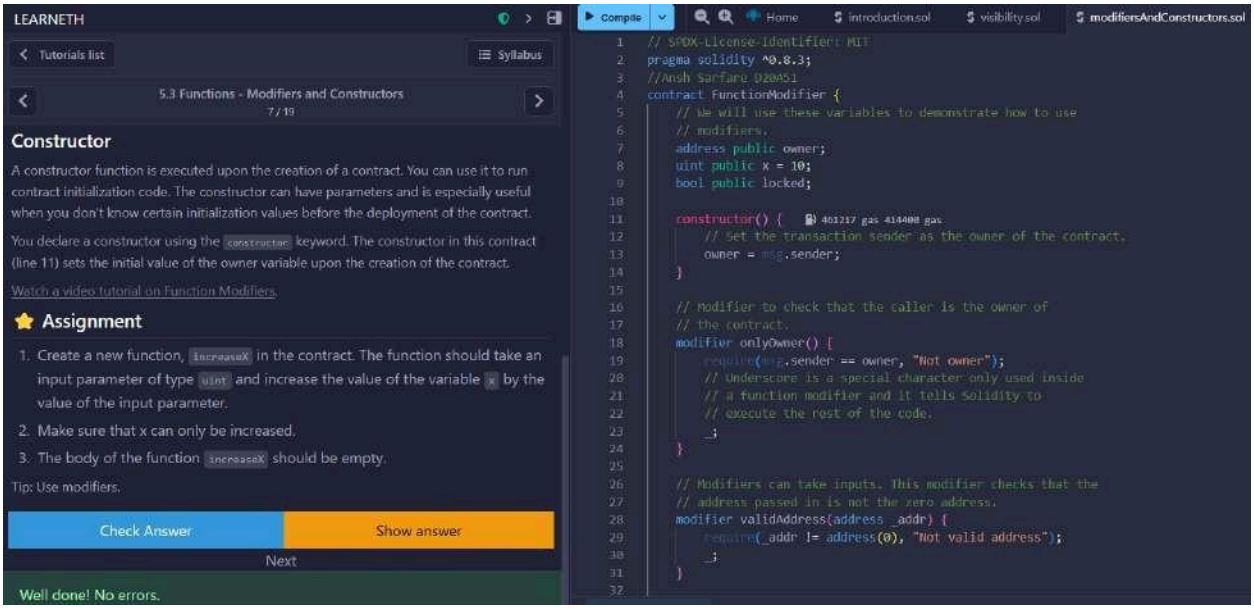
Tutorial - 5



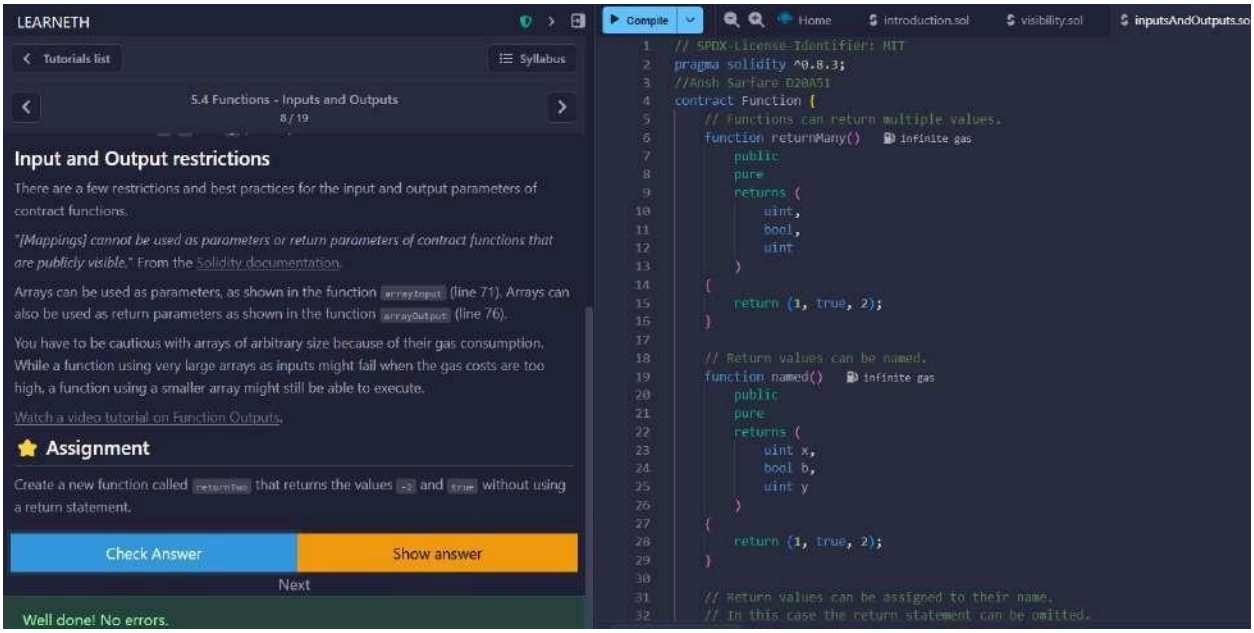
Tutorial - 6



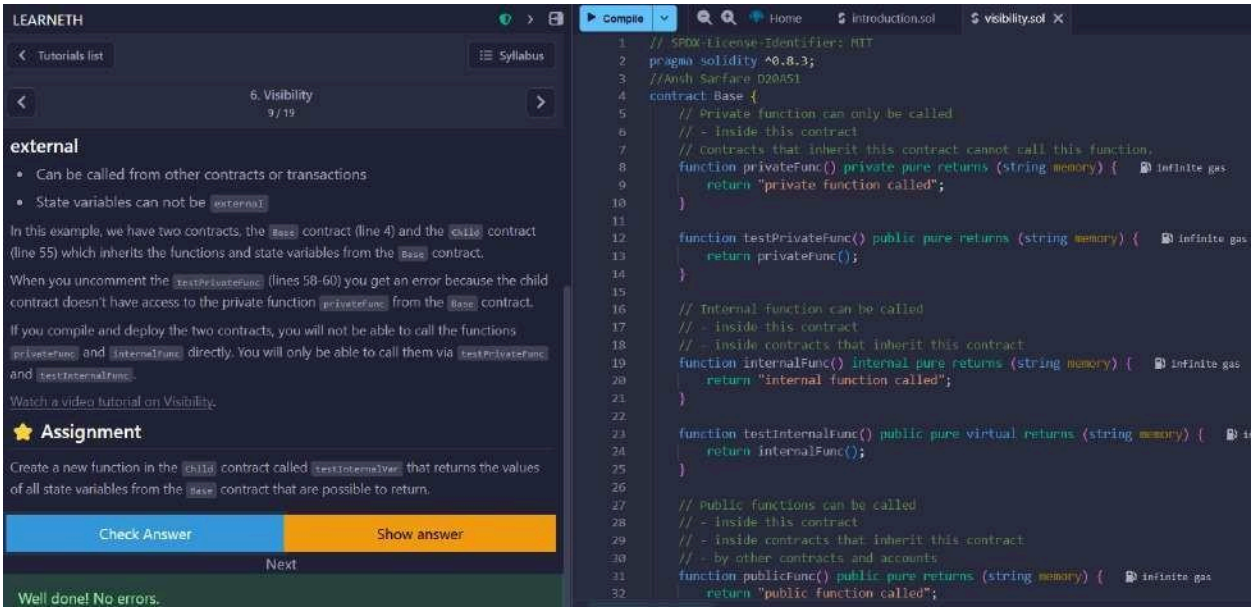
Tutorial - 7



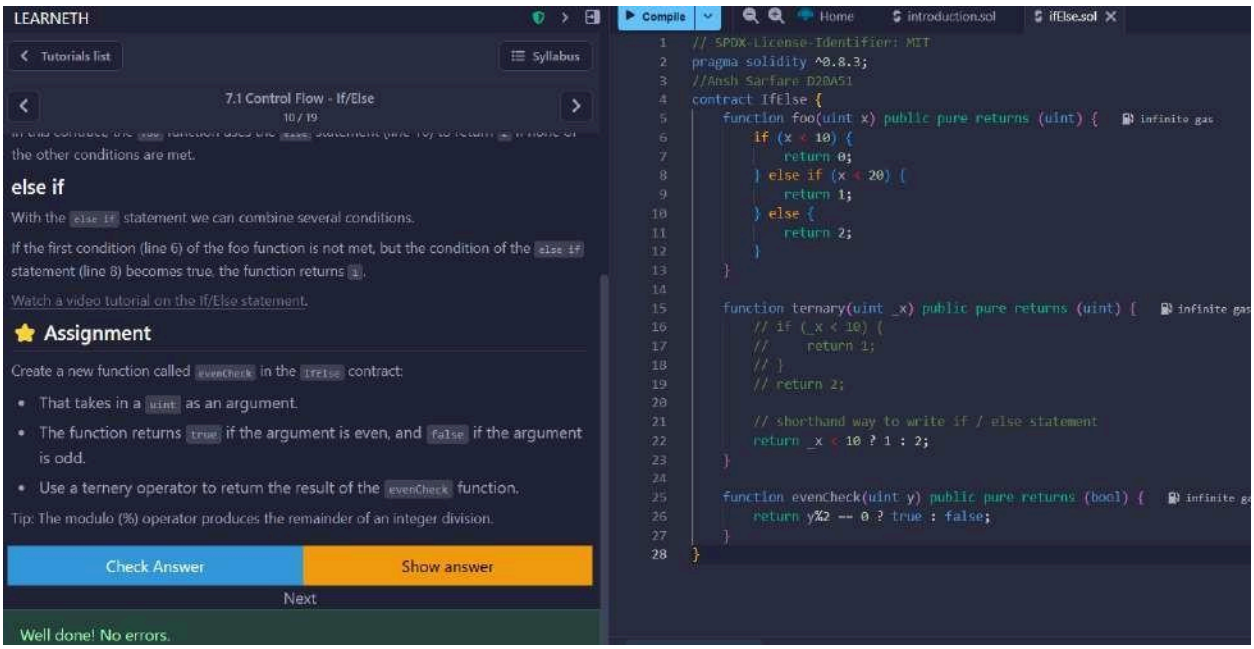
Tutorial - 8



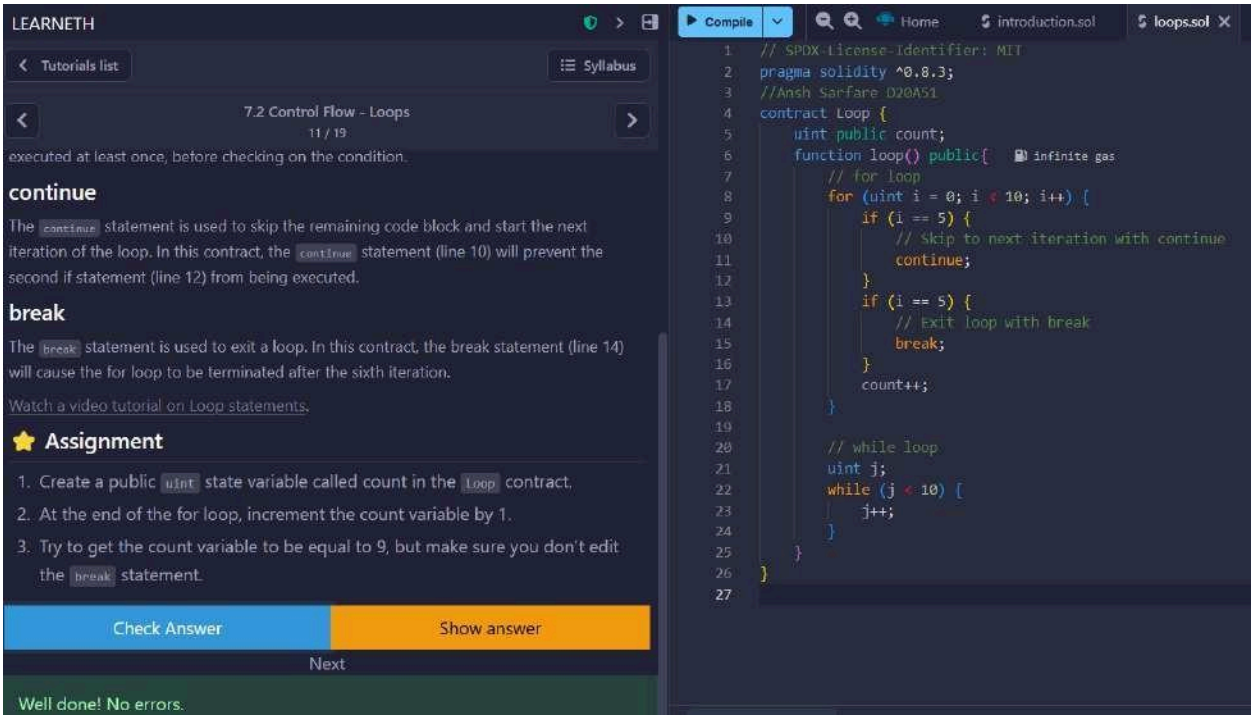
Tutorial - 9



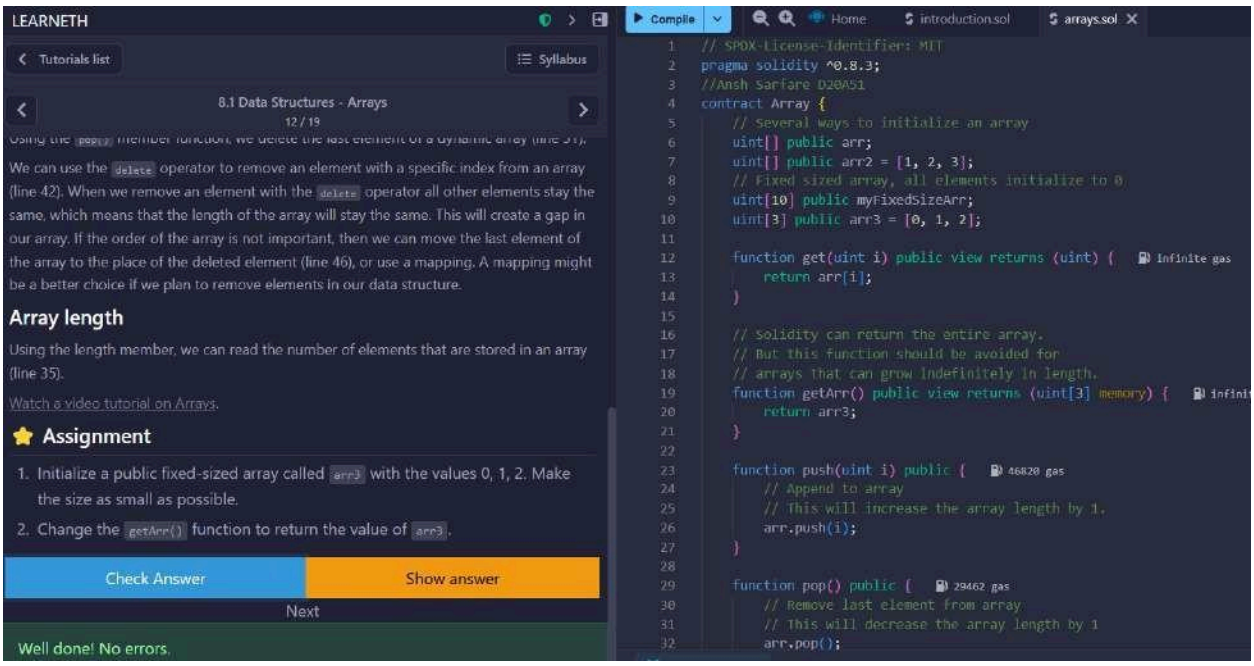
Tutorial - 10



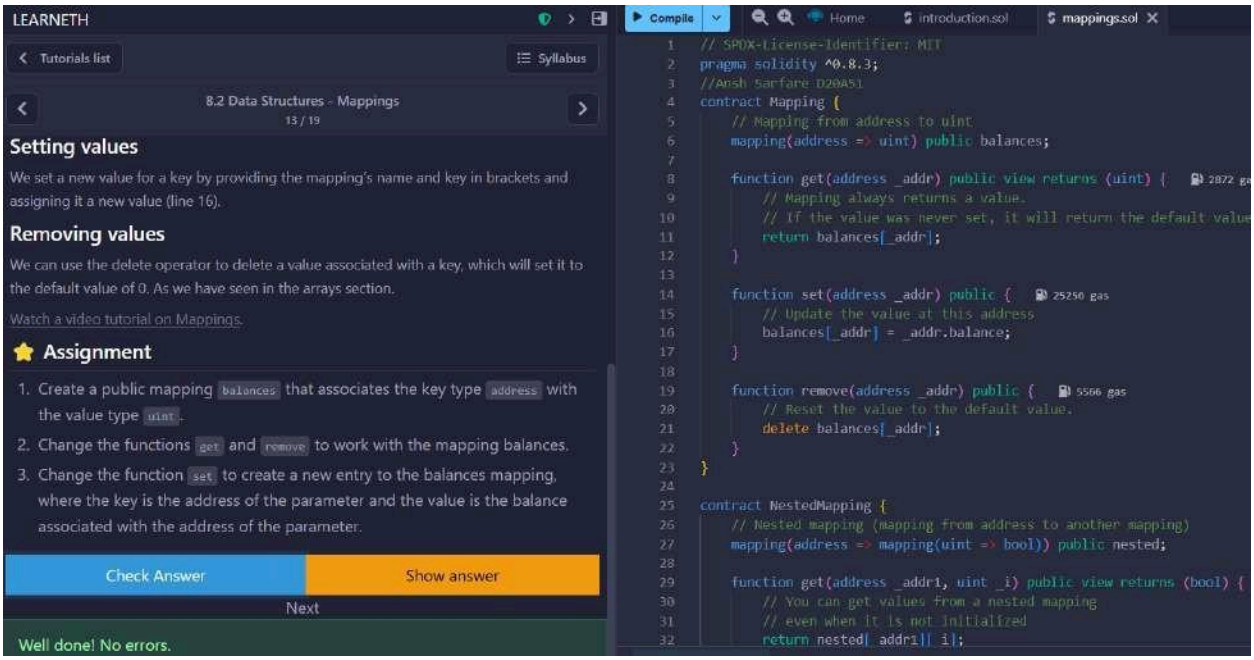
Tutorial - 11



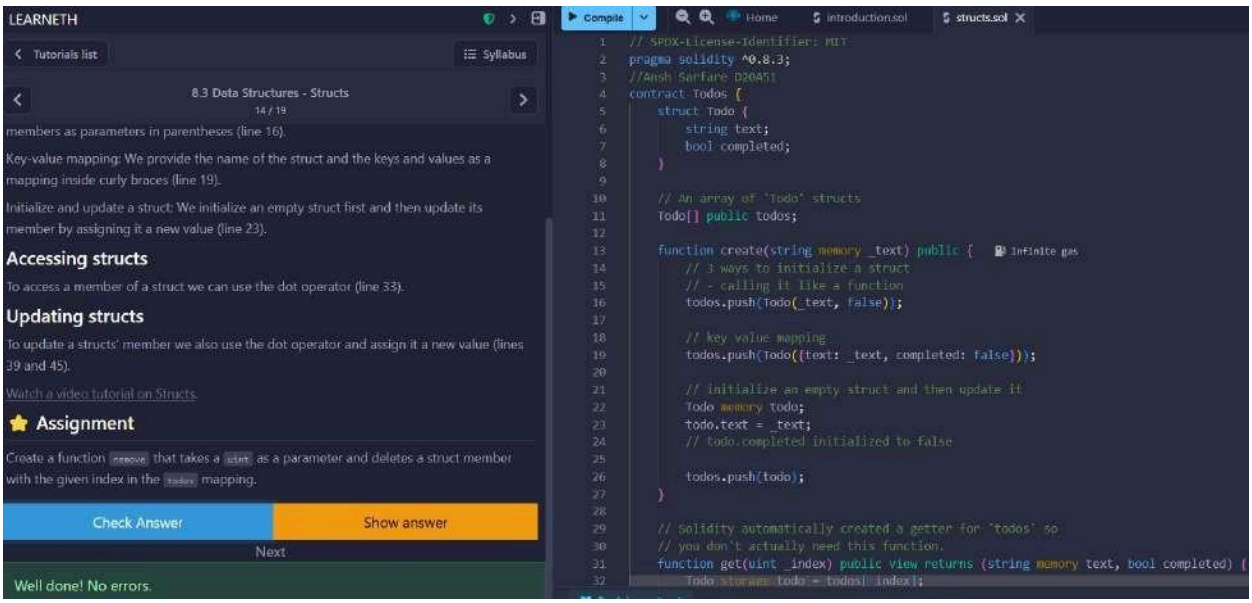
Tutorial - 12



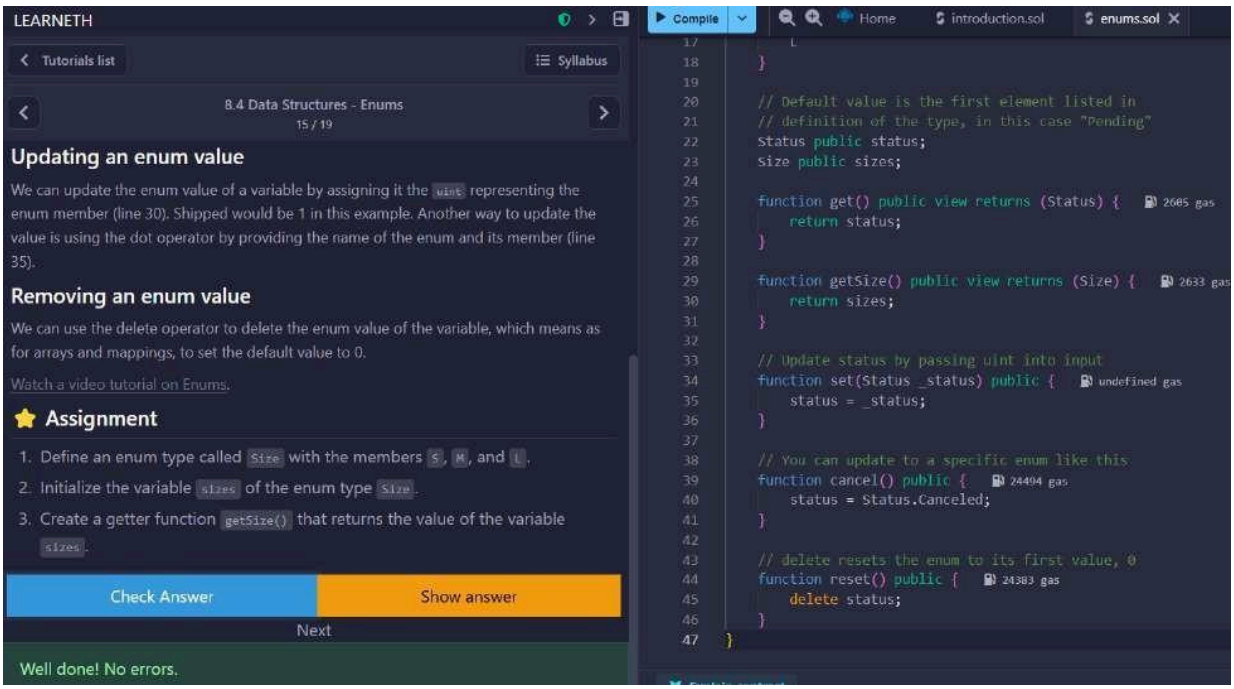
Tutorial - 13



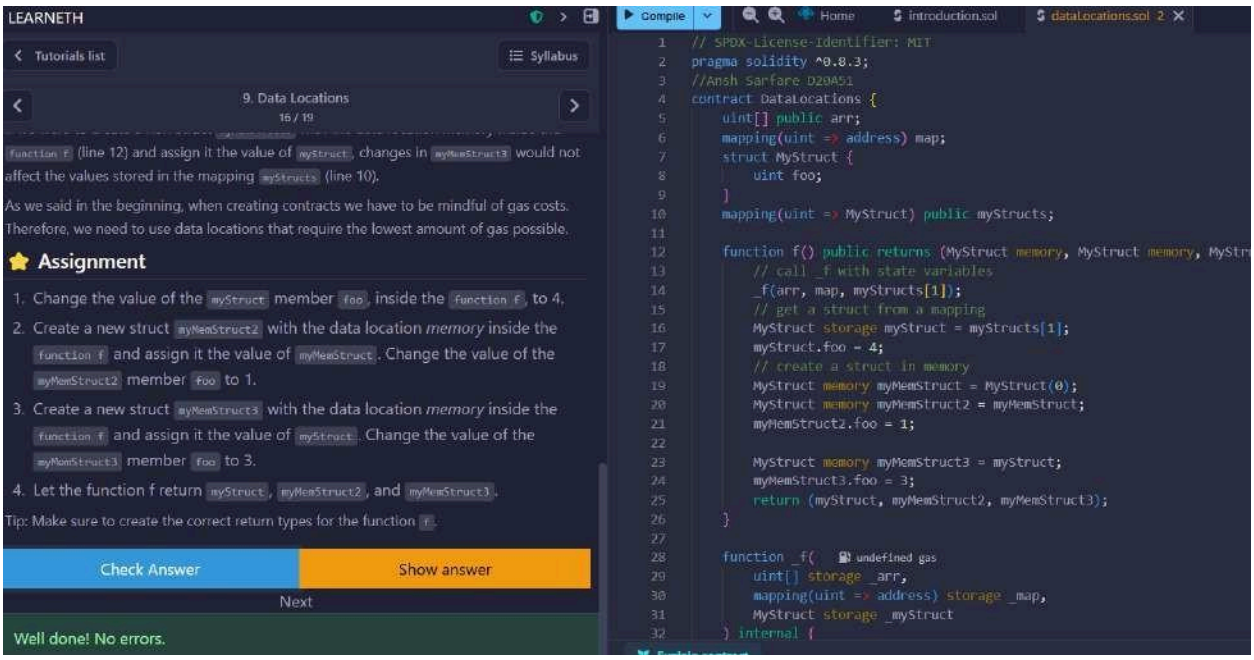
Tutorial - 14



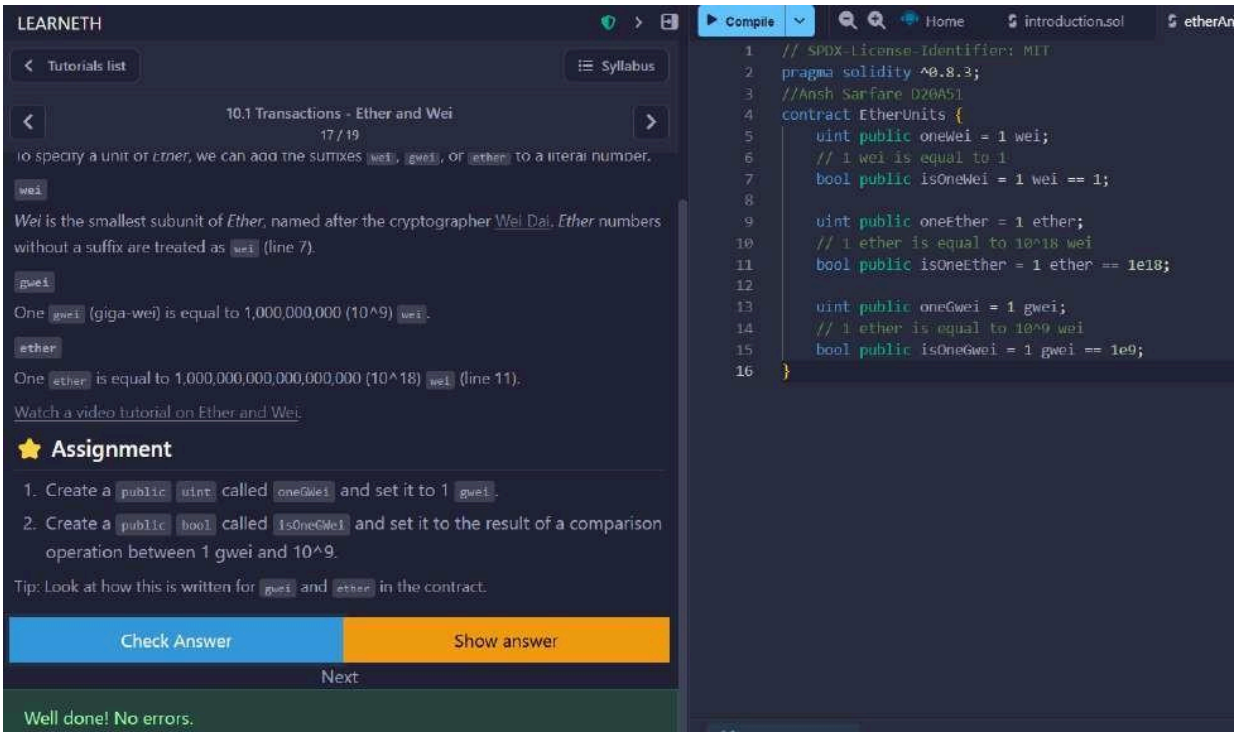
Tutorial - 15



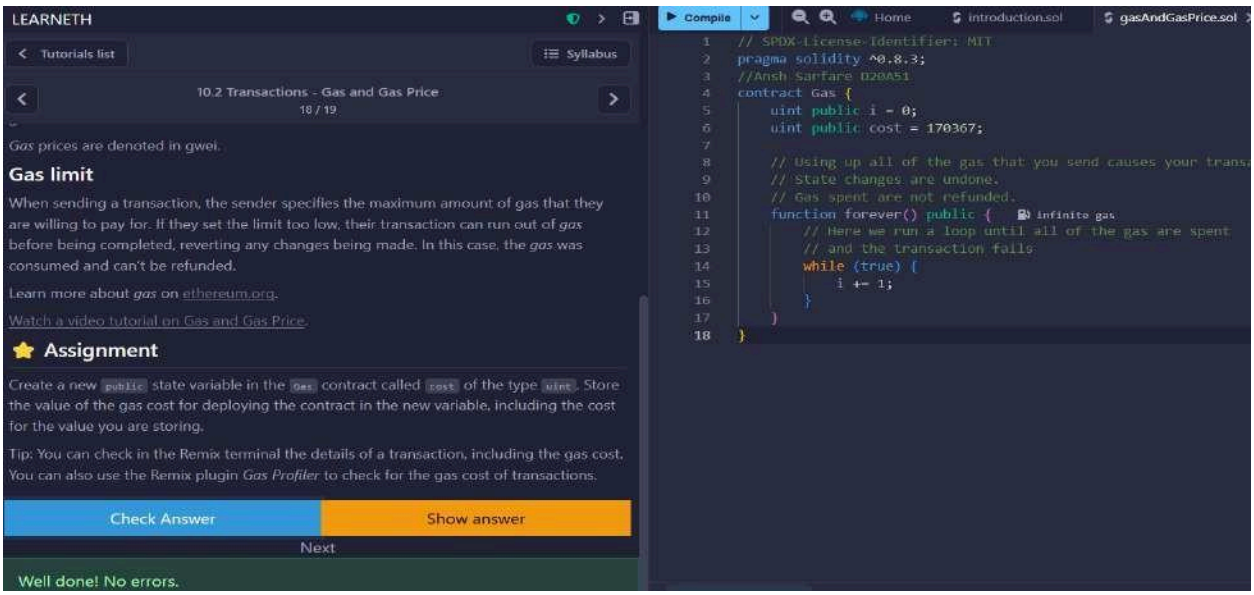
Tutorial - 16



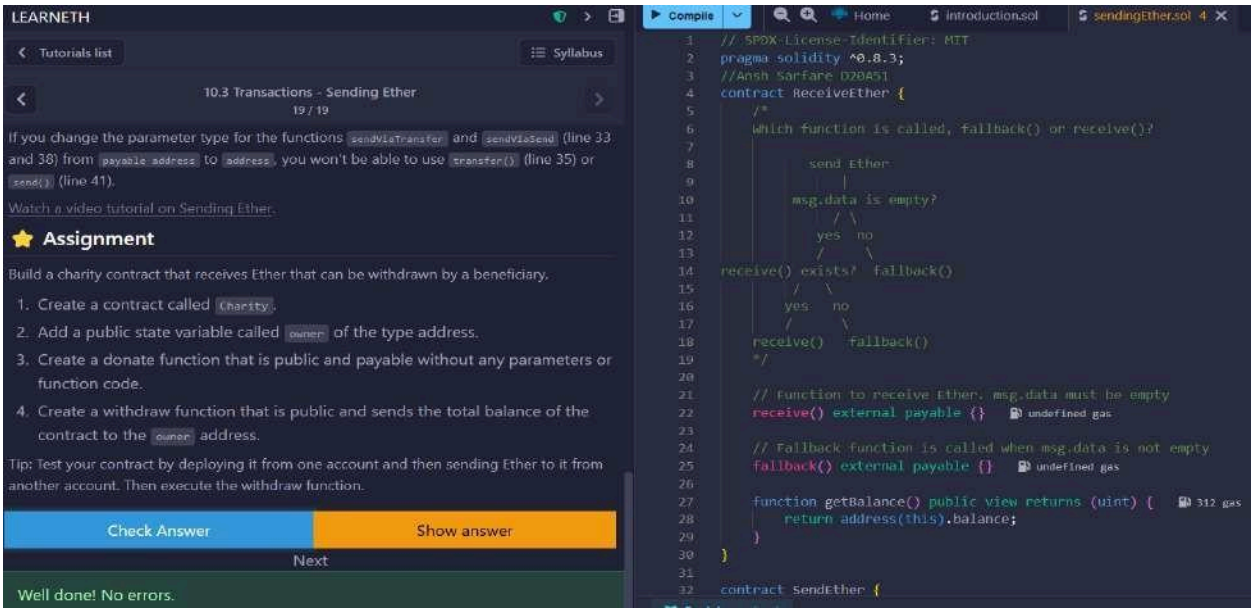
Tutorial - 17



Tutorial - 18



Tutorial - 19



Conclusion: Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Ansh Sarfare **D20A – 51**

Blockchain Exp- 5

AIM: Deploying a Voting/Ballot Smart Contract

Tasks:

1. Open Remix IDE
2. Under **Workspaces,** open **contracts** folder
3. Open **Ballot.sol**, contract.
4. Understand **Ballot.sol** contract.
5. Deploy the contract by changing the Proposal name from **bytes32 → string**

THEORY:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote (require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and

memory mapping:

- A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

mapping(address => Voter) public voters;

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

storage:

- In Solidity, storage refers to the permanent memory of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's
  information saved in the voters mapping remains available throughout the contract's lifecycle.

memory:

- In contrast, memory is temporary storage, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

- **bytes32** is a fixed-size type, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a dynamically sized type, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the Web3 Type Converter help developers easily switch between these two types for deployment and testing.

CODE:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

   struct  Voter  {
      uint
      weight;
      bool voted;
      address
      delegate;      uint
      vote;
   }

   struct Proposal {
      string name;     // CHANGED from bytes32 →
      string uint voteCount;
   }

   address public chairperson;
   mapping(address => Voter) public voters;
   Proposal[] public proposals;

   /**
    * @dev Create a new ballot
    * @param proposalNames names of proposals
    */
   constructor(string[] memory proposalNames)
      { chairperson = msg.sender;
      voters[chairperson].weight = 1;
```

```
    for (uint i = 0; i < proposalNames.length; i++) {
        proposals.push(
            Proposal({
                name: proposalNames[i],
                voteCount: 0
            })
        );
    }
}

function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted");
    require(voters[voter].weight == 0, "Voter already has right");

    voters[voter].weight = 1;
}

function delegate(address to) external {
    Voter storage sender =
    voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");
    require(to != msg.sender, "Self-delegation not allowed");

    while (voters[to].delegate != address(0))
        { to = voters[to].delegate;
        require(to != msg.sender, "Delegation loop detected");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right");

    sender.voted = true;
```

```
        sender.delegate = to;

        if (delegate_.voted) {
            proposals[delegate_.vote].voteCount +=
            sender.weight;
        } else {
            delegate_.weight += sender.weight;
        }
    }

    function vote(uint proposal) external {
        Voter storage sender = voters[msg.sender];

        require(sender.weight != 0, "No right to vote");
        require(!sender.voted, "Already voted");

        sender.voted = true;
        sender.vote = proposal;

        proposals[proposal].voteCount += sender.weight;
    }

    function winningProposal() public view returns (uint winningProposal_) {
        uint winningVoteCount = 0;

        for (uint p = 0; p < proposals.length; p++) {
            if (proposals[p].voteCount > winningVoteCount) {
                winningVoteCount = proposals[p].voteCount;
                winningProposal_ = p;
            }
        }
    }

    function winnerName() external view returns (string memory) {
        return proposals[winningProposal()].name;
```

```
    }
}
```

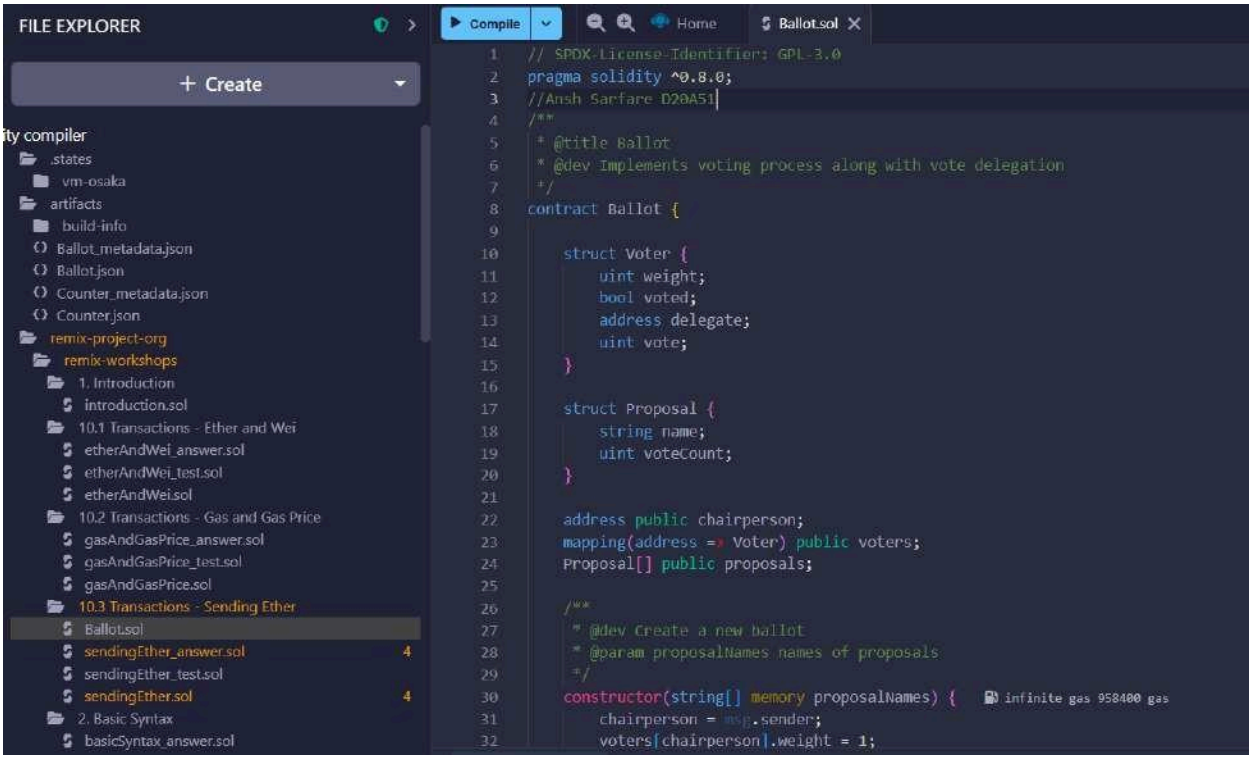
OUTPUT:

Step 1: Open Remix

Go to −− https://remix.ethereum.org
No install needed.

Step 2: Create the file

- **In File Explorer**



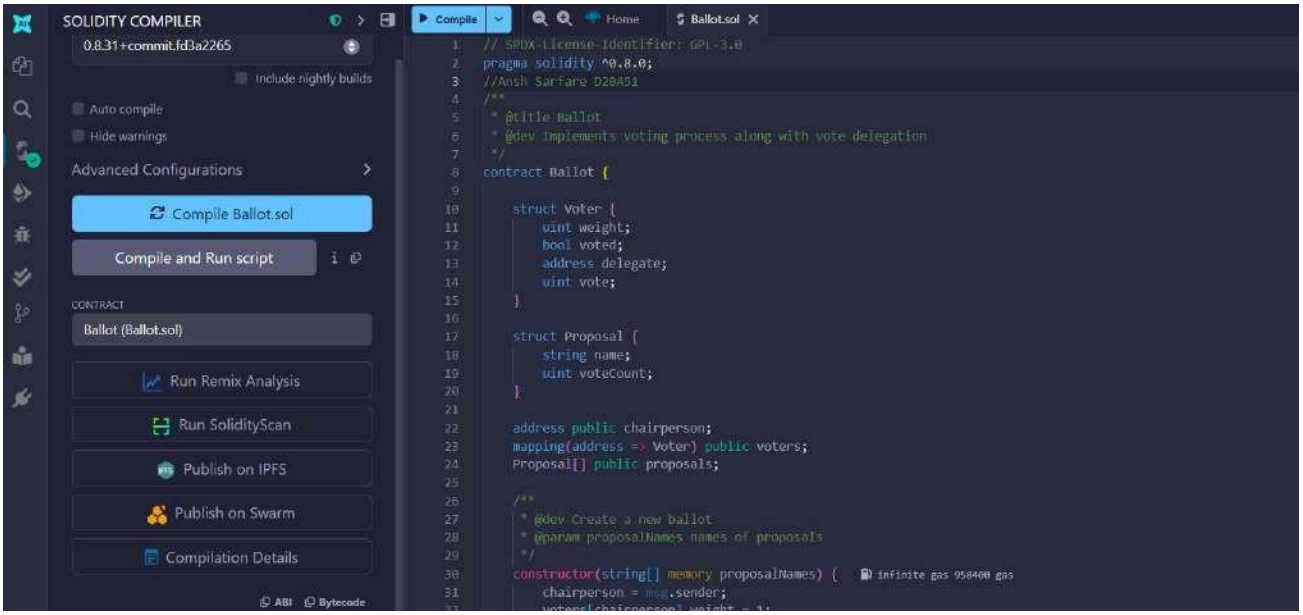
Ballot.sol

Step 3: Compile the contract

1. **Open Solidity Compiler (left sidebar)**
2. **Click Compile Ballot.sol**
3. **Make sure green tick appears (no**

errors) Step 4: Deploy the contract

1. **Open Deploy & Run Transactions**
2. **Set:**
    - **Environment → Remix VM**
    - **Account → keep default**
    - **Contract → Ballot**



Step 5: Enter constructor input

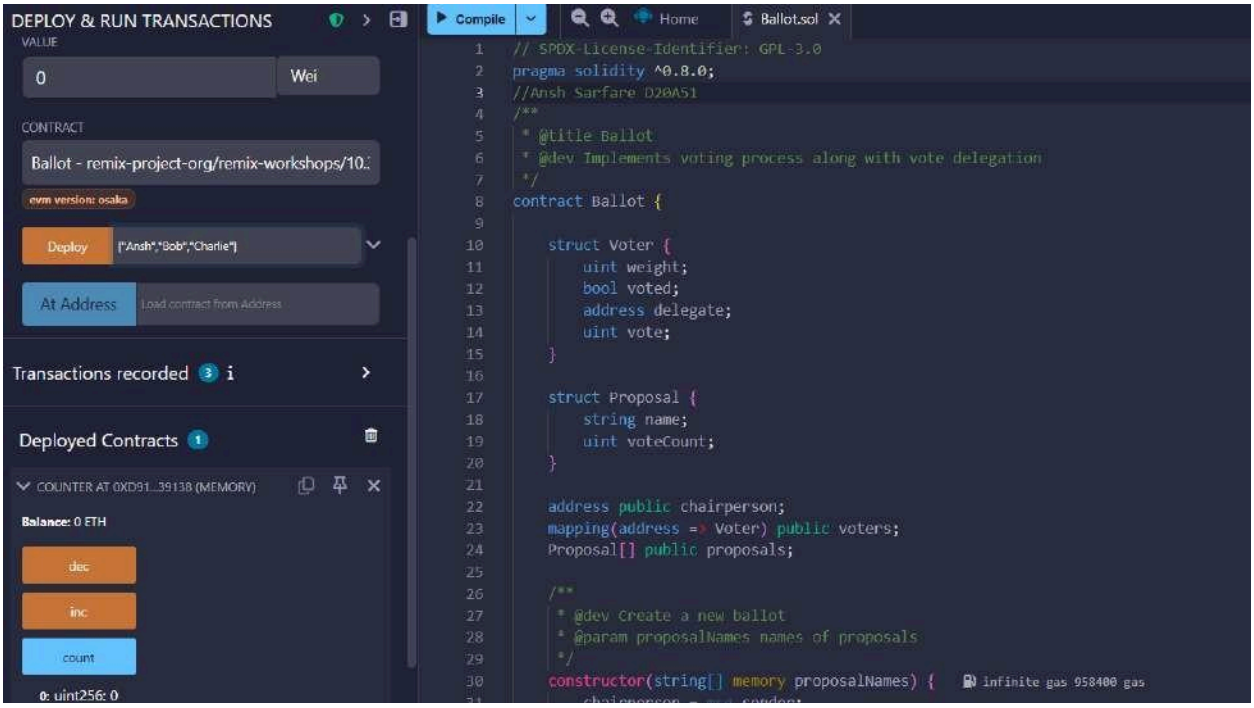
(IMPORTANT) Your constructor is:

constructor(string[] memory proposalNames)

So input must be a string array)rJ

Example:

["Ansh", "Bob", "Charlie"]



Step 6: Click Deploy

- **Click Deploy**
- **Contract appears under Deployed Contracts**
- **The deployer is automatically the**

chairperson giveRightToVote (address voter)

What to add:

Paste a Remix account address (NOT the chairperson).

Important:

- **You must be using Account 1 (chairperson) when clicking this**
- **Switch account at the top if**

needed Click Transact

Then vote
2 vote (uint256 proposal)
What to add:

A number, based on proposal index:

0 → Ansh

1 → Bob

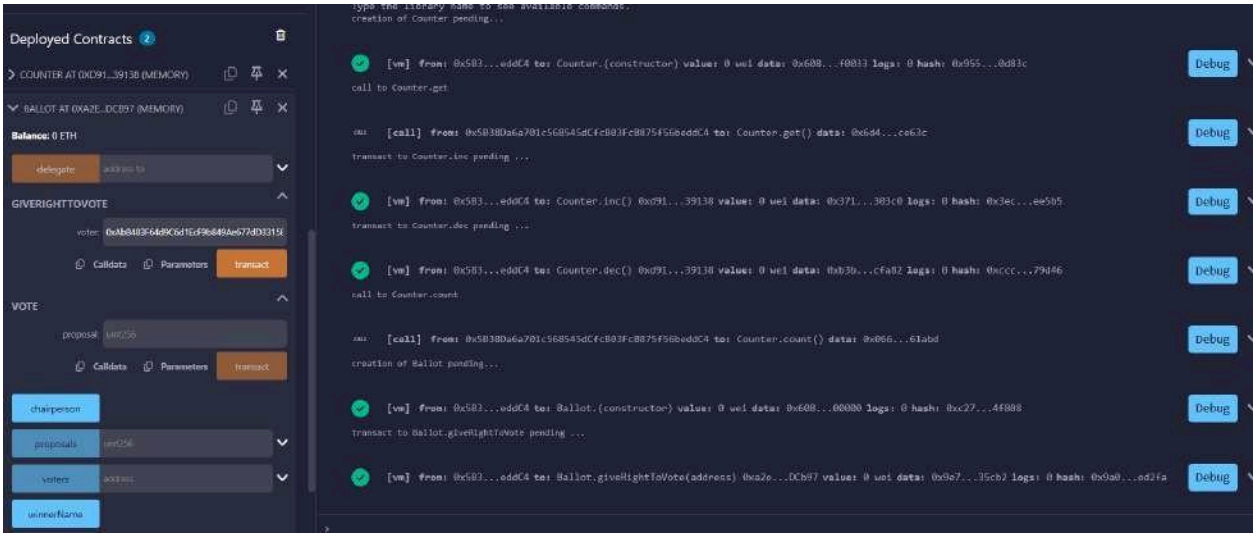
2 → Charlie

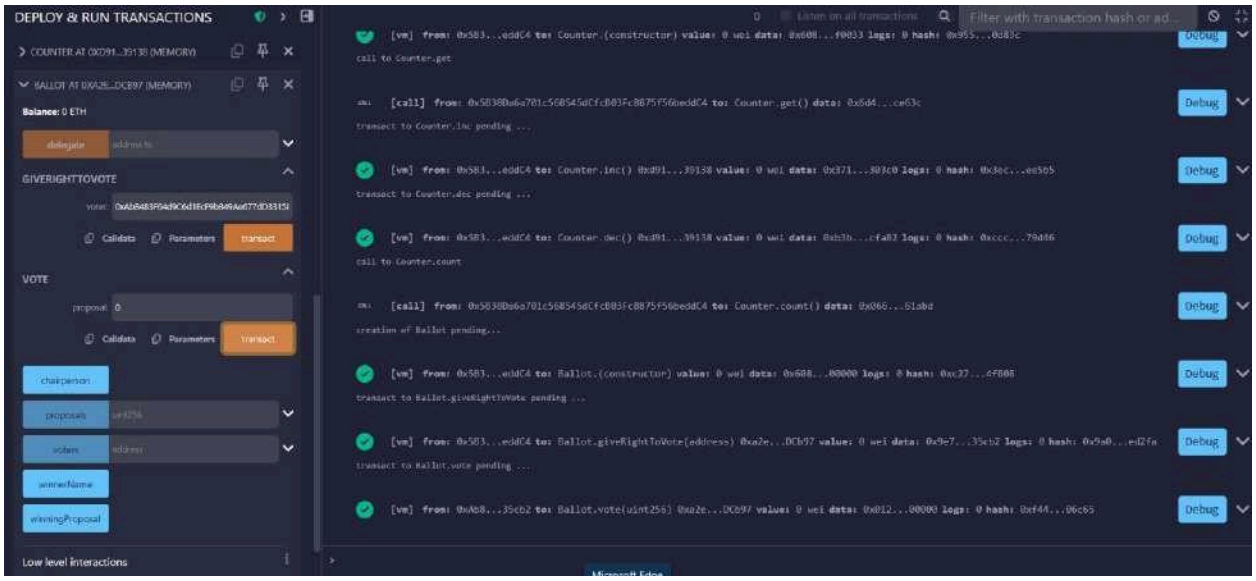
Example:

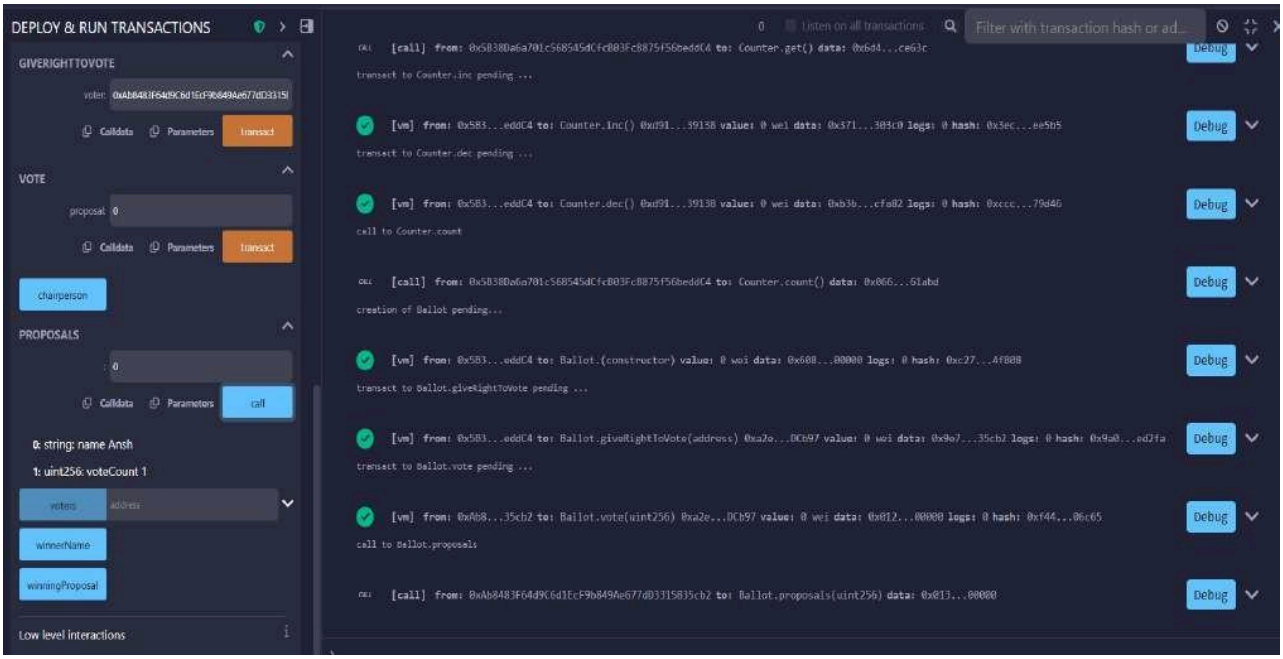
0

Important:

- **Switch to the voter account (the one you gave rights to)**
- **Then click Transact**




DEPLOY & RUN TRANSACTIONS
GIVERIGHTTOVOTE
VOTE
proposal
Calldata
Parameters
transact
chairperson
proposals
voters
winnerName
winningProposal
Low level interactions
[vm] from: 0x583...eddC4 to: Counter.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0x955...0d83c
call to Counter.get
[call] from: 0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c
transact to Counter.inc pending ...
[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5
transact to Counter.dec pending ...
[vm] from: 0x583...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xccc...79d46
call to Counter.count
[call] from: 0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4 to: Counter.count() data: 0x066...51abd
creation of Ballot pending...
[vm] from: 0x583...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xc27...4f808
transact to Ballot.giveRightToVote pending ...
[vm] from: 0x583...eddC4 to: Ballot.giveRightToVote(address) 0xa2e...DCb97 value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x9a0...ed2fa
transact to Ballot.vote pending ...
[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xa2e...DCb97 value: 0 wei data: 0x012...00000 logs: 0 hash: 0xf44...06c65
Debug


DEPLOY & RUN TRANSACTIONS
GIVERIGHTTOVOTE
VOTE
proposal
Calldata
Parameters
transact
chairperson
PROPOSALS
call
0: string: name Ansh
1: uint256: voteCount 1
voters
winnerName
winningProposal
Low level interactions
[call] from: 0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c
transact to Counter.inc pending ...
[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x3ec...ee5b5
transact to Counter.dec pending ...
[vm] from: 0x583...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0xccc...79d46
call to Counter.count
[call] from: 0x5B38Da6a701c568545dCfcB03Fc8875f56beddC4 to: Counter.count() data: 0x066...51abd
creation of Ballot pending...
[vm] from: 0x583...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xc27...4f808
transact to Ballot.giveRightToVote pending ...
[vm] from: 0x583...eddC4 to: Ballot.giveRightToVote(address) 0xa2e...DCb97 value: 0 wei data: 0x9e7...35cb2 logs: 0 hash: 0x9a0...ed2fa
transact to Ballot.vote pending ...
[vm] from: 0xAb8...35cb2 to: Ballot.vote(uint256) 0xa2e...DCb97 value: 0 wei data: 0x012...00000 logs: 0 hash: 0xf44...06c65
call to Ballot.proposals
[call] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 to: Ballot.proposals(uint256) data: 0x013...00000
Debug

proposals (uint256)

What to add:

Proposal index:

0

Returns proposal details.

Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchai

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Ansh Sarfare **D20A - 51**

BC Experiment-6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

What is a Smart Contract? A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- **Automation:** Executes actions automatically when conditions are satisfied.
- **Transparency:** Contract code and transactions are visible on the blockchain.
- **Security:** Once deployed, contracts are immutable and tamper-proof.
- **Cost-effective:** Removes intermediaries and reduces transaction costs.
- **Trustless System:** Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

About the Crowdfunding Platform: A blockchain-based crowdfunding platform replaces centralized authorities with smart contracts. Two contracts are designed — **CrowdfundingContract.sol**, which manages campaign creation and tracks donations, and **DonorContract.sol**, which handles donor interactions and communicates with the CrowdfundingContract through a Solidity interface. Together they ensure a transparent, trustless, and intermediary-free fundraising system.

Tools Used:

- **Solidity** — High-level language for writing smart contracts on Ethereum.
- **Remix IDE** — Browser-based IDE for writing, compiling, deploying, and testing smart contracts using a built-in Remix VM.

IMPLEMENTATION:

CrowdfundingContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract CrowdfundingContract {
```

```
    struct Campaign {
        uint256
        campaignId; string
        title;
        uint256 goalAmount;
        uint256 raisedAmount;
        uint256 deadline;
        address payable creator;
        bool isActive;
    }

    uint256 public campaignCount = 0;
    mapping(uint256 => Campaign) public campaigns;

         event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount,
address creator);
    event FundsReceived(uint256 campaignId, uint256 amount);

    function
        createCampaign(
        string memory _title,
        uint256 _goalAmount,
        uint256 _durationDays
    ) public {
        campaignCount++;
        uint256 deadline = block.timestamp + (_durationDays * 1 days);
        campaigns[campaignCount] = Campaign(
            campaignCount,
            _title,
            _goalAmount,
            0,
            deadline,
            payable(msg.sender),
            true
        );
        emit CampaignCreated(campaignCount, _title, _goalAmount, msg.sender);
    }

    function receiveDonation(uint256 _campaignId, uint256 _amount) external {
        Campaign storage c = campaigns[_campaignId];
        require(c.isActive, "Campaign is not active");
        c.raisedAmount += _amount;
        emit FundsReceived(_campaignId, _amount);
    }

    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
```

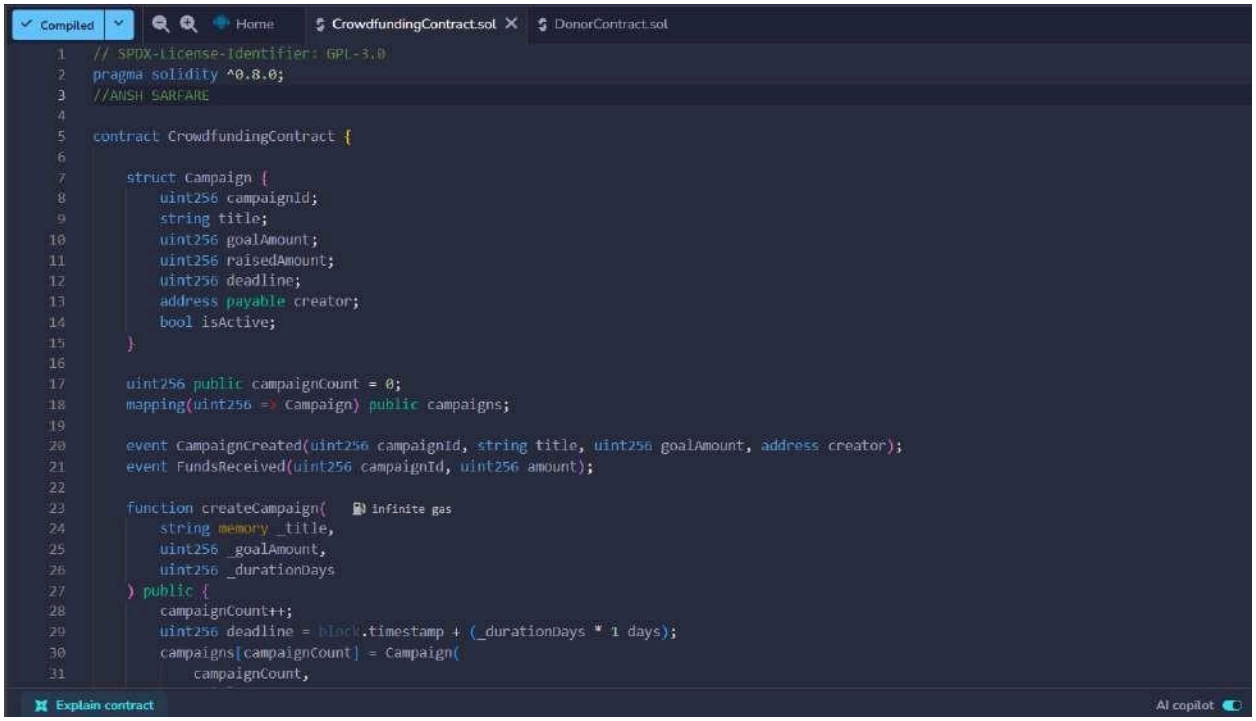
```
    ) {
        Campaign storage c = campaigns[_campaignId];
        return (
            c.campaignId,
            c.title,
            c.goalAmount,
            c.raisedAmount,
            c.deadline,
            c.creator,
            c.isActive
        );
    }

    function closeCampaign(uint256 _campaignId) external {
        campaigns[_campaignId].isActive = false;
    }
}
```

DonorContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}
contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256
        campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }
    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256
amount);
    constructor(address _crowdfundingAddress) {
        crowdfundingContractAddress = _crowdfundingAddress;
    }
```

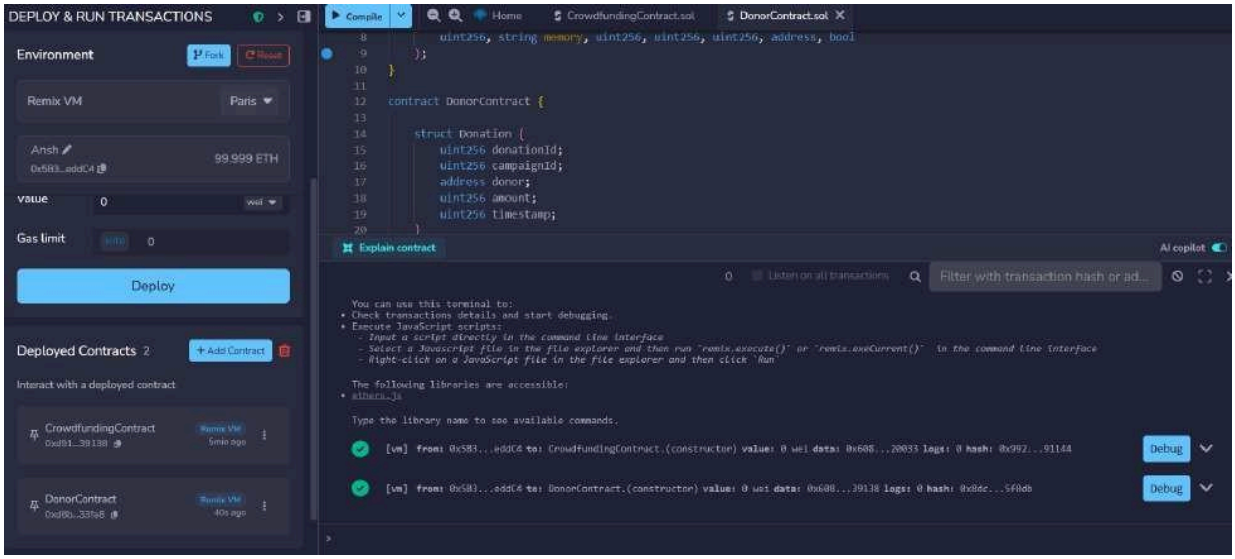
```
    function donate(uint256 _campaignId, uint256 _amount) public {
        donationCount++;
        donations[donationCount] = Donation(
            donationCount,
            _campaignId,
            msg.sender,
            _amount,
            block.timestamp
        );
        ICrowdfundingContract(crowdfundingContractAddress)
            .receiveDonation(_campaignId, _amount);

        emit DonationMade(donationCount, _campaignId, msg.sender, _amount);
    }
    function getDonation(uint256 _donationId) public view returns (
        uint256, uint256, address, uint256, uint256
    ) {
        Donation storage d = donations[_donationId];
        return (d.donationId, d.campaignId, d.donor, d.amount, d.timestamp);
    }
    function getCrowdfundingContract() public view returns (address) {
        return crowdfundingContractAddress;
    }
}
```

Compilation:

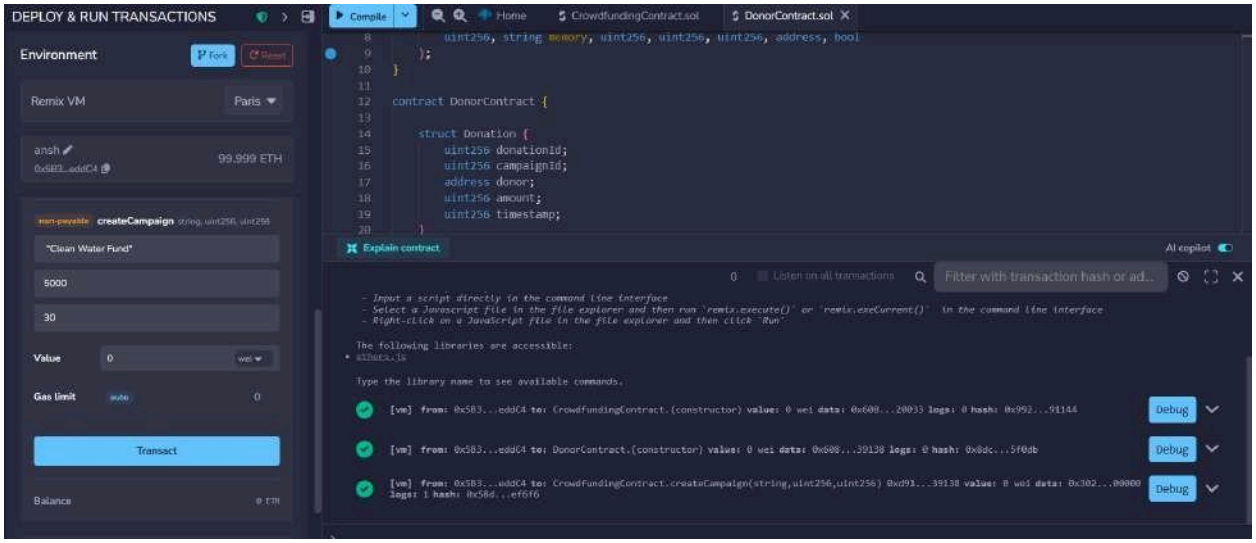


```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
//ANSH SARFARE

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}

contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }

    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256 amount);

    constructor(address _crowdfundingAddress) {
        crowdfundingContractAddress = _crowdfundingAddress;
    }
```

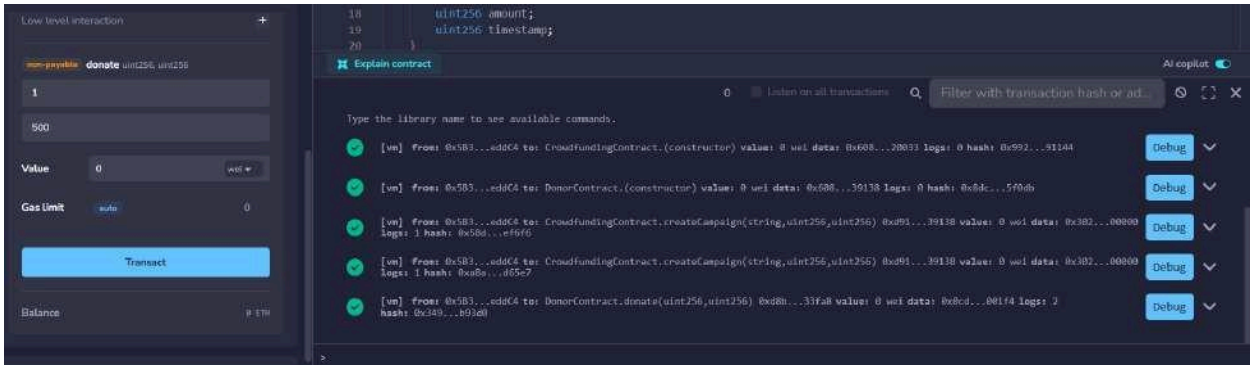
DEPLOYMENT:

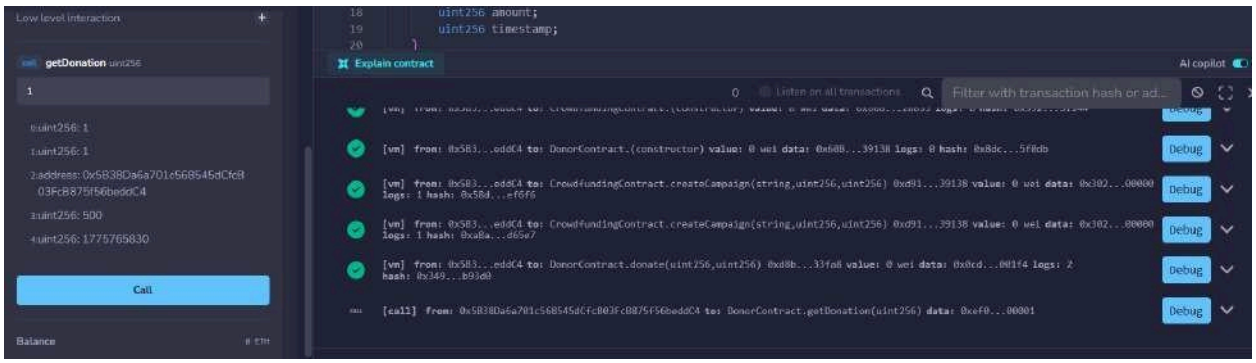


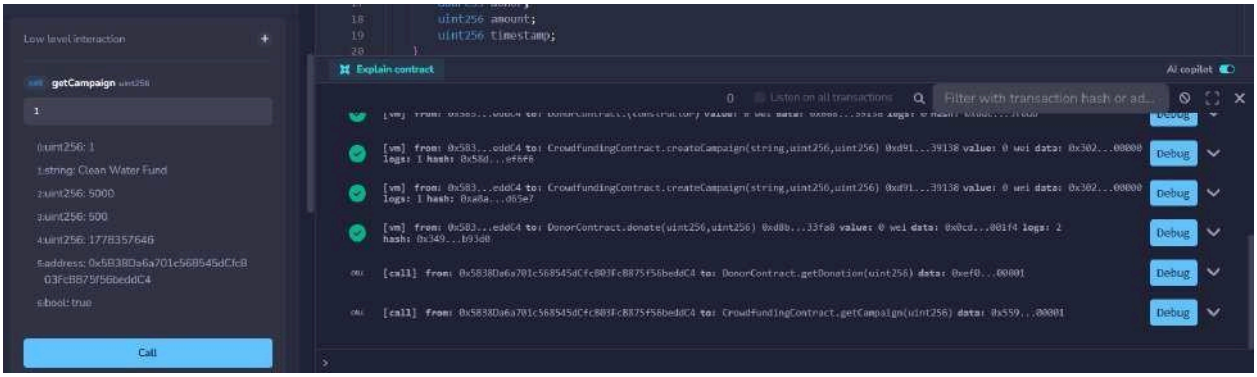
TRANSACTIONS:



DONATIONS







CONCLUSION:

This experiment helped in understanding how smart contracts can automate and secure a crowdfunding platform on the Ethereum blockchain. Using Solidity and Remix IDE, two contracts — CrowdfundingContract and DonorContract — were designed, deployed, and tested. The CrowdfundingContract manages campaign creation and tracks donations, while the DonorContract handles donor interactions. Together they demonstrate a trustless, transparent, and intermediary-free fundraising system.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Ansh Sarfare **D20A - 51**

BC Experiment-7

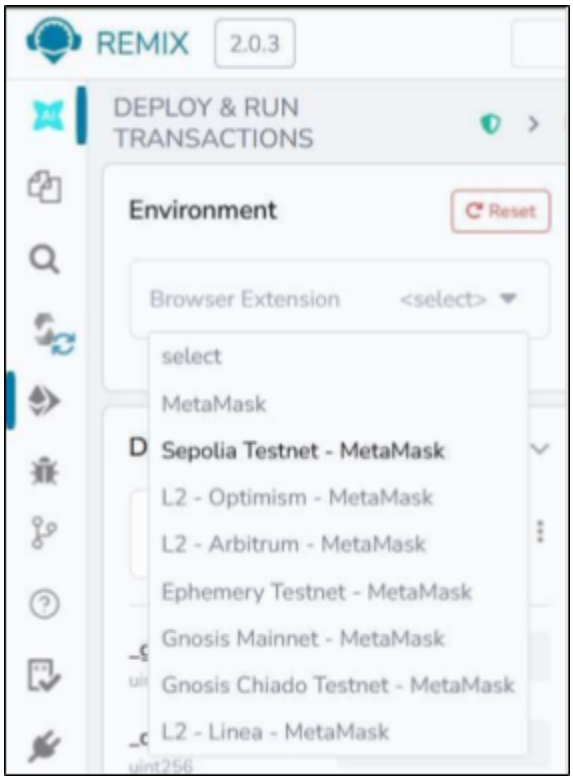
Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:
MetaMask is a cryptocurrency wallet and gateway to blockchain applications. It allows users to store, send, and receive Ether or tokens, and connect to decentralized applications (DApps) directly from a web browser. MetaMask acts as a bridge between a user's browser and the Ethereum blockchain (or other compatible networks).

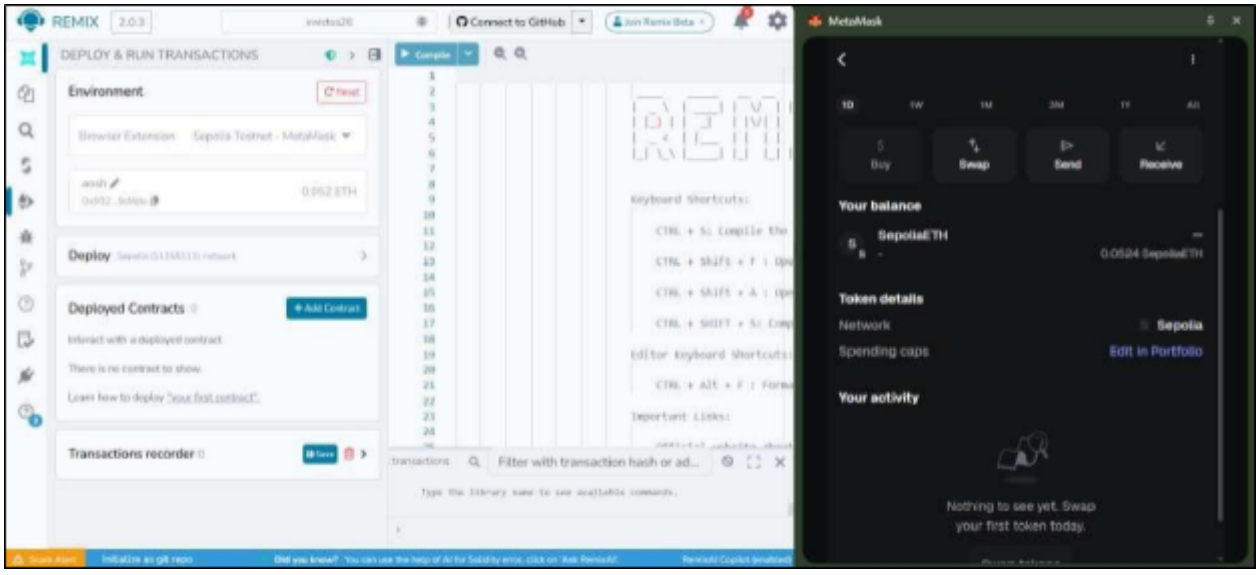
- **Features**:
  Easy management of multiple blockchain accounts
  Secure private key storage
  Integration with test networks like Sepolia, Linea, or RSK
  Interaction with DApps and smart contracts via Remix
  IDE
- **Testnet**:
  A Testnet is a blockchain network used for testing and development without using real cryptocurrency. It mimics the main Ethereum network but uses test Ether (fake ETH). Common testnets include Sepolia Testnet, Linea Testnet, and RSK Testnet.
- **Advantages**:
  No real money loss during testing
  Safe environment for developers
  Helps test smart contracts and transactions before mainnet deployment
  Steps to Connect MetaMask with Remix IDE for Performing Transactions:
- **Set Up MetaMask**:
  Install the MetaMask extension on your browser.
  Create or import a wallet account using a recovery phrase.
- **Add a Test Network**:
  Open MetaMask → Settings → Networks → Add Network.
  Enter details for the desired Testnet (e.g., Sepolia or RSK).
  Example RPC URL: https://sepolia.infura.io/v3/<your_project_id>
  Fund the Wallet:
  Use a faucet to receive free test Ether (e.g., 0.5 ETH/day).
  Check the balance in MetaMask.

Connect MetaMask with Remix IDE:
Open Remix IDE.
In the Deploy & Run Transactions tab, choose Injected Web3 as the environment.
Approve connection in MetaMask.

Step 1: Select metamask sepolia testnet in environment



Initial setup:



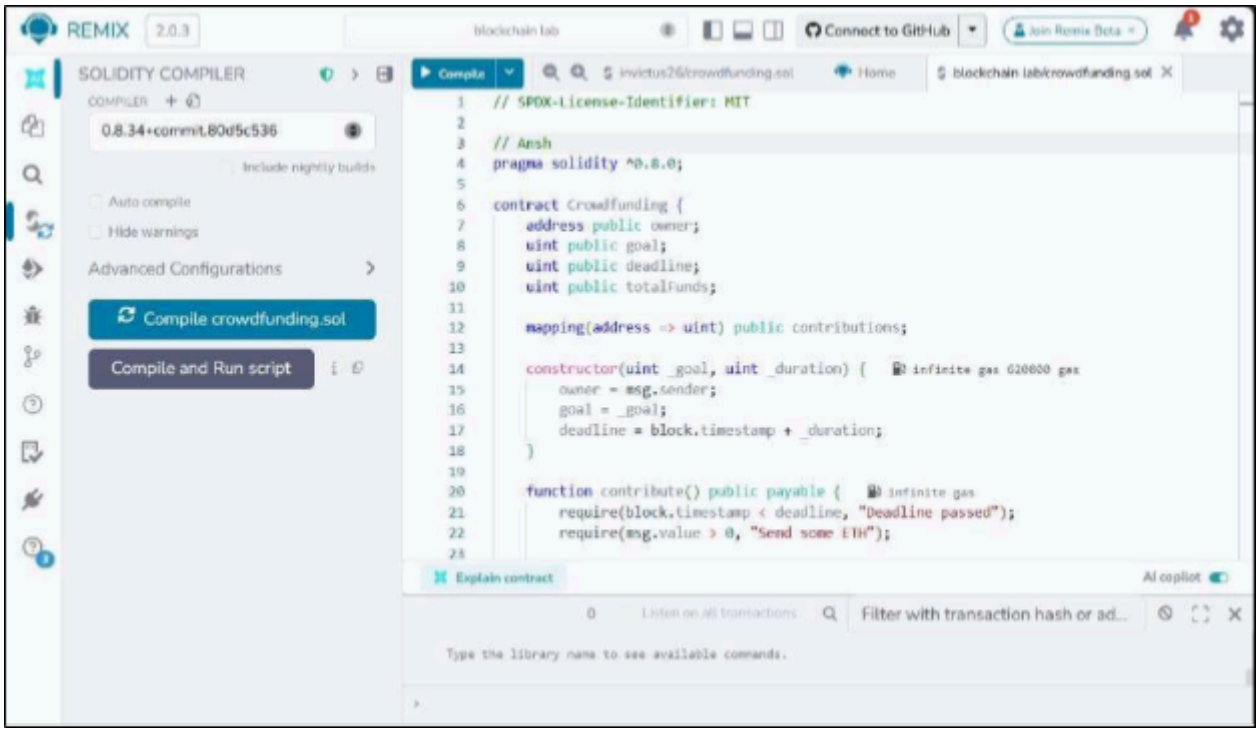
Step 2: Create a contract for a crowdfunding platform

Code:

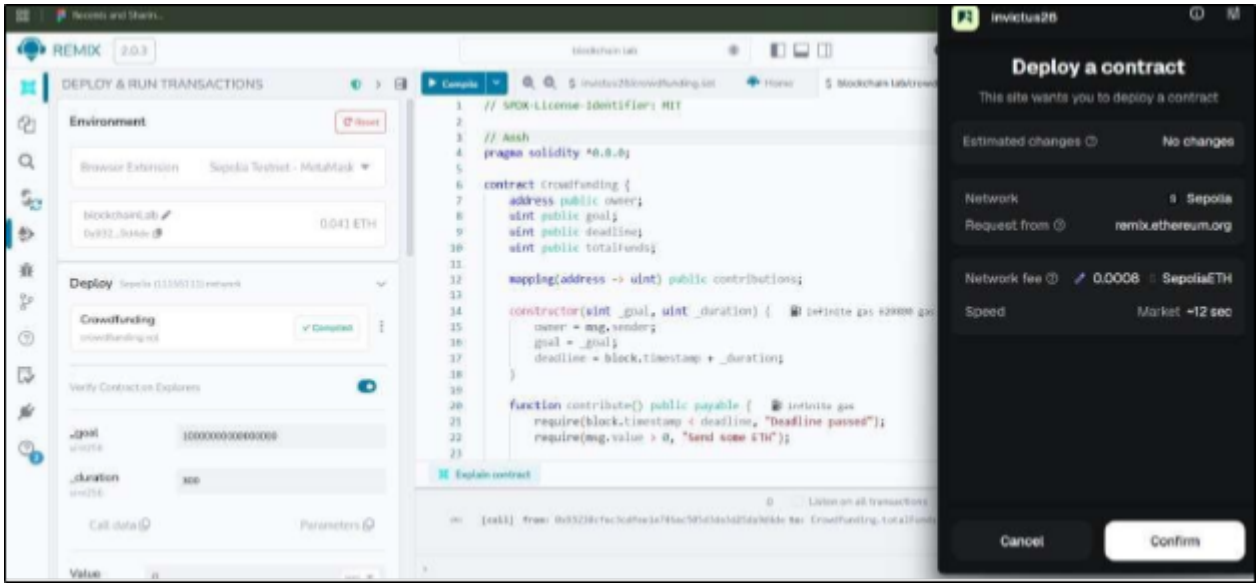
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;
    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {

        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }
    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");
```

```
        contributions[msg.sender] = 0;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Refund failed");
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

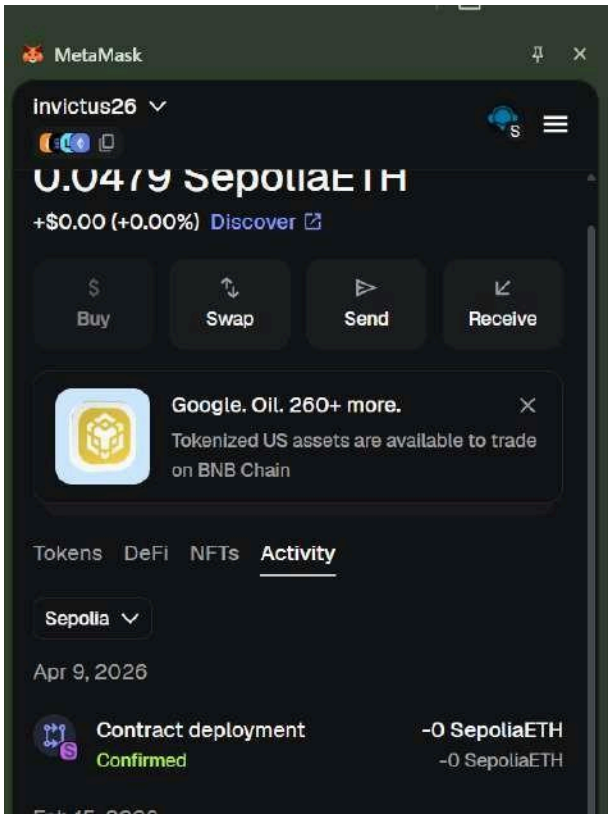
Step 3: save it as **crowdfunding.sol** and compile it



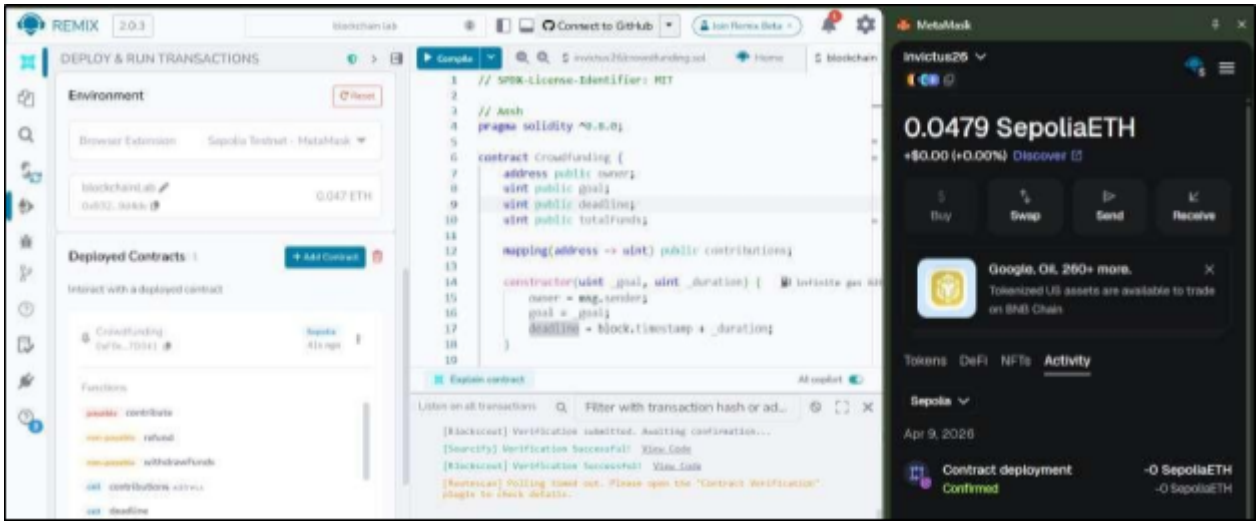
Step 4: deploy the contract with an initial goal and time limit. Confirm the transaction from metamask



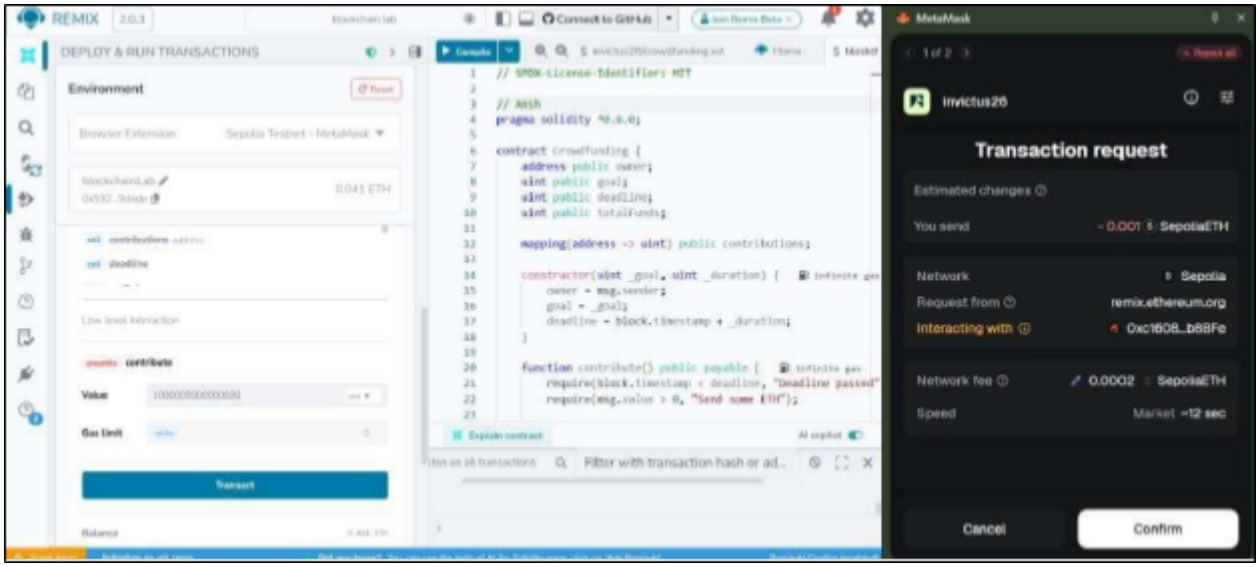
After deployment you can see: transaction recorded in metamask

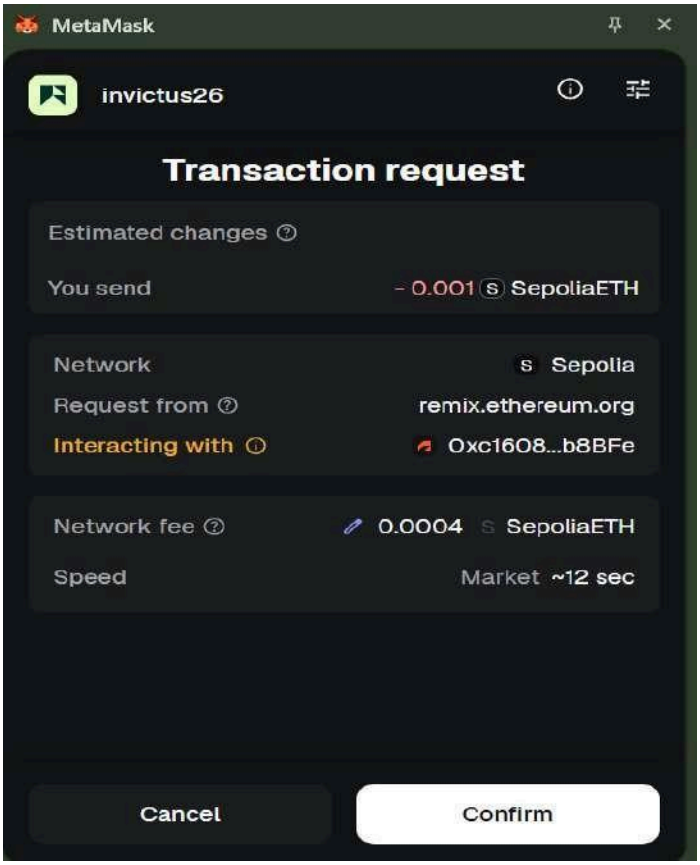


Step 5: After deployment in remix ide, we can see the deployed contract and its functions

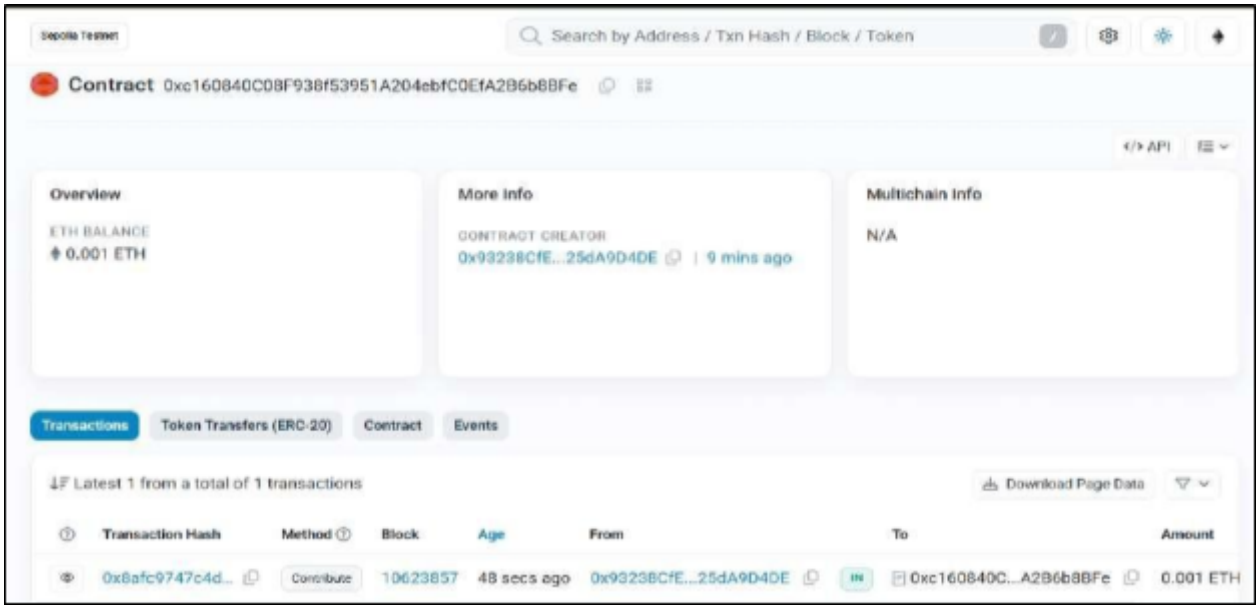


Step 6: Add a contribution of 0.01 ETH = 1000000000000000 wei. Transaction request appears in metamask. Click on confirm

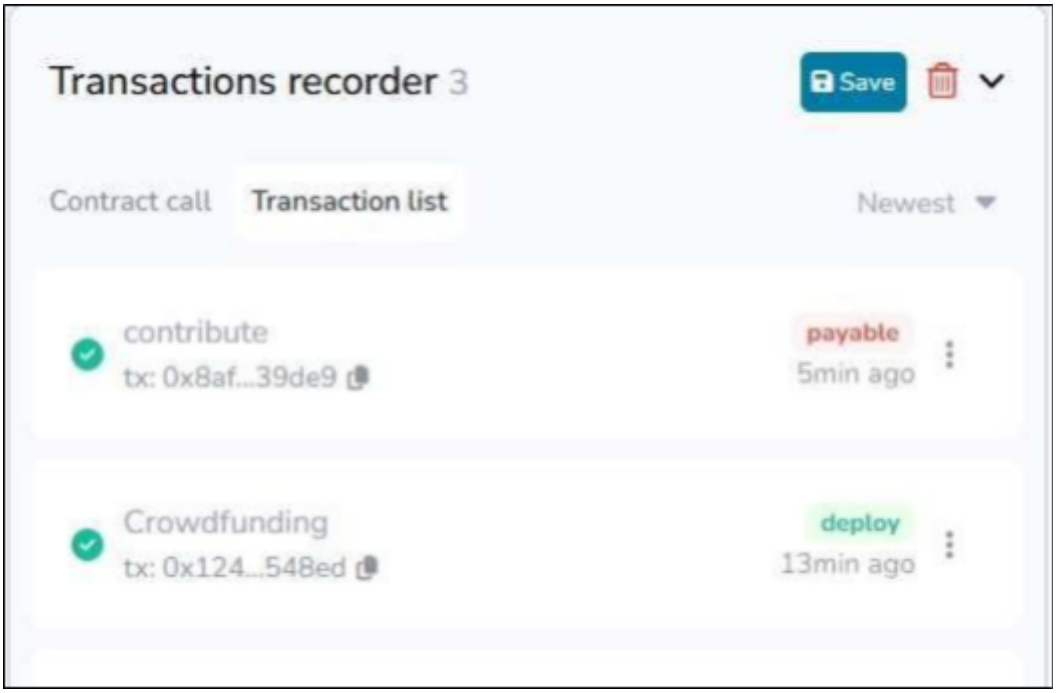




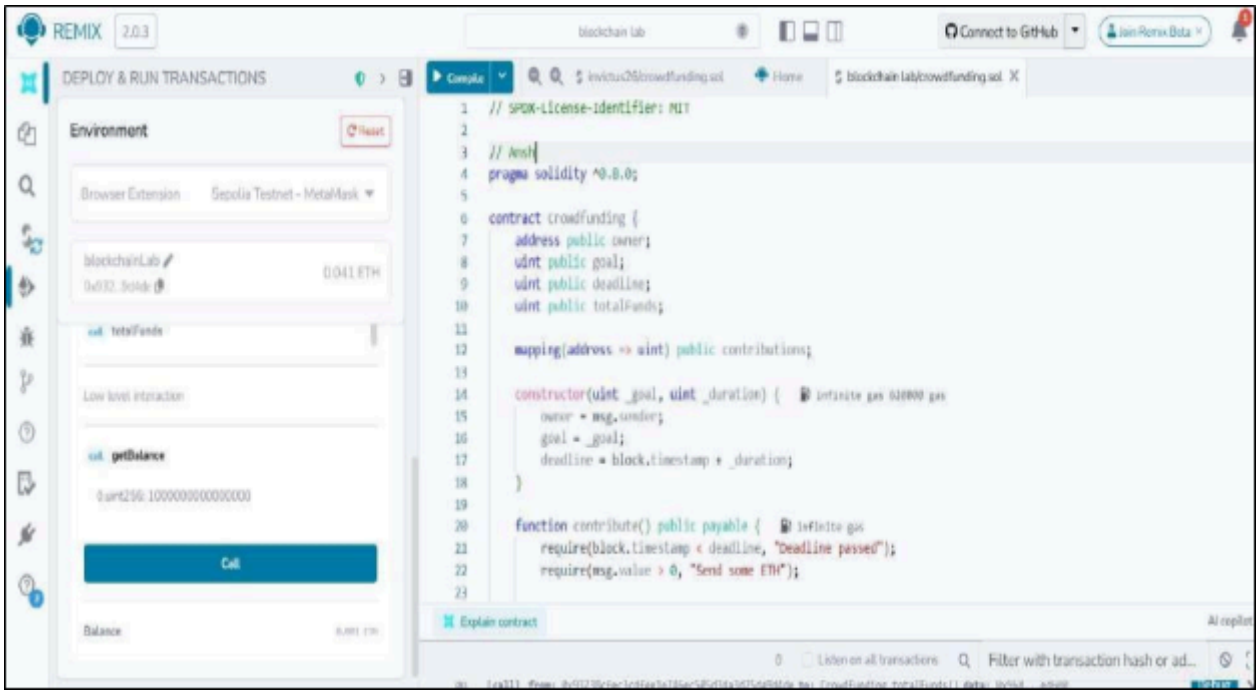
Step 7: copy paste the contracts address in https://sepolia.etherscan.io/ to see all transactions:



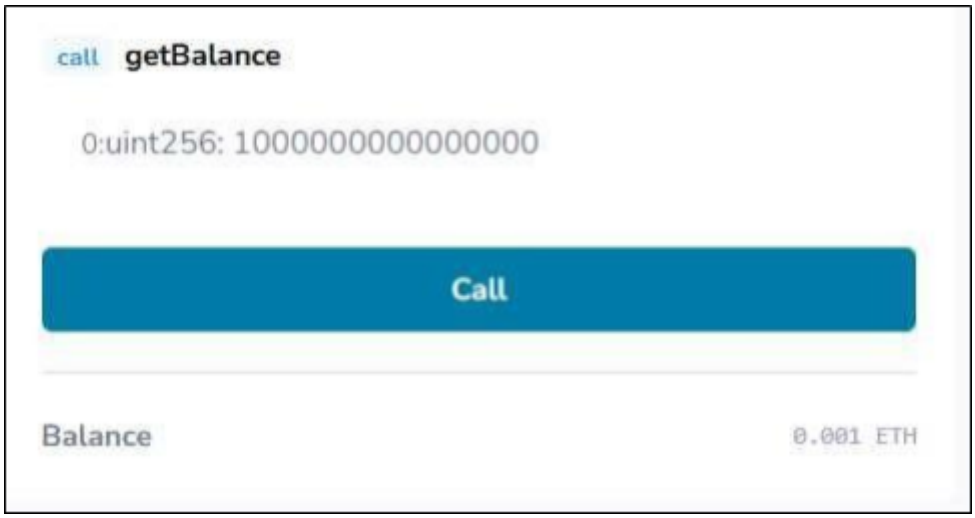
Also can see the transactions under all transactions in remix IDE



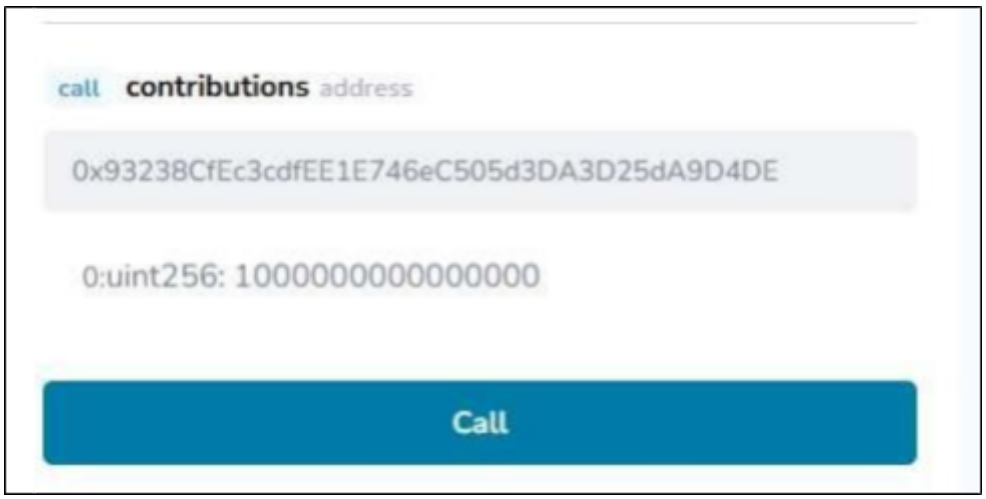
Step 8: use getBalance function to check the total funds received



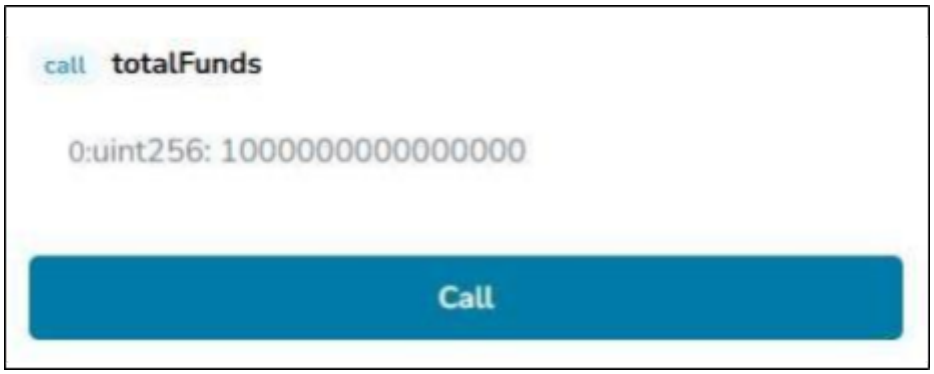
Output:



Step 9: Paste wallet address from metamask into contributions to see all transactions by the particular use



Step 10: use totalFunds function to see totalFunds recieved



Conclusion:
Using MetaMask with Remix IDE provides a practical way to test and deploy smart contracts on test networks. It helps developers understand real blockchain interactions, transaction confirmations, and wallet integrations in a secure, no-risk environment.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Ansh Sarfare **D20A - 51**

BC Experiment-8

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connecat Ganache with Metamask and Remix IDE:

1. Install Ganache:
    - Download and install Ganache from the Truffle Suite website.
    - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
    - Open Metamask and switch to a Custom RPC Network.
    - Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
    - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
    - Open Remix IDE in the browser.
      Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
    - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
    - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

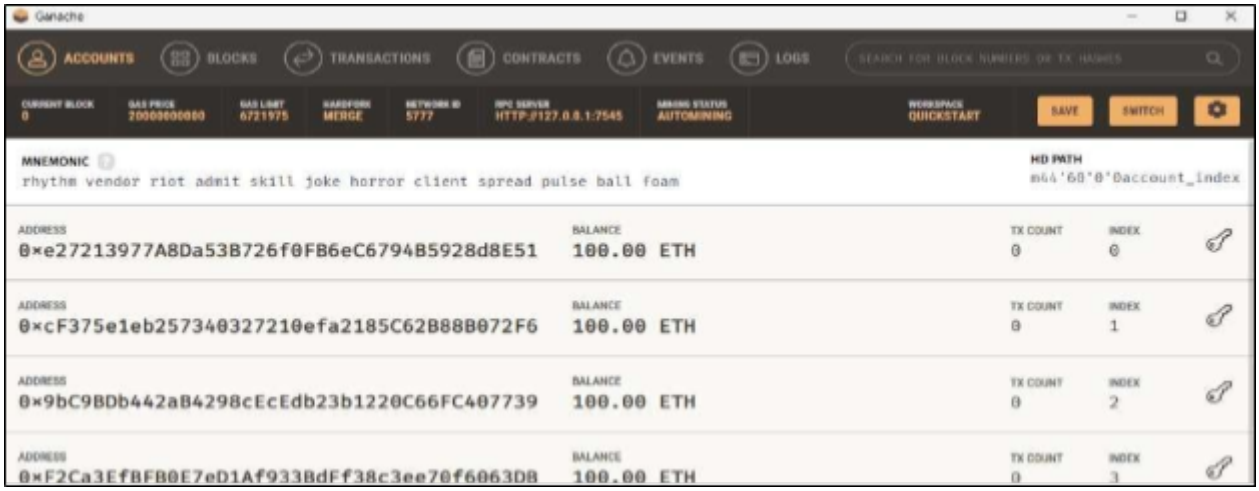
```
pragma solidity ^0.8.0;
    contract SimpleStorage {
      uint public data;
      function set(uint x) public {
        data = x;
      }
```

5. Compile and Deploy the Contract:
   Compile using Remix.
   - Deploy via the Metamask account connected to Ganache.
   - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
   - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
   - Use Remix to call contract functions (set() or get()).
   - Observe the state changes and corresponding transactions on Ganache.

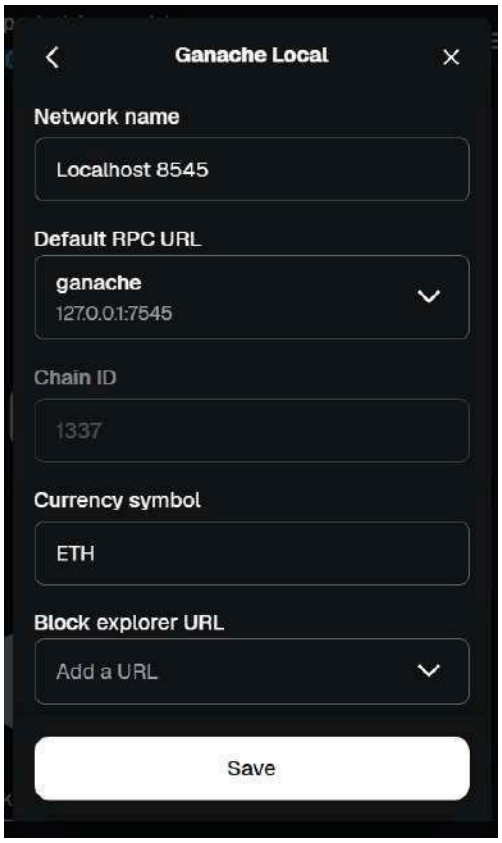
Implementation:

Step 1: Download and install ganache

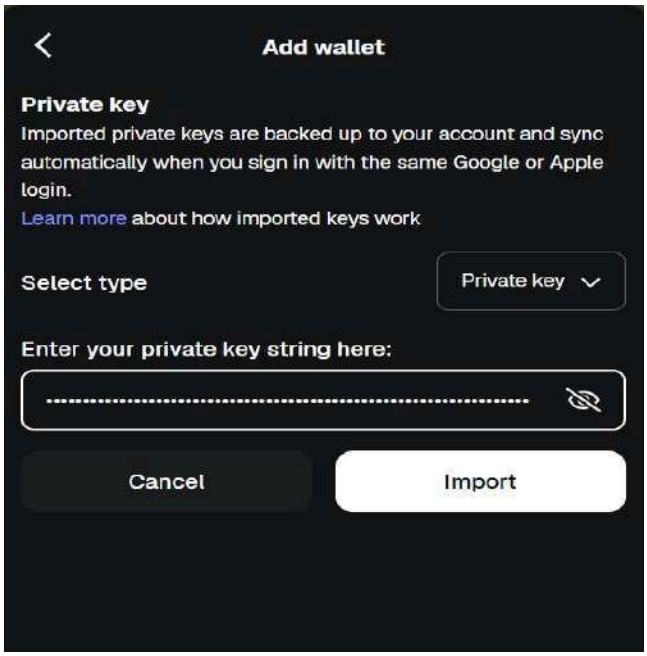
Once installed, open ganache and click on Quickstart Ethereum to start localhost.



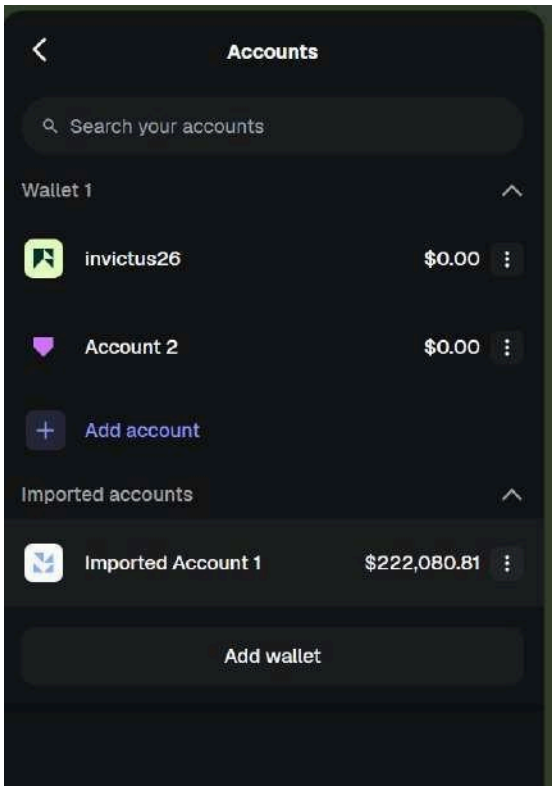
Step 2: In MetaMask, click on the networks tab and add a custom network.

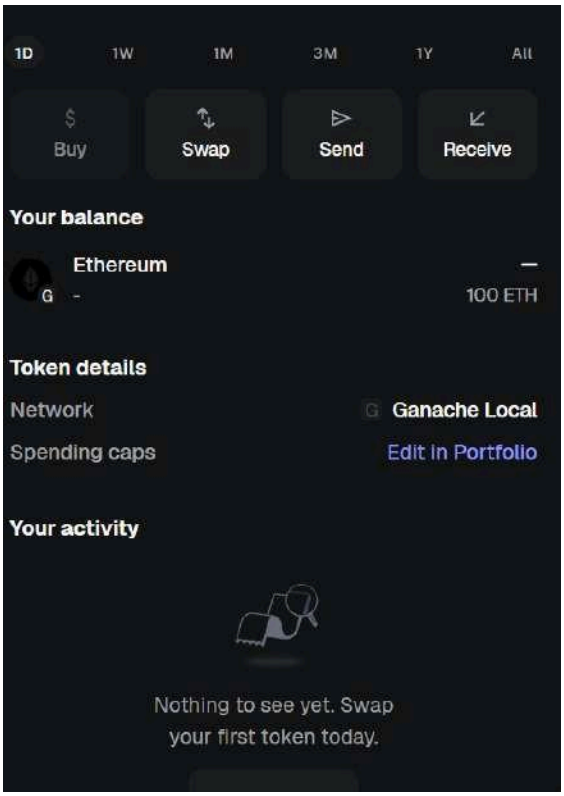


Step 3: Import ganache account into metamask



Wallet balance appears in metaMask account





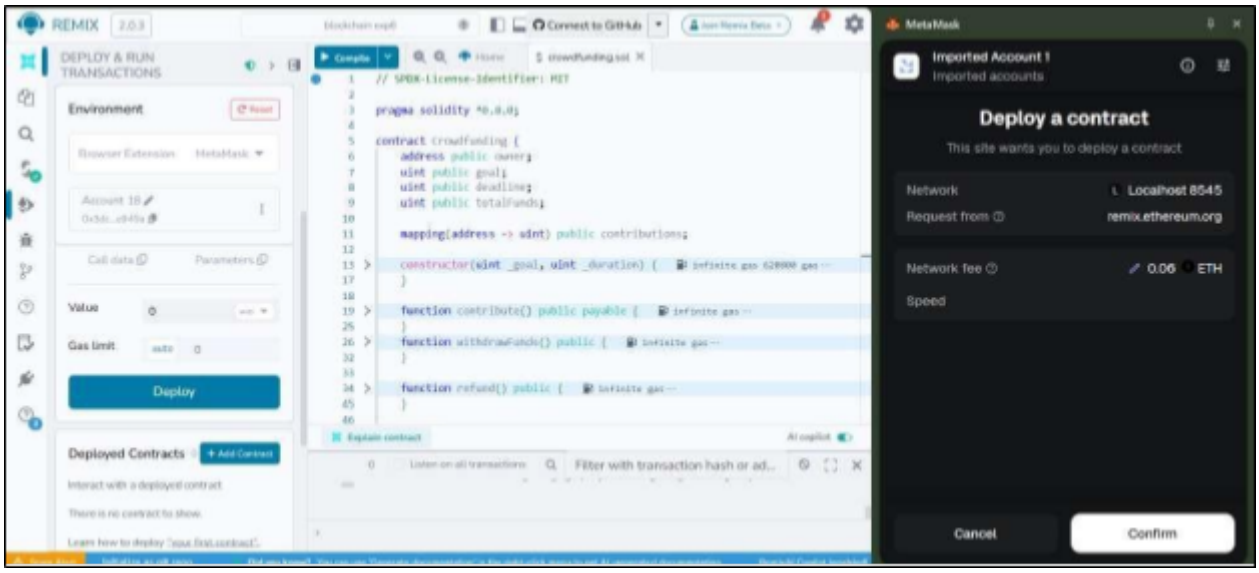
Step 4: In Remix IDE, create a new file **crowdfunding.sol**

Code:

```
// SPDX-License-Identifier:
MIT pragma solidity ^0.8.0;
contract Crowdfunding {
    address public owner;
    uint public goal;
    uint public deadline;
    uint public totalFunds;
    mapping(address => uint) public contributions;

    constructor(uint _goal, uint _duration) {

        owner = msg.sender;
        goal = _goal;
        deadline = block.timestamp + _duration;
```

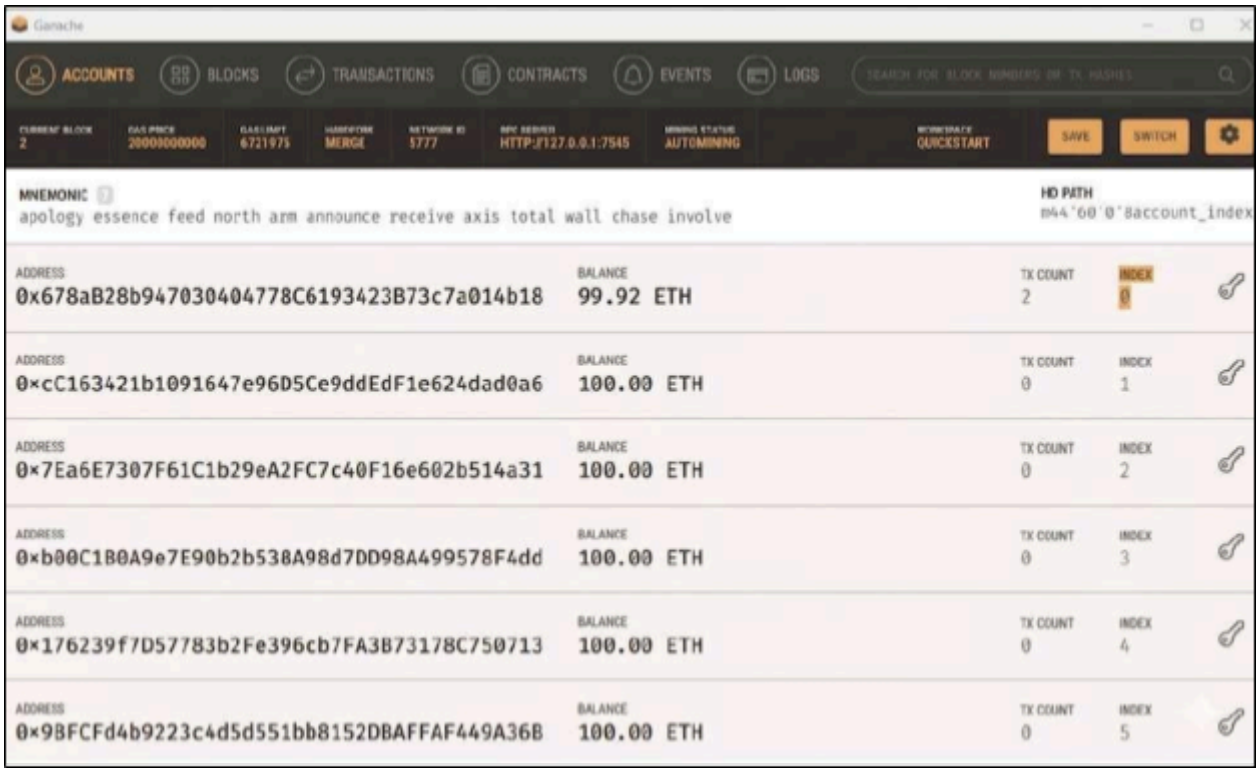
```
    }

    function contribute() public payable {
        require(block.timestamp < deadline, "Deadline passed");
        require(msg.value > 0, "Send some ETH");

        contributions[msg.sender] += msg.value;
        totalFunds += msg.value;
    }
    function withdrawFunds() public {
        require(msg.sender == owner, "Only owner");
        require(totalFunds >= goal, "Goal not reached");

        (bool success, ) = payable(owner).call{value: totalFunds}("");
        require(success, "Transfer failed");
    }

    function refund() public {
        require(block.timestamp > deadline, "Not ended");
        require(totalFunds < goal, "Goal was reached");

        uint amount = contributions[msg.sender];
        require(amount > 0, "No contribution");

        contributions[msg.sender] = 0;

        (bool success, ) = payable(msg.sender).call{value: amount}("");
        require(success, "Refund failed");
    }

    function getBalance() public view returns (uint) {
        return address(this).balance;
    }
}
```

Step 5: Compile and run the contact.
When prompted in metamask, confirm the transaction.



The contract is deployed!

The deducted amount can be seen in ganache app along with transaction count



Conclusion:

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with **Metamask** and **Remix**, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Ansh Sarfare **D20A - 51**

BC Experiment - 9

AIM: Implement the Private Ethereum Blockchain using Geth.

THEORY:

Introduction to Ethereum

Ethereum is an open-source, decentralized blockchain platform that goes beyond simple cryptocurrency transactions by supporting programmable smart contracts and decentralized applications (DApps). It was proposed by Vitalik Buterin in 2013 and launched in 2015. Unlike Bitcoin which is primarily a digital currency, Ethereum functions as a programmable distributed computing platform where developers can deploy self-executing code called smart contracts. The native cryptocurrency of the Ethereum network is called Ether (ETH), which is used to pay for computational operations on the network, referred to as gas fees.

Ethereum maintains a global state that is updated with every transaction and smart contract execution. Every node in the network holds a copy of this state, ensuring decentralization and fault tolerance. The network uses an account-based model where each account has a balance and can interact with other accounts or smart contracts.

What is a Private Ethereum Network?

A private Ethereum network is a standalone blockchain that replicates the Ethereum protocol but operates in complete isolation from the public mainnet. Organizations, developers, and academic institutions use private networks to test and develop blockchain applications without incurring real transaction costs or exposing data to the public chain. All network parameters such as consensus mechanism, block time, gas limits, and initial account balances are fully customizable. This makes private networks ideal for controlled experimentation, proof-of-concept development, and educational purposes.

What is Geth (Go-Ethereum)?

Geth, short for Go-Ethereum, is the most widely used official implementation of the Ethereum protocol, written in the Go programming language. It allows a computer to function as a full Ethereum node capable of mining Ether, validating transactions, executing smart contracts, and interacting with the blockchain. Geth can be operated through a command-line interface, a built-in JavaScript console, or via JSON-RPC API calls. It supports both public Ethereum networks as well as custom private networks, making it the preferred tool for setting up and managing private Ethereum deployments.

Consensus Mechanism — Clique (Proof of Authority)

Private Ethereum networks typically use the Clique consensus algorithm instead of Proof of Work. Clique is a Proof of Authority (PoA) protocol where only a set of pre-approved accounts called signers are authorized to produce and validate new blocks. Since block production does not involve solving computationally intensive puzzles, Clique is significantly faster and more energy efficient than traditional mining. The list of authorized signers is defined at the time of network creation and is embedded in the genesis block. This makes Clique well suited for private and consortium blockchain environments where participants are known and trusted.

Steps Involved in Implementing a Private Ethereum Network

1. **Installing Geth:** The first step involves downloading and installing the appropriate version of the Geth client on the system. Once installed, the binary is made accessible via the system PATH so it can be executed from any directory.

2. **Creating Node Accounts:** Each participating node requires an Ethereum account which consists of a public address and a corresponding private key stored in a keystore file. These accounts are created using the `geth account new` command and are protected by a password. The public addresses of these accounts are used in the genesis configuration.

3. **Setting Up Password Files:** To allow nodes to automatically unlock their accounts during startup without manual password entry each time, password files are created containing the account passwords in plain text.

4. **Creating the Genesis Block:** The genesis block is configured through a `genesis.json` file. This file defines the chain ID, consensus settings (Clique period and epoch), gas limit, initial Ether balances for each account, and the extradata field which encodes the authorized signer address. Every node in the network must be initialized with the same genesis file.

5. **Initializing the Nodes:** Each node's data directory is initialized using the `geth init` command along with the genesis file. This sets up the initial blockchain state and prepares the node to begin operating on the private network.

6. **Running the Nodes:** Node 1 is started as the signer node with mining enabled, responsible for sealing and producing new blocks. Node 2 is started as a member node that participates in the network but does not produce blocks. Both nodes are started with the Geth JavaScript console enabled for direct interaction.

7. **Connecting the Nodes:** Once both nodes are running, they are connected to each other using enode URLs. An enode is a unique identifier for a Geth node containing its public key and network address. Node 1's enode is retrieved using `admin.nodeInfo.enode` and then added to Node 2 using `admin.addPeer()`. Successful peering can be verified using `admin.peers`.

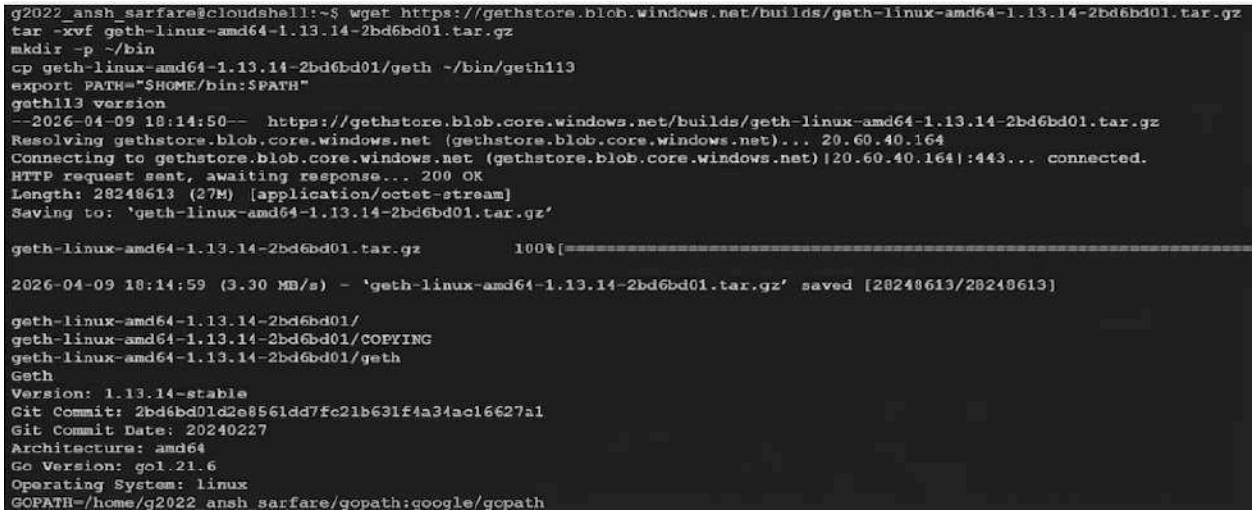
8. **Sending Transactions:** After the network is established and nodes are connected, Ether can be transferred between accounts using `eth.sendTransaction()` in the Geth console. The

transaction is picked up by the signer node, included in a block, and confirmed on the private chain. Account balances can be verified using `eth.getBalance()`.

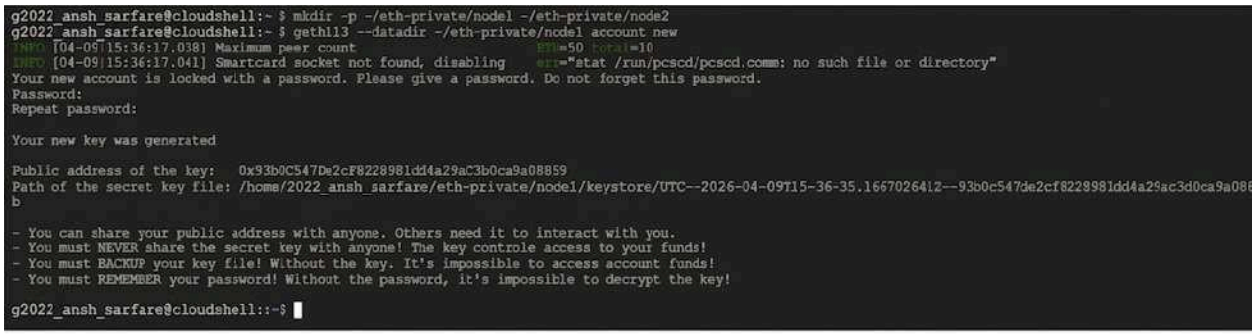
PROCEDURE:

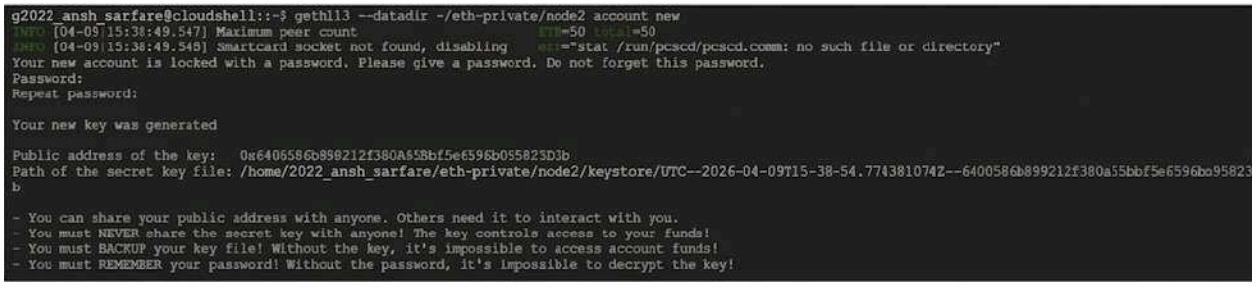
Step 1: Install Geth

wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
tar -xvf geth-linux-amd64-1.13.14-2bd6bd01.tar.gz
mkdir -p ~/bin
cp geth-linux-amd64-1.13.14-2bd6bd01/geth ~/bin/geth113
export PATH="$HOME/bin:$PATH"
geth113 version



Step 2: Create folders and accounts for both nodes





Step 3: Create password files

```
g2022_ansh_sarfare@cloudshell:~$ echo "Ansh" > ~/eth-private/node1/password.txt
echo "Ansh" > ~/eth-private/node2/password.txt
g2022_ansh_sarfare@cloudshell:~$
```

Step 4: Create genesis.json and verify it looks correct

```
NODE1="PASTE_NODE1_ADDRESS_WITHOUT_0x_HERE"
NODE2="PASTE_NODE2_ADDRESS_WITHOUT_0x_HERE"
cat > ~/eth-private/genesis.json << EOF
{
  "config": {
    "chainId": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata":
"0x0000000000000000000000000000000000000000000000000000000000000000${NODE1}0
000000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0x${NODE1}": { "balance": "100000000000000000000" },
    "0x${NODE2}": { "balance": "100000000000000000000" }
  }
}
EOF
```

```
g2022_ansh_sarfare@cloudshell:~$ NODE1="9380C547De2cf82229811dd4a2ac33D0cA9a08859"
NODE2="6406656b899212380A55BbfSe6596ED95522D3b"

cat > ~/eth-private/genesis.json << EOF
{
  "config": {
    "chainid": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
  "gasLimit": "8000000",
  "extradata": "0x0000000000000000000000000000000000000000000000000000000000000000${NODE1}0000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000000000000000000000000000000000000000000000",
  "alloc": {
    "0x${NODE1}": { "balance": "10000000000000000000" },
    "0x${NODE2}": { "balance": "10000000000000000000" }
  }
}
Total=10000000
g2022_ansh_sarfare@cloudshell:~$
```

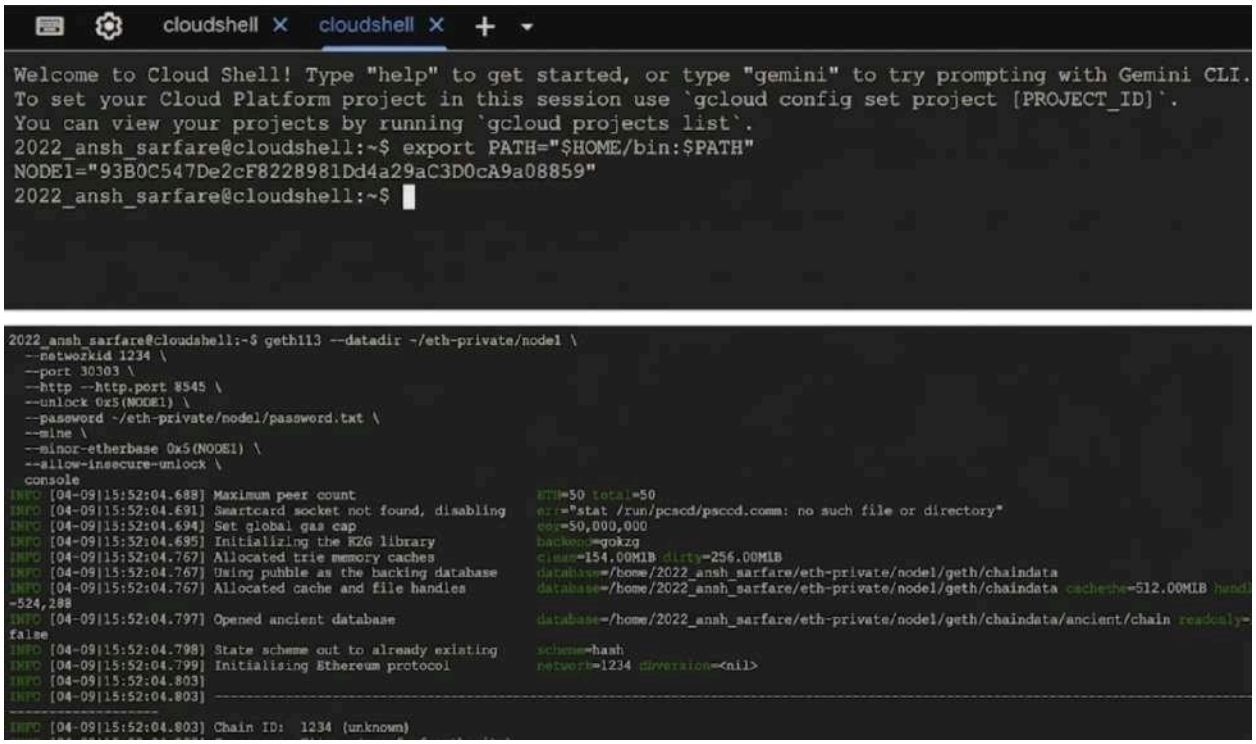
```
g2022_ansh_sarfare@cloudshell:~$  cat ~/eth-private/genesis.json
{
  "config": {
    "chainid": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "clique": {
      "period": 5,
      "epoch": 30000
    }
  },
  "difficulty": "1",
```

Step 5: Initialize both nodes

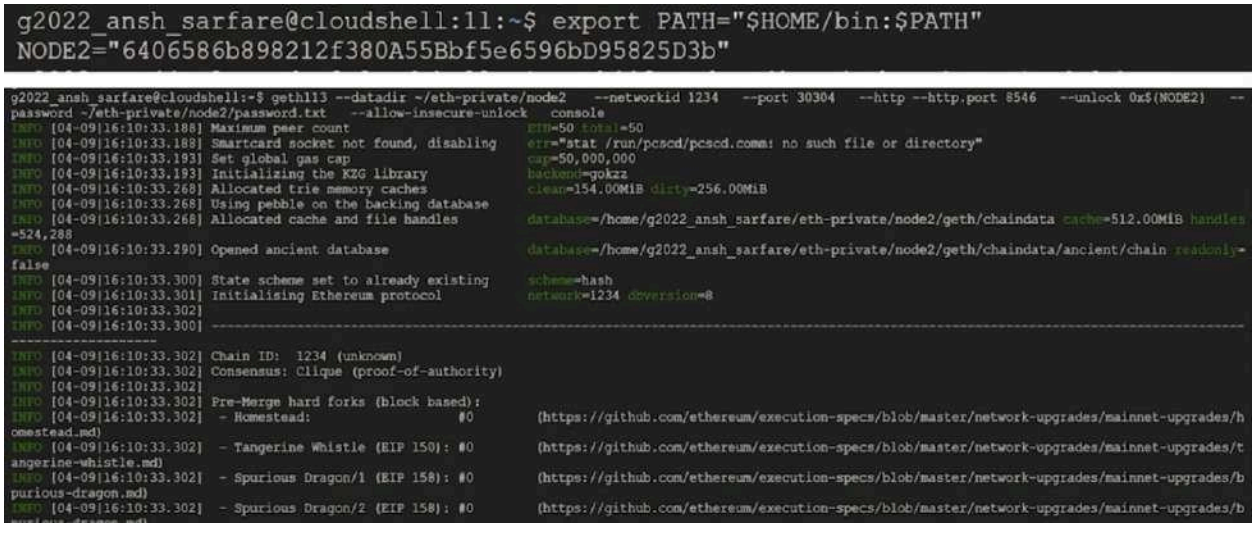
```
g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node1 init ~/eth-private/genesis.json
INFO [04-09|15:49:20.956] Maximum peer count                       ETH=50 total=50
INFO [04-09|15:49:20.959] Smartcard socket not found, disabling    err="Stat /run/pcscd/pcscd.comm: no much file or directory"
INFO [04-09|15:49:20.963] Set global gas cap                       cap=50,000,000
INFO [04-09|15:49:20.963] Initializing the KZG library             backend=gokzg
INFO [04-09|15:49:21.035] Defaulting to pebble as the backing database
INFO [04-09|15:49:21.095] Allocated cache and file handlee         database=/home/g2022_ansh_sarfare/eth-private/node1/geth/chaindata cache=16.00MiB handles=
16
INFO [04-09|15:49:21.104] Opened ancient database                  database=/home/g2022_ansh_sarfare/eth-private/node1/geth/chaindata/ancient/chain readonly=
false
INFO [04-09|15:49:21.105] State schema set to default              scheme=hash
INFO [04-09|15:49:21.105] Writing custom genesis block
INFO [04-09|15:49:21.106] Persisted tile from memory database      nodes=3 size=411.00B time=3.93038ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.0
0B
INFO [04-09|15:49:21.152] Successfully wrote genesis state         database=chaindata hash=e03c67..887468
INFO [04-09|15:49:21.152] Defaulting to pebble as the backing database
INFO [04-09|15:49:21.152] Allocated cache and file handlee         database=/home/g2022_ansh_sarfare/eth-private/node1/geth/lightchaindata cache=16.00MiB han
dles=16
INFO [04-09|15:49:21.219] Opened ancient database                  database=/home/g2022_ansh_sarfare/eth-private/node1/geth/lightchaindata/ancient/chain read
only=false
INFO [04-09|15:49:21.219] State schema set to default              scheme=hash
INFO [04-09|15:49:21.219] Writing custom genesis block
INFO [04-09|15:49:21.223] Persisted tile from memory database      nodes=3 size=411.00B time=4.12979ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.0
0B
INFO [04-09|15:49:21.237] Successfully wrote genesis state         database=lightchaindata hash=e03c67..887468
g2022_ansh_sarfare@cloudshell:~$
```

```
g2022_ansh_sarfare@cloudshell:~$ geth113 --datadir ~/eth-private/node1 init ~/eth-private/genesis.json
INFO [04-09|15:49:52.109] Maximum peer count                       ETH=50 total=10
INFO [04-09|15:49:52.111] Smartcard socket not found, disabling    err="Stat /run/gcood/pcscd.ccmm: no much file or directory"
INFO [04-09|15:49:52.115] Set global gas cap                       cap=50,000,000
INFO [04-09|15:49:52.118] Initializing the KZG library             backend=gokzg
INFO [04-09|15:49:52.189] Defaulting to pebble as the backing database
INFO [04-09|15:49:52.189] Allocated cache and file handles         database=/home/g2022_ansh_sarfare/eth-private/node1/geth/chaindata cache=16.00MiB handles=
16
INFO [04-09|15:49:52.257] Opened ancient database                  database=/home/g2022_ansh_sarfare/eth-private/node2/geth/chaindata/ancient/chain readonly=
false
INFO [04-09|15:49:52.257] State schema set to default              scheme=hash
INFO [04-09|15:49:52.257] Writing custom genesis block
INFO [04-09|15:49:52.262] Persisted tsie from memory database      nodes=3 size=411.00B time=4.507603ms gcnodes=0 gcsize=0.00B gctime=0s livenodes=0 livesize=0.
00B
INFO [04-09|15:49:52.306] Successfully wrote genesis state         database=chaindata hash=e03c67..887468
INFO [04-09|15:49:52.306] Defaulting to pebble as the backing database
```

Step 6: Run Node 1 (signer)

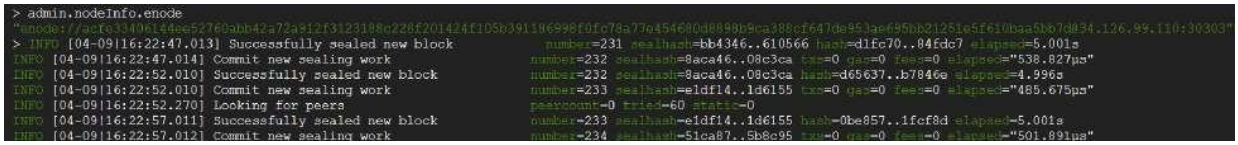


Step 7: Open new terminal tab and run Node 2 (member)

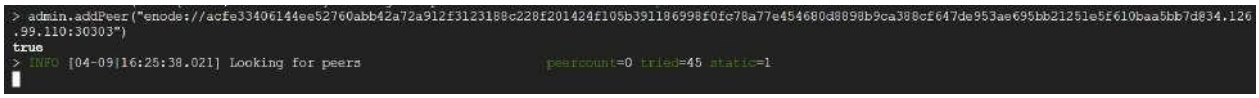


Step 8: Connect the two nodes

In Node 1 console, get its enode



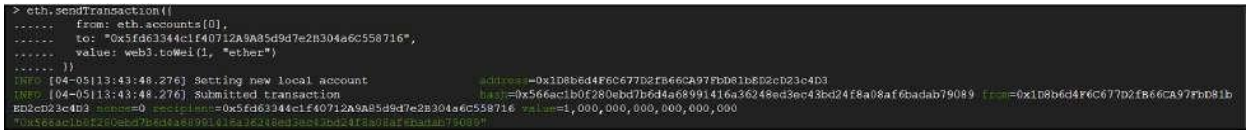
Now, switch to Node 2 console and paste the Node 1 enode



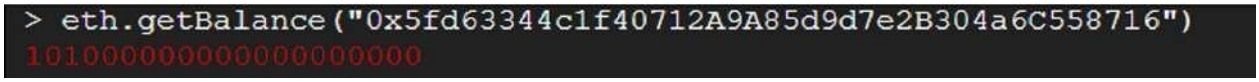
In Node 2 console, check they are connected

Step 9: Send a transaction

In Node 1 console, send transaction to Node 2



and check the balance of Node 2



CONCLUSION

This experiment successfully demonstrated the implementation of a private Ethereum blockchain network using the Geth (Go-Ethereum) client. A two-node private network was set up from scratch, beginning with the installation of Geth, creation of individual node accounts, and configuration of the genesis block using the Clique Proof of Authority consensus mechanism. Both nodes were initialized with the same genesis file, started with appropriate network parameters, and successfully connected to each other using enode URLs.

Once the network was operational, the signer node (Node 1) began producing and sealing blocks at regular intervals, confirming that the consensus mechanism was functioning correctly. The two nodes were peered together and their connection was verified using the `admin.peers` command. A transaction was then executed from Node 1 to Node 2 using `eth.sendTransaction()`, and the updated balance of Node 2 was confirmed using `eth.getBalance()`, validating that transactions are being processed and recorded on the private chain.

Through this experiment, a practical understanding of how Ethereum nodes operate, how private blockchain networks are configured, and how peer-to-peer communication works

between nodes was gained. It also highlighted the advantages of using Proof of Authority consensus in private networks, particularly in terms of faster block production and reduced computational overhead compared to Proof of Work. Overall, this experiment provided hands-on exposure to core blockchain concepts including genesis block configuration, node initialization, peer discovery, and on-chain transaction verification.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Ansh Sarfare **D20A - 51**

BC Experiment - 10

Aim: Explore the Cryptocurrency Landscape

Theory:

This experiment explores the **cryptocurrency landscape** by combining real-world market data analysis with blockchain technology research. The focus asset for this analysis is **Chainlink (LINK)**, and data was collected from **three reputable sources** as per the lab requirement:

1. **CoinGecko API** – Provided historical price data, market capitalization, and trading volumes in a structured JSON format, enabling direct processing in Python.
2. **CoinMarketCap** – Offered real-time and historical price charts, circulating supply, and market rank for visual cross-verification.
3. **Crypto.com** – Supplied quick-access market statistics, simplified price trends, and cryptocurrency news updates.

Cryptocurrency Landscape

1. **Definition** – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
2. **Market Diversity** – Thousands of cryptocurrencies exist, each serving unique roles:
    - **Store of Value** – Bitcoin, Litecoin.
    - **Utility Tokens** – Chainlink, Ethereum.
    - **Stablecoins** – USDT, USDC.
    - **Governance Tokens** – UNI, AAVE.
3. **Volatility** – Prices are highly sensitive to market sentiment, technological developments, and regulatory updates.
4. **Data Analysis Importance** – Tracking historical trends, market capitalization, and trading volume is essential for understanding asset performance.
5. **Advancements** – Growing use of **DeFi platforms**, **NFT ecosystems**, **layer-2 scaling**, and **cross-chain protocols**.

Data Collection Process

The program used the **CoinGecko API** to automate historical data fetching for Chainlink. The retrieved data was processed with **Pandas** to compute:

- **Daily Returns** – Percentage change between consecutive days to measure volatility.
- **Moving Averages (MA7, MA30)** – Short-term and long-term trend indicators.
- **Cumulative Returns** – Growth factor over time.
- **Rolling Volatility** – Short-term risk measurement based on daily return fluctuations.

Visualization & Dashboard

Using **Matplotlib**, a six-panel dashboard was created to display:

1. **Price Trend** – Daily price movement over the selected period.
2. **Price vs Volume** – Correlation between trading activity and price changes.
3. **Daily Returns Distribution** – Statistical spread of daily gains/losses.

4. **Moving Averages** – Technical analysis trend lines for price movement.
5. **Cumulative Returns** – Overall growth performance.
6. **Rolling Volatility** – Market risk trends over time.

Technological Analysis

Beyond market data, the study covered **advancements in blockchain technology**, including:

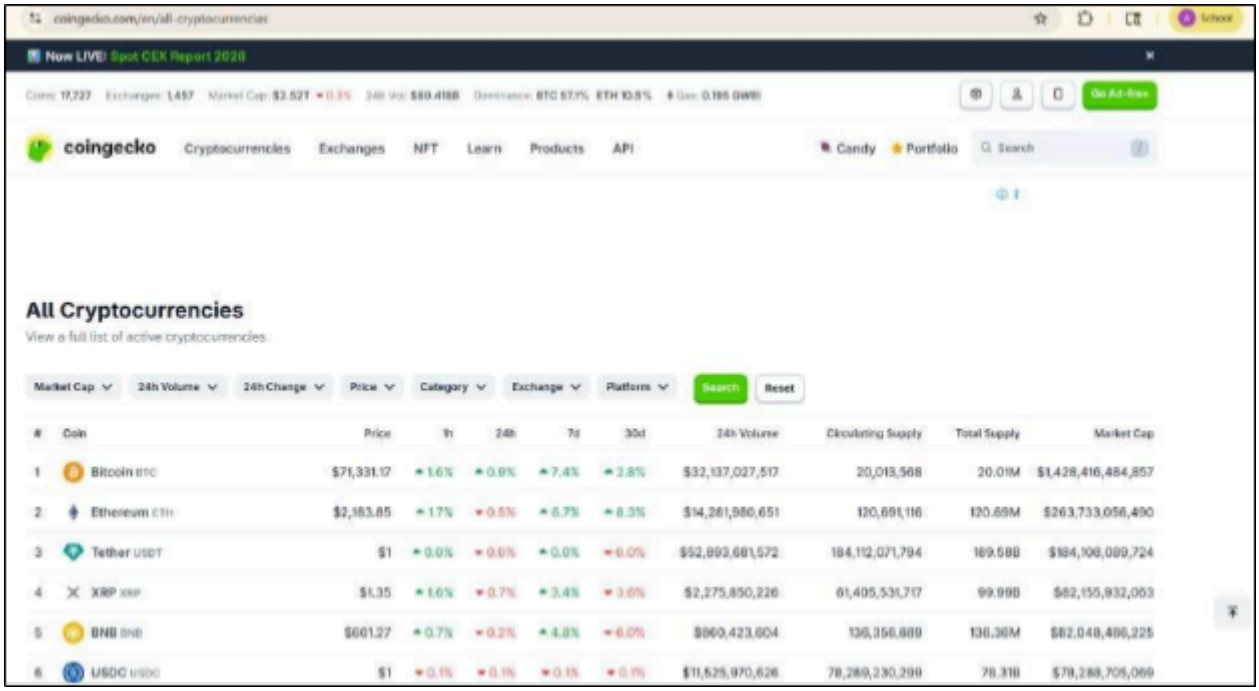
- **Consensus Mechanisms** – PoW (Proof of Work), PoS (Proof of Stake), and hybrid systems.
- **Scalability Solutions** – Layer-2 protocols, sharding, and sidechains to increase throughput.
- **Interoperability Protocols** – Chainlink's **CCIP**, Polkadot, and Cosmos enabling cross-chain data exchange.
- **Privacy Features** – Zero-knowledge proofs (zk-SNARKs) and ring signatures to ensure transaction confidentiality.

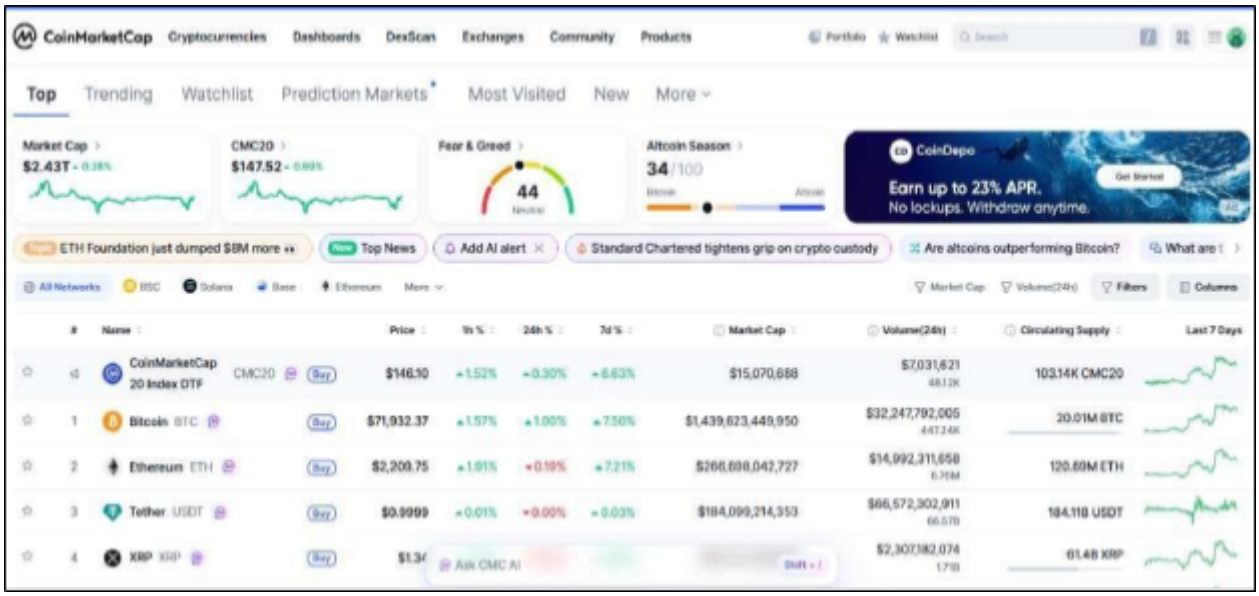
Chainlink's **roadmap** highlights staking implementation, advanced oracle features, and enhanced cross-chain capabilities, aligning with industry trends toward **decentralization, interoperability, and security**.

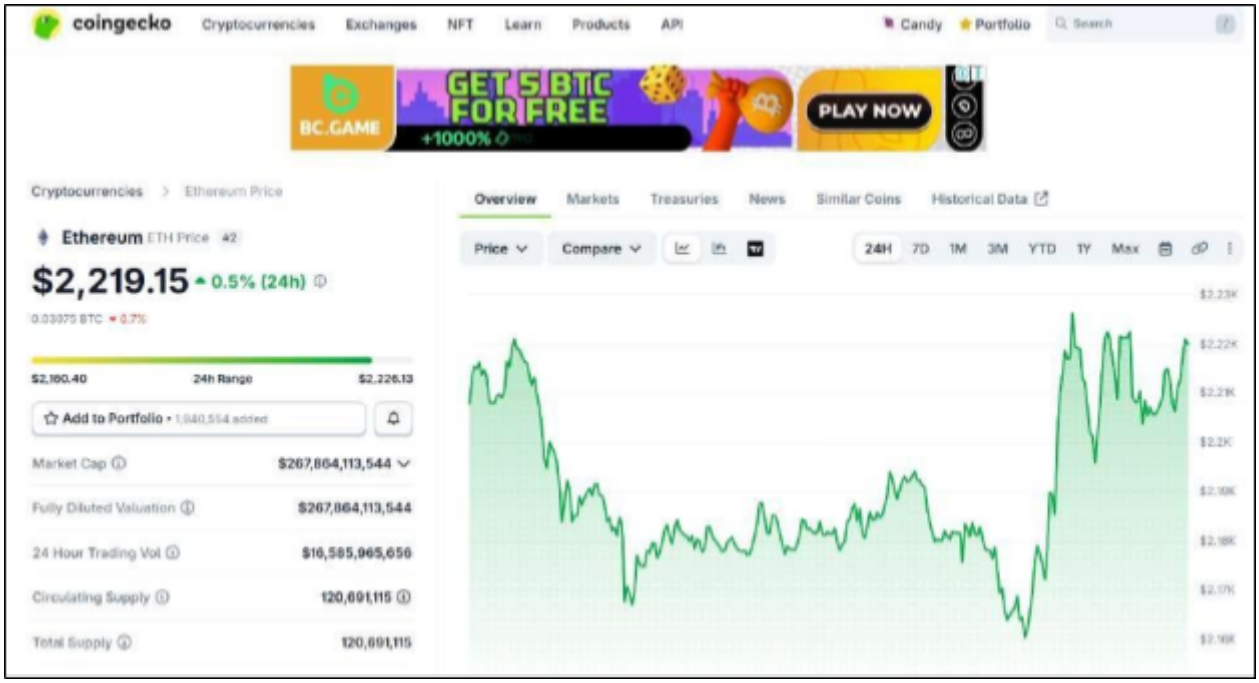
Market Trends & Adoption

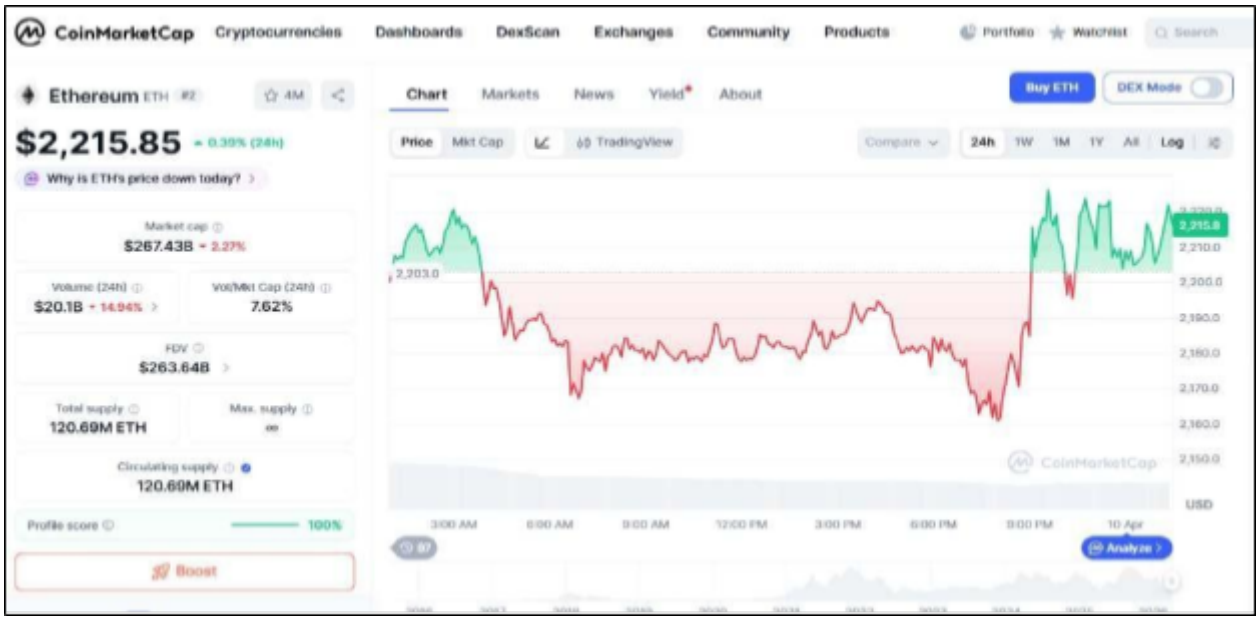
- **Growing Merchant Acceptance** – More businesses accepting cryptocurrency payments.
- **Wallet Growth** – Increasing downloads and installations of crypto wallets.
- **Network Activity** – Higher transaction volumes and active addresses, signaling strong adoption.
- **Integration with Traditional Finance** – Partnerships between crypto platforms and fintech/payment providers.
- **Regulatory Developments** – Countries exploring frameworks for safe cryptocurrency adoption.

Implementation




CoinMarketCap
Cryptocurrencies
Dashboards
DexScan
Exchanges
Community
Products
Top
Trending
Watchlist
Prediction Markets
Most Visited
New
More
Market Cap
$2.43T
CMC20
$147.52
Fear & Greed
44
Neutral
Altcoin Season
34/100
CoinDepo
Earn up to 23% APR.
No lockups. Withdraw anytime.
Get Started
ETH Foundation just dumped $8M more
Top News
Add AI alert
Standard Chartered tightens grip on crypto custody
Are altcoins outperforming Bitcoin?
Name
Price
1h %
24h %
7d %
Market Cap
Volume(24h)
Circulating Supply
Last 7 Days
CoinMarketCap 20 Index DTF
CMC20
$146.10
$15,070,688
$7,031,621
103.14K CMC20
Bitcoin BTC
$71,932.37
$1,439,623,449,950
$32,247,792,005
20.01M BTC
Ethereum ETH
$2,209.75
$266,698,042,727
$14,992,311,658
120.69M ETH
Tether USDT
$0.9999
$184,099,214,353
$66,572,302,911
184.11B USDT
XRP XRP
$2,307,182,074
61.4B XRP
Ask CMC AI


coingecko
Cryptocurrencies
Exchanges
NFT
Learn
Products
API
Candy
Portfolio
BC.GAME
GET 5 BTC FOR FREE
+1000%
PLAY NOW
Cryptocurrencies
Ethereum Price
Ethereum ETH Price
$2,219.15
0.5% (24h)
0.03075 BTC
$2,160.40
24h Range
$2,226.13
Add to Portfolio
Market Cap
$267,864,113,544
Fully Diluted Valuation
$267,864,113,544
24 Hour Trading Vol
$16,585,965,656
Circulating Supply
120,691,115
Total Supply
120,691,115
Overview
Markets
Treasuries
News
Similar Coins
Historical Data
Price
Compare
24H
7D
1M
3M
YTD
1Y
Max



Utilizing Api for analysis and visualization :
Code:

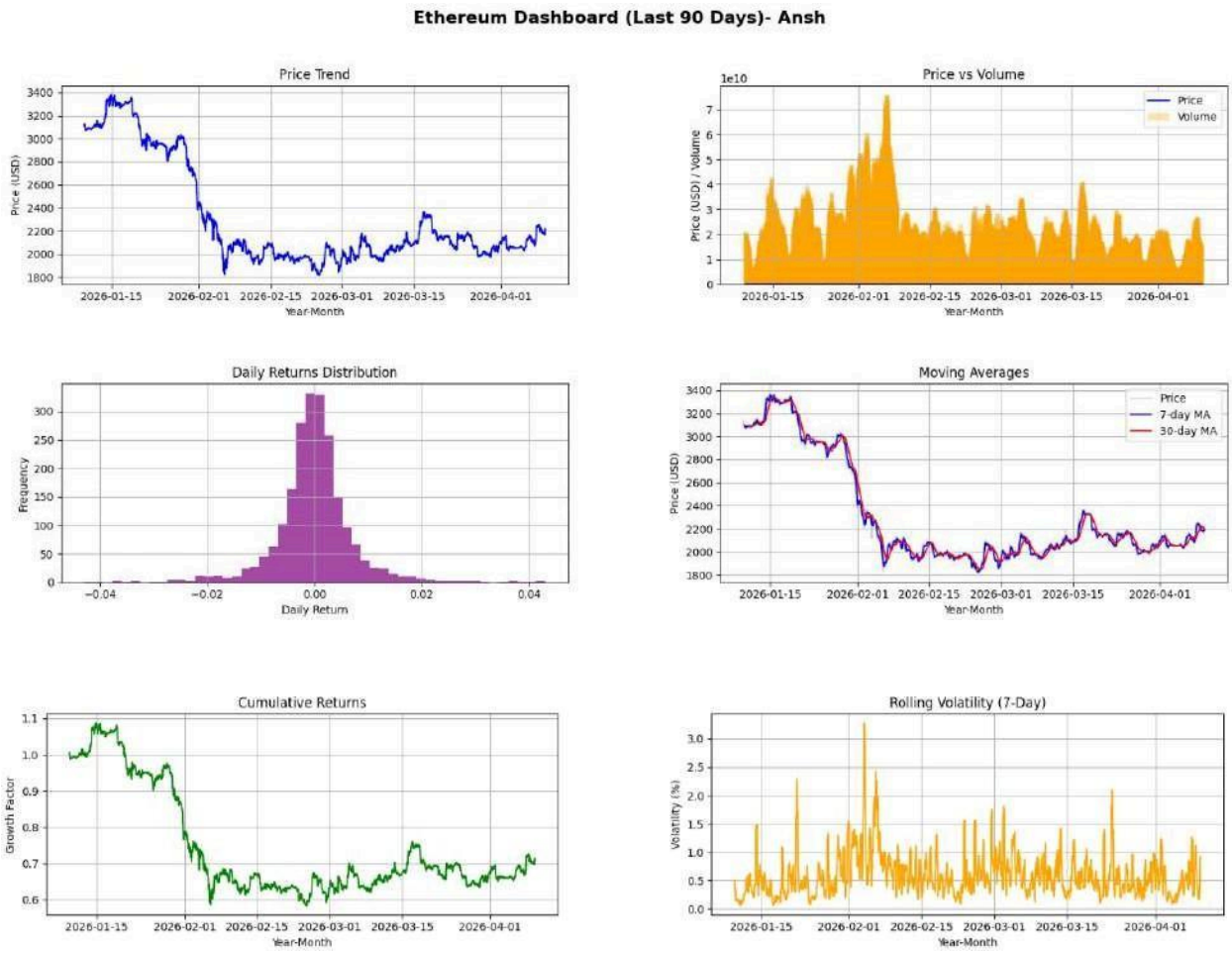
```
import requests

import matplotlib.pyplot as plt

import pandas as pd


# ------- Fetch Data -------

def get_historical_data(coin_id, days='180', currency='usd'):

    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'

    params = {'vs_currency': currency, 'days': days}

    response = requests.get(url, params=params)

    return response.json()


# ---------- Dashboard Function

----------def

ethereum_dashboard(days='180'):

    coin_id = 'ethereum'

    data = get_historical_data(coin_id, days)


    # Create DataFrame
```

```
prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

df = pd.merge(prices_df, volumes_df, on='timestamp')
df.set_index('timestamp', inplace=True)

# Calculate extra metrics
df['returns'] = df['price'].pct_change()
df['MA7'] = df['price'].rolling(window=7).mean()
df['MA30'] = df['price'].rolling(window=30).mean()
df['cumulative'] = (1 + df['returns']).cumprod()
df['volatility'] = df['returns'].rolling(window=7).std() * 100

# ---------- Plot Dashboard ----------
fig, axes = plt.subplots(3, 2, figsize=(16, 12))
fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

# 1. Price Trend
axes[0, 0].plot(df.index, df['price'], color='blue')
axes[0, 0].set_title('Price Trend')
axes[0, 0].set_ylabel('Price (USD)')
axes[0, 0].set_xlabel('Year-Month') # Add xlabel
axes[0, 0].grid(True)

# 2. Price vs Volume
axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
```

```
axes[0, 1].set_title('Price vs Volume')
axes[0, 1].set_xlabel('Year-Month') # Add xlabel
axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)

# 3. Daily Returns Distribution
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True)

# 4. Moving Averages
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
axes[1, 1].set_title('Moving Averages')
axes[1,  1].set_xlabel('Year-Month')
axes[1, 1].set_ylabel('Price (USD)')
axes[1, 1].legend()
axes[1, 1].grid(True)

# 5. Cumulative Returns
axes[2, 0].plot(df.index, df['cumulative'], color='green')
axes[2, 0].set_title('Cumulative Returns')
axes[2, 0].set_ylabel('Growth Factor')
axes[2, 0].set_xlabel('Year-Month')
axes[2, 0].grid(True)
# 6. Volatility
```

```
    axes[2, 1].plot(df.index, df['volatility'], color='orange')
    axes[2, 1].set_title('Rolling Volatility (7-Day)')
    axes[2, 1].set_ylabel('Volatility (%)')
    axes[2, 1].set_xlabel('Year-Month')
    axes[2, 1].grid(True)

    plt.tight_layout(rect=[0, 0, 1, 0.96])
    plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
    plt.show()
# ---------- Run Dashboard
----------ethereum_dashboard('90')
```

Output:



Conclusion:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801 : Blockchain & DLT Lab

Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

Ansh Sarfare **D20A - 51**

BC Experiment-11

Title:

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim:

To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Prerequisites:

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

Procedure:

Step 1: Update the System

Command:

sudo apt update
sudo apt upgrade -y

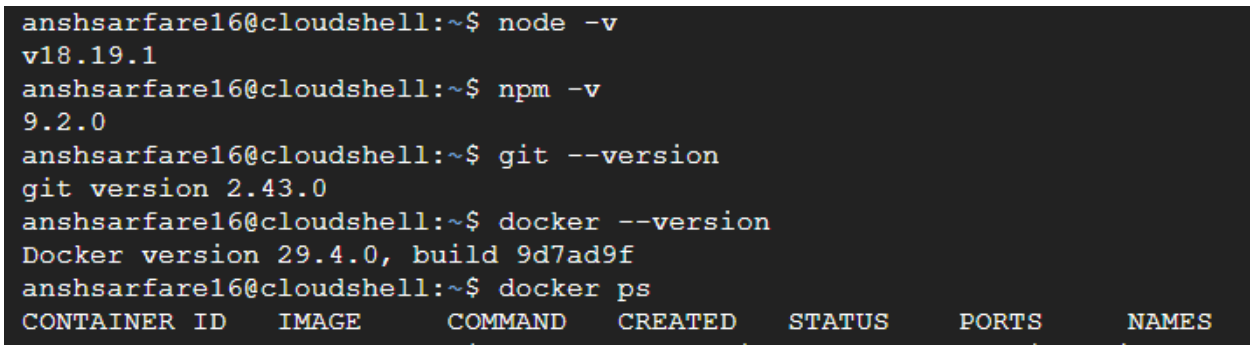
```
anshsarfare16@cloudshell:~$ sudo apt update
Hit:1 https://download.docker.com/linux/ubuntu noble InRelease
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:3 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2,377 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2,155 kB]
Hit:9 https://packages.cloud.google.com/apt gcsfuse-noble InRelease
Hit:10 https://packages.cloud.google.com/apt cloud-sdk InRelease
Hit:11 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease
Get:12 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:13 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Fetched 4,914 kB in 6s (778 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
12 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Explanation:
This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

sudo apt install jq curl git nodejs npm -y
node -v
npm -v
git --version
docker
--version
docker ps



Explanation:
This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while `docker ps` ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples

```
anshsarfare16@cloudshell:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (238/238), done.
remote: Compressing objects: 100% (129/129), done.
remote: Total 15618 (delta 161), reused 109 (delta 109), pack-reused 15380 (from 3)
Receiving objects: 100% (15618/15618), 24.07 MiB | 17.07 MiB/s, done.
Resolving deltas: 100% (8809/8809), done.
anshsarfare16@cloudshell:~$ cd fabric-samples
```

Explanation:
This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```
anshsarfare16@cloudshell:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/instal
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.g
===> Will unpack to: /home/anshsarfare16/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0  29.5M      0  0:00:04  0:00:04 --:--:-- 33.3M
==> Done.
===> Downloading version 1.5.17 platform specific fabric-ca-client binary
===> Downloading:  https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17
===> Will unpack to: /home/anshsarfare16/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 31.6M  100 31.6M    0     0  16.2M      0  0:00:01  0:00:01 --:--:-- 36.4M
==> Done.

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
====>  ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
d41d93850539: Pull complete
b1cba2e842ca: Pull complete
10a7c8361220: Pull complete
f8c55d757df2: Pull complete
eb2bee6aa5bd: Pull complete
```

```
Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
====>  ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====>  ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====>  ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====>  ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
===> Pulling fabric ca Image
====>  ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
===> List out hyperledger images
```

Explanation:
This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

docker images | grep fabric

```
anshsarfare16@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15     654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15      86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15    9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15       3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5                654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15             654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest             654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                    453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                 453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                 453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                 86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15              86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest              86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5               9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15            9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest            9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                  3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15               3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest               3e25adc31a75        230MB         66.2MB
```

Explanation:
This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:

export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version

```
anshsarfare16@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
anshsarfare16@cloudshell:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
anshsarfare16@cloudshell:~/fabric-samples$ peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

Explanation:
This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running `peer version` verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:

cd ~/fabric-samples/test-network
./network.sh up createChannel

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-composeic-samples/test-network$ ./network.sh up createChannel
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using d
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/anshsarfare16/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] Running 3/7
 ⠿ Network fabric_test                      Creat...                    1.7s
 ⠿ Volume "compose_peer0.org1.example.com"  Created                     1.5s
 ⠿ Volume "compose_peer0.org2.example.com"  Created                     1.5s
 ⠿ Volume "compose_orderer.example.com"     Created                     1.5s
 ✓ Container orderer.example.com            Started                     1.4s
```

```
Status: 201
{
        "name": "mychannel",
        "url": "/participation/v1/channels/mychannel",
        "consensusRelation": "consenter",
        "status": "active",
        "height": 1
}

Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:34.429 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:34.463 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Joining org2 peer to the channel...
Using organization 2
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-09 17:35:37.555 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.592 UTC 0002 INFO [channelCmd] executeJoin -> Successfully submitted pro
Setting anchor peer for org1...
Using organization 1
Fetching channel config for channel mychannel
Using organization 1
Fetching the most recent configuration block for the channel
++ peer channel fetch config /home/anshsarfare16/fabric-samples/test-network/channel-artifac
tls --cafile /home/anshsarfare16/fabric-samples/test-network/organizations/ordererOrganizati
2026-04-09 17:35:37.685 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer co
2026-04-09 17:35:37.690 UTC 0002 INFO [cli.common] readBlock -> Received block: 0
2026-04-09 17:35:37.691 UTC 0003 INFO [channelCmd] fetch -> Retrieving last config block: 0
2026-04-09 17:35:37.693 UTC 0004 INFO [cli.common] readBlock -> Received block: 0
```

Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers.

It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

./network.sh deployCC \
 -ccn basic \
 -ccp ../asset-transfer-basic/chaincode-javascript \
 -ccl javascript

```
anshsarfare16@cloudshell:~/fabric-samples/test-network$ ./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/ch
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
```

Explanation:
This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

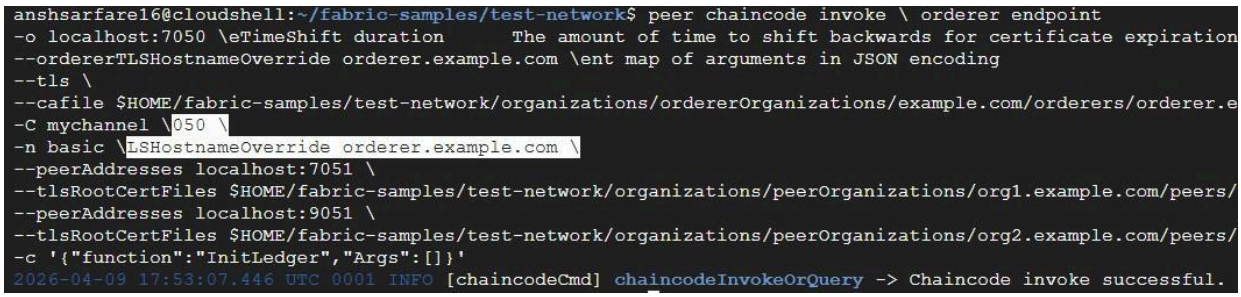
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

Explanation:
This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

peer chaincode invoke ... InitLedger

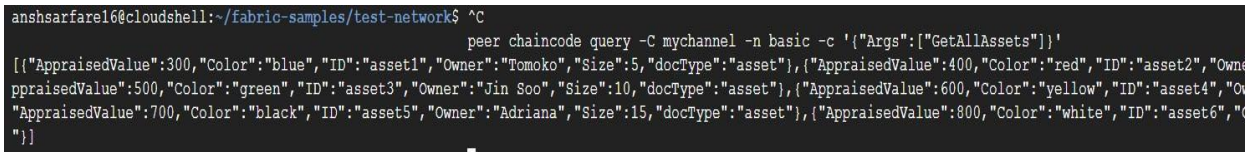


Explanation:
This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets

Command:

peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

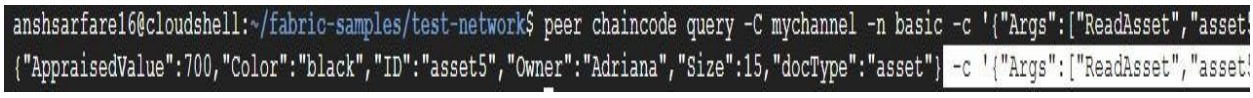


Explanation:
This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset

Command:

peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'



Explanation:
This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

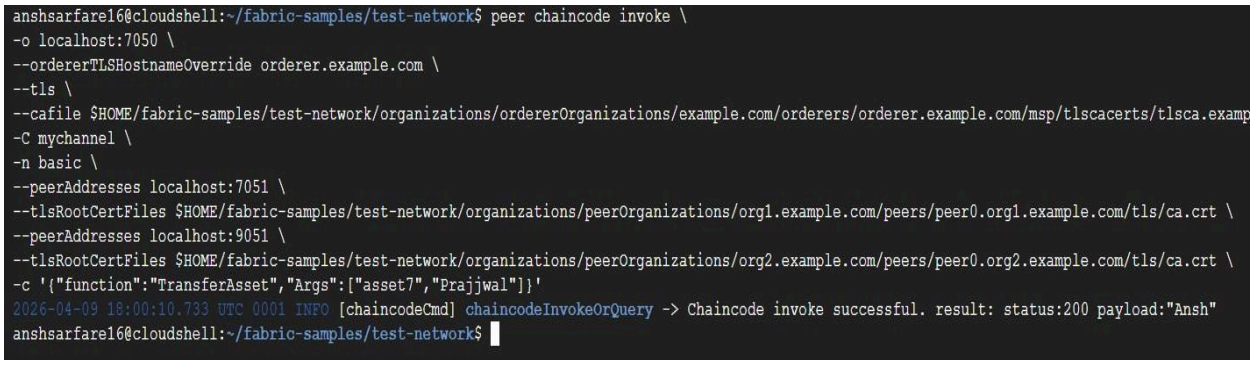
peer chaincode invoke ... CreateAsset (Owner: Manav)

Explanation:
This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

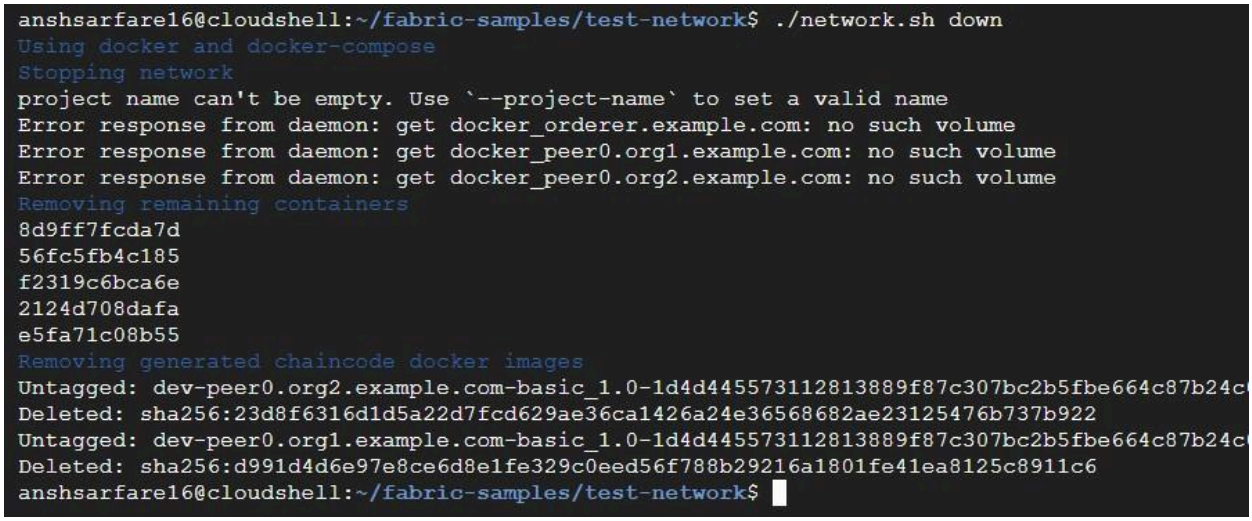


Explanation:
This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

./network.sh down



Explanation:
This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Result:

The Hyperledger Fabric network was successfully implemented. Chaincode was deployed and asset operations such as creation, querying, and transfer were performed successfully. The asset was created with owner name **Manav**, confirming correct execution.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 12

Aim: Mini Project

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Hyperledger Fabric Crowdfunding Platform

Mini Project Presentation

Project Type: Decentralized Crowdfunding on Blockchain

Network: 4-Organization Hyperledger Fabric (v2.5)

Date: March 2026

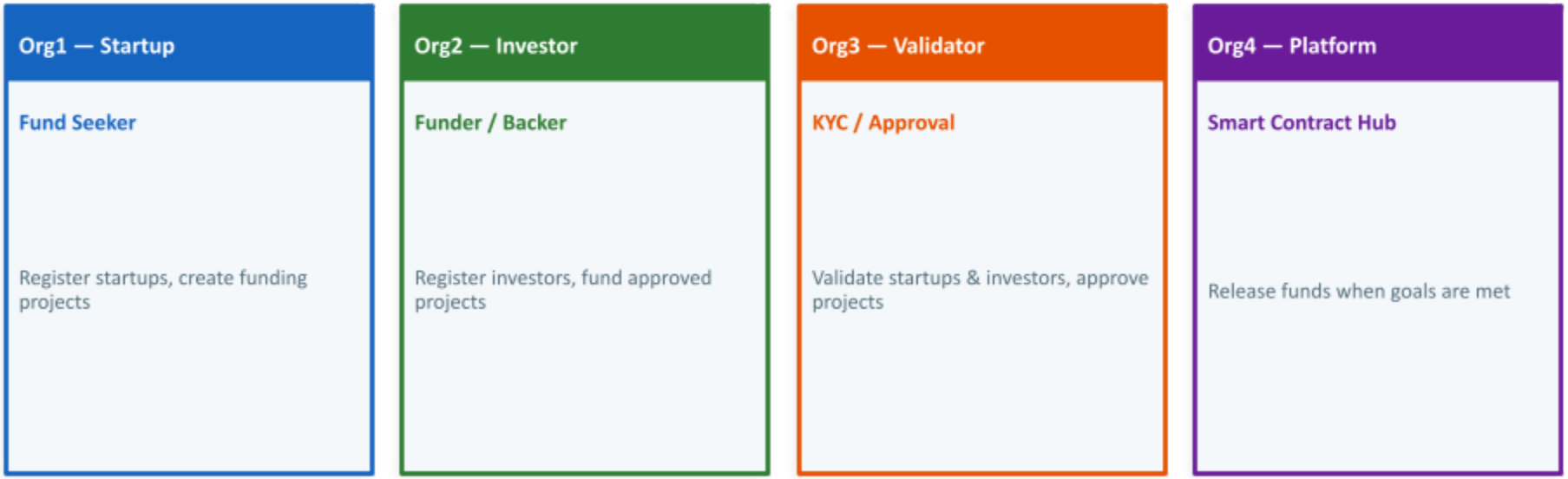
Group 4 – INFT – VESIT

Shraeyaa Dhaigude – 22 Prajjwal Pandey – 37 Ansh Sarfare – 51 Mahvish Siddiqui - 58

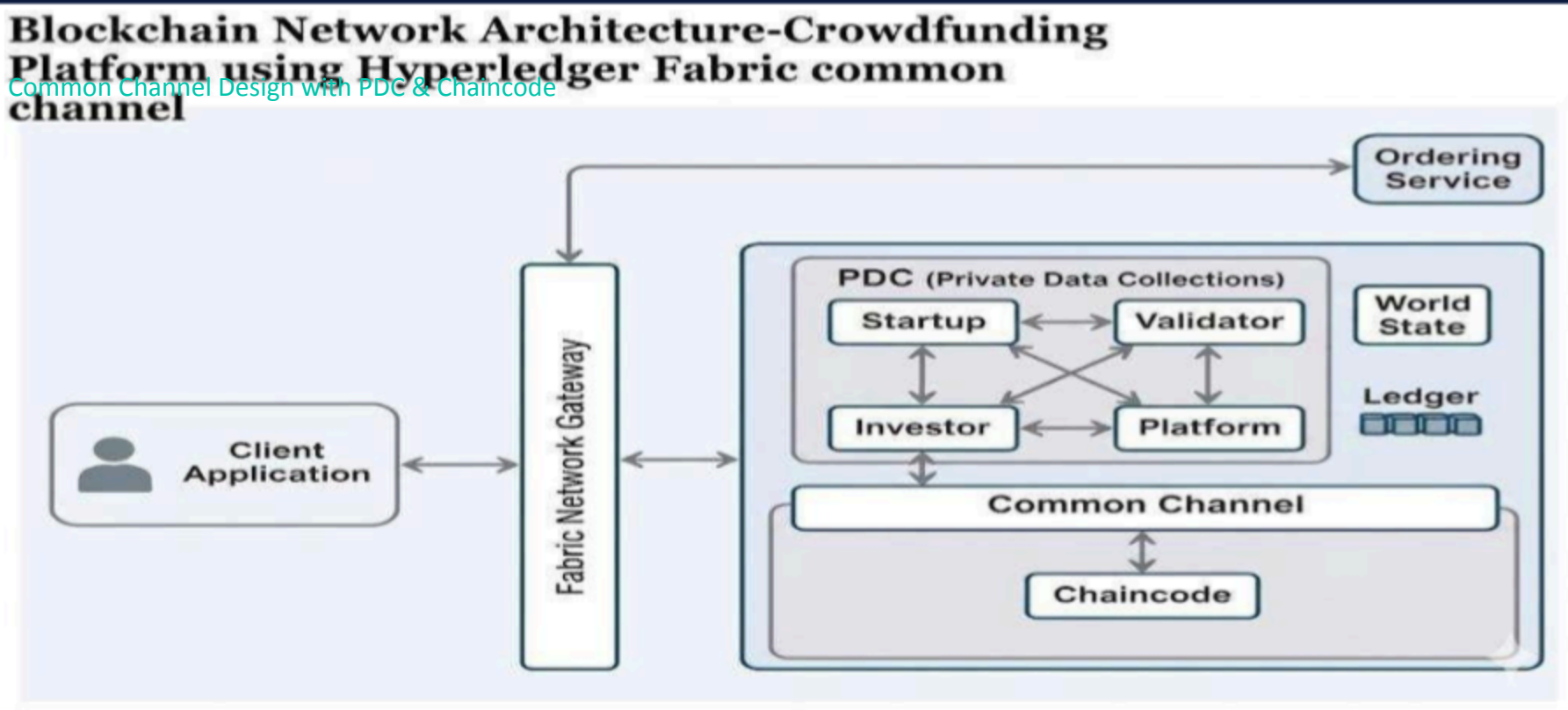
Project Overview

What this project does

A decentralized crowdfunding platform built on Hyperledger Fabric, enabling startups to raise funds from verified investors with multi-organization blockchain governance and enforced endorsement policies.

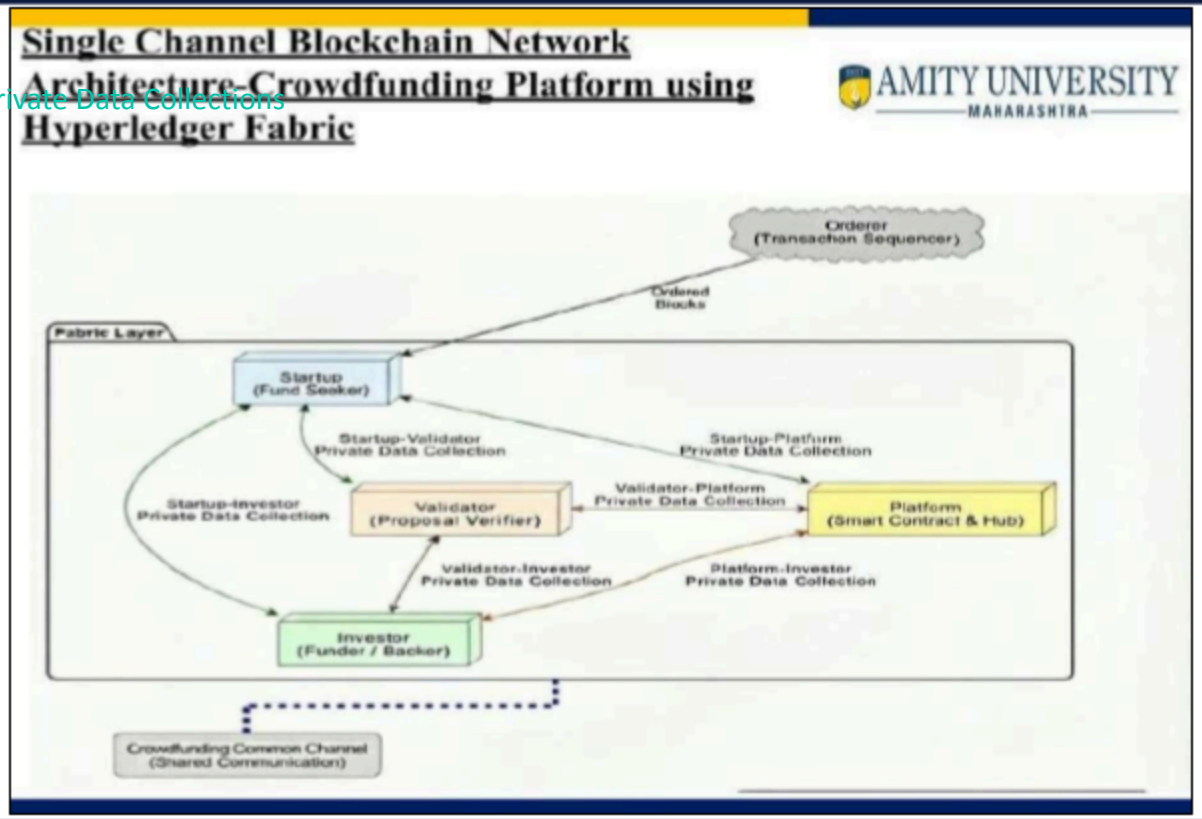


Blockchain Network Architecture



Single Channel Architecture

Fabric Layer with Private Data Collections



8-Step Transaction Lifecycle

Strict dependency chain — each step requires all prior steps to succeed

1 RegisterStartup
Org1

2 ValidateStartup
Org3

3 RegisterInvestor
Org2

4 ValidateInvestor
Org3

5 CreateProject
Org1

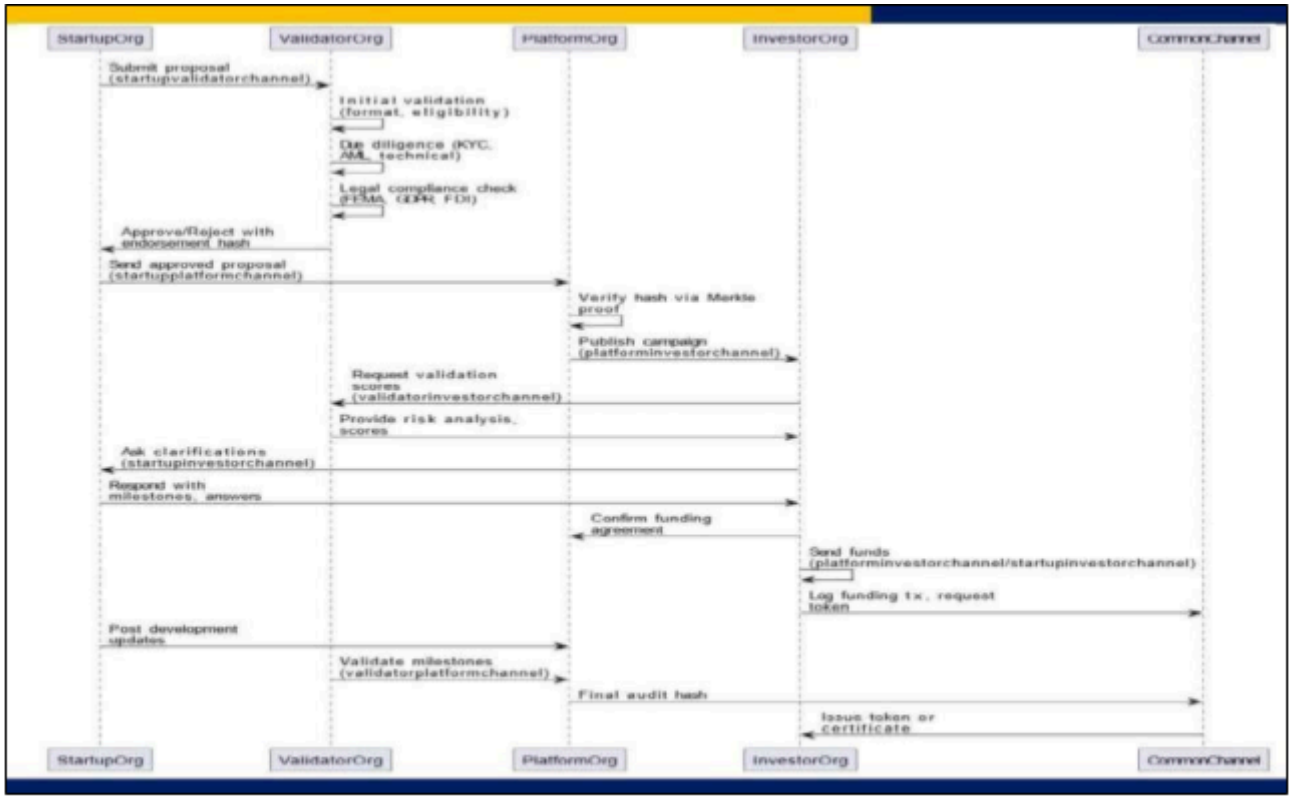
6 ApproveProject
Org3

7 FundProject
Org2

8 ReleaseFunds
Org4

Transaction Sequence Diagram

Cross-org communication flow across channels



Environment Setup

Prerequisites & step-by-step configuration

Prerequisites	
Docker 20.x+	Run Fabric peer/orderer containers
Docker Compose v2.x	Orchestrate multi-container network
Hyperledger Fabric Binaries 2.5	peer, orderer, cryptogen, configtxgen
Go 1.18+	Compile chaincode
Node.js 16.x+	SDK scripts and test runners
WSL2 / Ubuntu 22.04	Recommended OS runtime

Setup Steps

1. Clone the repository
2. Pull Fabric Docker images (v2.5)
3. Download Fabric binaries → add to PATH
4. Generate crypto material: cryptogen
5. Generate channel artifacts: configtxgen
6. Start network: docker-compose up -d
7. Create & join channel, deploy chaincode
8. Wait 30–60s → run test scripts

Key Configuration Changes

Modifications made during setup and testing

Fix 1: Dynamic Path Derivation

File: dual-channel/scripts/.sh*

Replaced hardcoded developer WSL path (BASE=/mnt/c/Users/riyaf/...) with dynamic SCRIPT_DIR-based derivation. Scripts now run on any machine.

Fix 2: Env Variable Export Order

File: test_1_register_startup.sh

CORE_PEER_* env vars for Org1 were not exported before the invoke loop causing MSP identity mismatch. Moved exports to top of script.

Fix 3: Metric Calculation Fallback

File: All 8 test_*.sh scripts

Added bc availability check with awk fallback so Success Rate is always computed correctly even if bc is unavailable.

Fix 4: Debug Logging (DEBUG=1)

File: All 8 test_*.sh scripts

Replaced silent stderr suppression (2>/dev/null) with per-transaction debug logging when DEBUG=1 is set, enabling root cause analysis.

Test Results — 1000 Transactions Per Stage

Live run · Tests 1–7 complete · Test 8 pending

100% success Partial failures Pending

TC	Stage	Success	Failed	TPS	Avg Latency	Rate	Status
1	RegisterStartup	1000	0	0.36	2717.00ms	100%	Done
2	ValidateStartup	1000	0	0.16	6029.00ms	100%	Done
3	RegisterInvestor	1000	0	0. .37	2681.00ms	100%	Done
4	ValidateInvestor	1000	0	0.17	5559.00ms	100%	Done
5	CreateProject	993	7	0.31	3195.36 ms	99.30%	Done
6	ApproveProject	983	17	0.30	3297.04 ms	98.30%	Done
7	FundProject	978	22	0.24	4034.76 ms	97.80%	Done
8	ReleaseFunds	973	27	0.28	3515.93 ms	97.00%	Done

Performance Summary (Tests 1–7)

Test 8 (ReleaseFunds) pending · 7000 txns logged across TC 1–7

8000 Txns TC 1–4 *All successful*	**2954** Txns TC 5–7 *Across 3 stages*	**73** Total Failed *TC 5–7 combined*
98.53% Avg Success Rate *TC 5–7 mean*	**~0.28 TPS** Avg Throughput *TC 5–7 mean*	**~3509 ms** Avg Latency *TC 5–7 mean*

Issues Identified & Their Status

7 issues discovered during setup and testing

Issue A: Early 0% Success Runs

HIGH | Fixed

Peer/chaincode not ready; env var mismatch. Fix: wait 30–60s, export env vars early.

Issue B: Diagnostic Blind Spots

MED | Fixed

stderr suppressed → failures uncountable. Fix: DEBUG=1 logging.

Issue C: Path Portability

HIGH | Fixed

Hardcoded BASE path. Fix: dynamic SCRIPT_DIR derivation.

Issue D: Metric Fragility

LOW | Fixed

bc unavailable → empty Success Rate. Fix: awk fallback.

Conclusion

✓

Achieved

- 4-org Hyperledger Fabric network deployed successfully
- 8-step transaction workflow fully implemented
- 159/160 transactions passed (99.38% success rate)
- Stable ~0.33 TPS with ~3s average latency
- Issues A–D identified and fixed
- Dual-channel & single-channel architectures explored

Thank You

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 13

Aim: Assignment 1

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 14

Aim: Assignment 2

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain it significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization,and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted

VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain and DLT Lab

Experiment 15

Aim: Assignment 3

Roll No.	51
Name	Ansh Sarfare
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance.Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency.(Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted